



PYTHON PROGRAMMING

*Develop a variety of applications for different platforms using
Python v3*

Compiled by John Rey Goh
johnreygoh@gmail.com

Contents

Overview of Python Programming	5
Setting Up Programming Environment	5
Getting, Installing and Setting Up Path	5
Running Python.....	7
Basic Syntax	8
Identifiers and Keywords.....	8
Lines and Indentation.....	8
Quotation and Comments	9
Python Variables.....	10
Numbers, Strings, Lists, Tuples, Dictionary	11
Type Conversion	13
Operators	14
Decision Making.....	17
Loops	18
Numbers and Number Functions.....	24
Strings and String Functions	25
Lists and List Functions.....	29
Tuples and Tuple Functions.....	31
Dictionary and Dictionary Functions.....	34
Date and Time and Date-Time Functions.....	36
User-Defined Functions.....	42
Python Modules.....	48
Python – Files I/O.....	52
Opening and Closing Files.....	53
Reading and Writing Files.....	55
File Positions	56
Renaming and Deleting Files	56
Directories in Python.....	57
File & Directory Related Methods	58
Handling Exceptions.....	61
Classes and Objects.....	67
Creating Classes	68
Creating Instance Objects.....	69
Accessing Attributes.....	69

Built-In Class Attributes	70
Destroying Objects (Garbage Collection)	71
Class Inheritance	72
Overriding Methods	73
Base Overloading Methods	74
Data Hiding	74
Regular Expressions	75
CGI Programming.....	79
Web Server Support and Configuration	80
HTTP Header	81
CGI Environment Variables.....	81
Passing Information using GET method	82
Simple URL Example – Get Method	82
Simple FORM Example – GET Method	83
Passing Information Using POST Method.....	83
Passing Checkbox Data to CGI Program	84
Passing Radio Button Data to CGI Program.....	85
Passing Text Area Data to CGI Program	86
Passing Drop Down Box Data to CGI Program.....	86
Using Cookies in CGI.....	87
Setting up Cookies.....	87
Retrieving Cookies.....	88
File Upload Example.....	88
Database Access (MySQL)	89
Database Connection	90
Creating Database Table	91
INSERT Operation.....	91
READ Operation	93
UPDATE Operation	94
DELETE Operation	94
Disconnecting Database	95
GUI Programming	95
Tkinter Programming	95
Tkinter Button.....	96
Tkinter Checkbutton	97

Tkinter Entry	99
Tkinter Frame	100
Tkinter Label	101
Tkinter Listbox	103
Tkinter Menubutton.....	104
Tkinter Menu	106
Tkinter Message.....	108
Tkinter Radiobutton	109
Tkinter Scale	111
Tkinter Scrollbar.....	113
Tkinter Text.....	114
Tkinter Toplevel	116
Tkinter Spinbox.....	118
Tkinter PanedWindow.....	119
Tkinter LabelFrame	120
Tkinter tkMessageBox.....	121
Standard Attributes.....	122
Tkinter Dimensions	122
Tkinter Colors.....	122
Tkinter Fonts	123
Tkinter Anchors.....	123
Tkinter Relief styles	124
Geometry Management.....	125
Tkinter pack() Method.....	125
Tkinter grid() Method.....	125
Tkinter place() Method.....	126

Overview of Python Programming

Python is an easy to learn, powerful programming language. It has efficient high-level data structures and a simple but effective approach to object-oriented programming. Python's elegant syntax and dynamic typing, together with its interpreted nature, make it an ideal language for scripting and rapid application development in many areas on most platforms.

The Python interpreter and the extensive standard library are freely available in source or binary form for all major platforms from the Python Web site, <https://www.python.org/>, and may be freely distributed. The same site also contains distributions of and pointers to many free third party Python modules, programs and tools, and additional documentation.

The Python interpreter is easily extended with new functions and data types implemented in C or C++ (or other languages callable from C). Python is also suitable as an extension language for customizable applications.

Python Features

- ✓ Easy-to-learn: Python has few keywords, simple structure, and a clearly defined syntax. This allows a student to pick up the language quickly.
- ✓ Easy-to-read: Python code is more clearly defined and visible to the eyes.
- ✓ Easy-to-maintain: Python's source code is fairly easy-to-maintain.
- ✓ A broad standard library: Python's bulk of the library is very portable and cross platform compatible on UNIX, Windows, and Macintosh.
- ✓ Interactive Mode: Python has support for an interactive mode, which allows interactive testing and debugging of snippets of code.
- ✓ Portable: Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- ✓ Extendable: You can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.
- ✓ Databases: Python provides interfaces to all major commercial databases.
- ✓ GUI Programming: Python supports GUI applications that can be created and ported to many system calls, libraries and windows systems, such as Windows MFC, Macintosh, and the X Window system of Unix.
- ✓ Scalable: Python provides a better structure and support for large programs than shell scripting.

Setting Up Programming Environment

Getting, Installing and Setting Up Path

Official Python development package and compiler can be found here: <https://www.python.org/downloads/>



To check for python, type

```
>python
```

```
C:\WINDOWS\system32\cmd.exe - python
Microsoft Windows [Version 10.0.17134.167]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\core360>python
Python 3.7.0 (v3.7.0:1bf9cc5093, Jun 27 2018, 04:59:51) [MSC v.1914 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Linux platform

Different flavors of Linux use different package managers for installation of new packages. On Ubuntu Linux, Python 3 is installed using the following command from the terminal.

```
$sudo apt-get install python3-minimal
```

Setting path at Unix/Linux

To add the Python directory to the path for a particular session in Unix:

1. In the csh shell: type `setenv PATH "$PATH:/usr/local/bin/python"` and press Enter.
2. In the bash shell (Linux): type `export PATH="$PATH:/usr/local/bin/python"` and press Enter.
3. In the sh or ksh shell: type `PATH="$PATH:/usr/local/bin/python"` and press Enter.

Note: `/usr/local/bin/python` is the path of the Python directory

Setting path at Windows

To add the Python directory to the path for a particular session in Windows:

1. At the command prompt: type `path %path%;C:\Python` and press Enter.

Note: C:\Python is the path of the Python directory

Running Python

There are three different ways to start Python:

1. Interactive Interpreter

You can start Python from Unix, DOS, or any other system that provides you a command-line interpreter or shell window.

Enter `python` the command line and Start coding right away in the interactive interpreter.

<code>\$python</code>	(Unix/Linux)
-----------------------	--------------

Or

<code>python%</code>	(Unix/Linux)
----------------------	--------------

<code>c:>python</code>	(windows)
---------------------------	-----------

Here is the list of all the available command line options:

Option	Description
-d	It provides debug output.
-O	It generates optimized bytecode (resulting in .pyo files).
-S	Do not run import site to look for Python paths on startup.
-v	verbose output (detailed trace on import statements).
-X	disable class-based built-in exceptions (just use strings); obsolete starting with version 1.6.
-c	cmd run Python script sent in as cmd string
file	run Python script from given file

2. Script from the Command-line

A Python script can be executed at command line by invoking the interpreter on your application, as in the following:

<code>\$python script.py</code>	(Unix/Linux)
---------------------------------	--------------

or

<code>python% script.py</code>	(Unix/Linux)
--------------------------------	--------------

or

<code>C:>python script.py</code>	(Windows/DOS)
-------------------------------------	---------------

Note: Be sure the file permission mode allows execution.

Basic Syntax

Identifiers and Keywords

A Python identifier is a name used to identify a variable, function, class, module or other object. An identifier starts with a letter A to Z or a to z or an underscore (_) followed by zero or more letters, underscores and digits (0 to 9). Python does not allow punctuation characters such as @, \$, and % within identifiers. Python is a case sensitive programming language.

Here are naming conventions for Python identifiers-

- ✓ Class names start with an uppercase letter. All other identifiers start with a lowercase letter.
- ✓ Starting an identifier with a single leading underscore indicates that the identifier is private.
- ✓ Starting an identifier with two leading underscores indicates a strong private identifier.
- ✓ If the identifier also ends with two trailing underscores, the identifier is a language-defined special name.

Reserved Words

The following list shows the Python keywords. These are reserved words and you cannot use them as constants or variables or any other identifier names. All the Python keywords contain lowercase letters only.

and	pass	if	elif
exec	break	return	is
Not	from	def	with
as	print	import	else
finally	class	try	lambda
or	global	del	yield
assert	raise	in	except
for	continue	while	

Lines and Indentation

Python does not use braces({}) to indicate blocks of code for class and function definitions or flow control. Blocks of code are denoted by line indentation, which is rigidly enforced.

The number of spaces in the indentation is variable, but all statements within the block must be indented the same amount. For example-

```
if True:
    print ("True")
else:
    print ("False")
```

However, the following block generates an error

```
If True:
    print ("Answer")
    print ("True")
else:
    print "(Answer)"
print ("False")
```

Thus, in Python all the continuous lines indented with the same number of spaces would form a block.

Statements in Python typically end with a new line. Python, however, allows the use of the line continuation character (\) to denote that the line should continue. For example:

```
total = item_one + \
        item_two + \
        item_three
```

The statements contained within the [], {}, or () brackets do not need to use the line continuation character. For example

```
days = ['Monday', 'Tuesday', 'Wednesday',
        'Thursday', 'Friday']
```

Quotation and Comments

Python accepts single ('), double (") and triple (""" or """) quotes to denote string literals, as long as the same type of quote starts and ends the string.

The triple quotes are used to span the string across multiple lines. For example, all the following are legal

```
word = 'word'
sentence = "This is a sentence."
paragraph = """This is a paragraph. It is
made up of multiple lines and sentences."""
```

Comments in Python

A hash sign (#) that is not inside a string literal is the beginning of a comment. All characters after the #, up to the end of the physical line, are part of the comment and the Python interpreter ignores them.

```
#!/usr/bin/python3
```

```
# First comment
print("Hello, Python!") # second comment
```

This produces the following result
Hello, Python!

You can type a comment on the same line after a statement or expression

```
name = "Madisetti" # This is again comment
```

Python does not have multiple-line commenting feature. You have to comment each line individually as follows-

```
# This is a comment.
# This is a comment, too.
# This is a comment, too.
# I said that already.
```

Waiting for the User

The following line of the program displays the prompt and the statement saying “Press the enter key to exit”, and then waits for the user to take action –

```
#!/usr/bin/python3
input("\n\nPress the enter key to exit.")
```

Multiple Statements on a Single Line

The semicolon (;) allows multiple statements on a single line given that no statement starts a new code block. Here is a sample snippet using the semicolon

```
import sys; x = 'foo'; sys.stdout.write(x + '\n')
```

Python Variables

Assigning Values to Variables

Python variables do not need explicit declaration to reserve memory space. The declaration happens automatically when you assign a value to a variable. The equal sign (=) is used to assign values to variables.

The operand to the left of the = operator is the name of the variable and the operand to the right of the = operator is the value stored in the variable. For example-

```
#!/usr/bin/python3
counter = 100          # An integer assignment
miles = 1000.0         # A floating point
name = "John"          # A string
print (counter)
print (miles)
print (name)
```

Multiple Assignment

Python allows you to assign a single value to several variables simultaneously.

```
a = b = c = 1
```

Here, an integer object is created with the value 1, and all the three variables are assigned to the same memory location. You can also assign multiple objects to multiple variables.

```
a, b, c = 1, 2, "john"
```

Here, two integer objects with values 1 and 2 are assigned to the variables a and b respectively, and one string object with the value "john" is assigned to the variable c.

Standard Data Types

The data stored in memory can be of many types. For example, a person's age is stored as a numeric value and his or her address is stored as alphanumeric characters. Python has various standard data types that are used to define the operations possible on them and the storage method for each of them.

Python has five standard data types-**Numbers, String, List, Tuple**, and **Dictionary**

Numbers, Strings, Lists, Tuples, Dictionary

Python Numbers

Number data types store numeric values. Number objects are created when you assign a value to them. For example

```
var1 = 1
var2 = 10
```

You can also delete the reference to a number object by using the del statement. The syntax of the del statement is –

```
del var1[,var2[,var3[....,varN]]]]
```

You can delete a single object or multiple objects by using the del statement.

```
del var
del var_a, var_b
```

Python supports three different numerical types –

- ✓ int (signed integers)
- ✓ float (floating point real values)
- ✓ complex (complex numbers)

All integers in Python 3 are represented as long integers. Hence, there is no separate number type as long.

Here are some examples of numbers

Int	float	complex
10	0.0	3.14j
100	15.20	45.j
-786	-21.9	9.322e-36j

Python Strings

Strings in Python are identified as a contiguous set of characters represented in the quotation marks. Python allows either pair of single or double quotes. Subsets of strings can be taken using the slice operator ([] and [:]) with indexes starting at 0 in the beginning of the string and working their way from -1 to the end. The plus (+) sign is the string concatenation operator and the asterisk (*) is the repetition operator.

```
#!/usr/bin/python3
str = 'Hello World!'
print (str)           # Prints complete string
print (str[0])        # Prints first character of the string
print (str[2:5])      # Prints characters starting from 3rd to 5th
print (str[2:])       # Prints string starting from 3rd character
print (str * 2)       # Prints string two times
print (str + "TEST")  # Prints concatenated string
```

Python Lists

Lists are the most versatile of Python's compound data types. A list contains items separated by commas and enclosed within square brackets ([]). To some extent, lists are similar to arrays in C. One of the differences between them is that all the items belonging to a list can be of different data type.

The values stored in a list can be accessed using the slice operator ([] and [:]) with indexes starting at 0 in the beginning of the list and working their way to end -1. The plus (+) sign is the list concatenation operator, and the asterisk (*) is the repetition operator.

```
#!/usr/bin/python3
list = [ 'abcd', 786 , 2.23, 'john', 70.2 ]
tinylist = [123, 'john']
print (list)          # Prints complete list
print (list[0])        # Prints first element of the list
print (list[1:3])      # Prints elements starting from 2nd till 3rd
print (list[2:])       # Prints elements starting from 3rd element
print (tinylist * 2)   # Prints list two times
print (list + tinylist) # Prints concatenated lists
```

This produces the following result-

```
['abcd', 786, 2.23, 'john', 70.2000000000000003]
abcd
[786, 2.23]
[2.23, 'john', 70.2000000000000003]
[123, 'john', 123, 'john']
['abcd', 786, 2.23, 'john', 70.2000000000000003, 123, 'john']
```

Python Tuples

A tuple is another sequence data type that is similar to the list. A tuple consists of a number of values separated by commas. Unlike lists, however, tuples are enclosed within parenthesis.

The main difference between lists and tuples is- Lists are enclosed in brackets ([]) and their elements and size can be changed, while tuples are enclosed in parentheses (()) and cannot be updated. Tuples can be thought of as read-only lists.

```
#!/usr/bin/python3
tuple = ( 'abcd', 786 , 2.23, 'john', 70.2 )
tinytuple = (123, 'john')
print (tuple) # Prints complete tuple
print (tuple[0]) # Prints first element of the tuple
print (tuple[1:3]) # Prints elements starting from 2nd till 3rd
print (tuple[2:]) # Prints elements starting from 3rd element
print (tinytuple * 2) # Prints tuple two times
print (tuple + tinytuple) # Prints concatenated tuple
```

This produces the following result-

```
('abcd', 786, 2.23, 'john', 70.2000000000000003)
abcd
(786, 2.23)
(2.23, 'john', 70.2000000000000003)
(123, 'john', 123, 'john')
('abcd', 786, 2.23, 'john', 70.2000000000000003, 123, 'john')
```

Python Dictionary

Python's dictionaries are kind of hash-table type. They work like associative arrays or hashes found in Perl and consist of key-value pairs. A dictionary key can be almost any Python type, but are usually numbers or strings. Values, on the other hand, can be any arbitrary Python object.

Dictionaries are enclosed by curly braces ({ }) and values can be assigned and accessed using square braces ([]).

```
#!/usr/bin/python3
dict = {}
dict['one'] = "This is one"
dict[2] = "This is two"
tinydict = {'name': 'john', 'code': 6734, 'dept': 'sales'}
print (dict['one'])           # Prints value for 'one' key
print (dict[2])              # Prints value for 2 key
print (tinydict)             # Prints complete dictionary
print (tinydict.keys())      # Prints all the keys
print (tinydict.values())    # Prints all the values
```

This produces the following result

```
This is one
This is two
{'dept': 'sales', 'code': 6734, 'name': 'john'}
['dept', 'code', 'name']
['sales', 6734, 'john']
```

Dictionaries have no concept of order among the elements. It is incorrect to say that the elements are "out of order"; they are simply unordered.

Type Conversion

Data Type Conversion

Sometimes, you may need to perform conversions between the built-in types. To convert between types, you simply use the type-name as a function.

Function	Description
int(x [,base])	Converts x to an integer. The base specifies the base if x is a string.
float(x)	Converts x to a floating-point number.
str(x)	Converts object x to a string representation.
repr(x)	Converts object x to an expression string.
eval(str)	Evaluates a string and returns an object.
tuple(s)	Converts s to a tuple.
list(s)	Converts s to a list.
set(s)	Converts s to a set.
dict(d)	Creates a dictionary. d must be a sequence of (key,value) tuples.
frozenset(s)	Converts s to a frozen set.
chr(x)	Converts an integer to a character.
unichr(x)	Converts an integer to a Unicode character.
ord(x)	Converts a single character to its integer value.
hex(x)	Converts an integer to a hexadecimal string.
oct(x)	Converts an integer to an octal string.

Operators

Types of Operator

Python language supports the following types of operators-

- ✓ Arithmetic Operators
- ✓ Comparison (Relational) Operators
- ✓ Assignment Operators
- ✓ Logical Operators
- ✓ Bitwise Operators
- ✓ Membership Operators
- ✓ Identity Operators

Arithmetic, Comparison, Assignment, Logical, Membership, and Identity Operators

Python Arithmetic Operators

Assume variable **a** holds the value 10 and variable **b** holds the value 21, then

Operator	Description	Example
+ Addition	Adds values on either side of the operator.	$a + b = 31$
- Subtraction	Subtracts right hand operand from left hand operand.	$a - b = -11$
* Multiplication	Multiplies values on either side of the operator	$a * b = 210$
/ Division	Divides left hand operand by right hand operand	$b / a = 2.1$
% Modulus	Divides left hand operand by right hand operand and returns remainder	$b \% a = 1$
** Exponent	Performs exponential (power) calculation on operators	$a ** b = 10$ to the power 20
//	Floor Division - The division of operands where the result is the quotient in which the digits after the decimal point are removed.	$9 // 2 = 4$ and $9.0 // 2.0 = 4.0$

Python Comparison Operators

These operators compare the values on either side of them and decide the relation among them. They are also called Relational operators.

Assume variable **a** holds the value 10 and variable **b** holds the value 20, then

Operator	Description
==	If the values of two operands are equal, then the condition becomes true. ($a == b$) is not true.
!=	If values of two operands are not equal, then condition becomes true. ($a != b$) is true.
>	If the value of left operand is greater than the value of right operand, then condition becomes true. ($a > b$) is not true.
<	If the value of left operand is less than the value of right operand, then condition becomes true. ($a < b$) is true.
>=	If the value of left operand is greater than or equal to the value of right operand, then condition becomes true. ($a >= b$) is not true.
<=	If the value of left operand is less than or equal to the value of right operand, then condition becomes true. ($a <= b$) is true.

Python Assignment Operators

Assume variable a holds 10 and variable b holds 20, then

Operator	Description
=	Assigns values from right side operands to left side operand $c = a + b$ assigns value of $a + b$ into c
+= Add AND	It adds right operand to the left operand and assign the result to left operand $c += a$ is equivalent to $c = c + a$
-= Subtract AND	It subtracts right operand from the left operand and assign the result to left operand $c -= a$ is equivalent to $c = c - a$
*= Multiply AND	It multiplies right operand with the left operand and assign the result to left operand $c *= a$ is equivalent to $c = c * a$
/= Divide AND	It divides left operand with the right operand and assign the result to left operand $c /= a$ is equivalent to $c = c / a$
%= Modulus AND	It takes modulus using two operands and assign the result to left operand $c \% = a$ is equivalent to $c = c \% a$
**= Exponent AND	Performs exponential (power) calculation on operators and assign value to the left operand $c ** = a$ is equivalent to $c = c ** a$
//= Floor Division	It performs floor division on operators and assign value to the left operand $c //= a$ is equivalent to $c = c // a$

Python Logical Operators

The following logical operators are supported by Python language. Assume variable a holds True and variable b holds False then:

Operator	Description
and Logical AND	If both the operands are true then condition becomes true. (a and b) is False.
or Logical OR	If any of the two operands are non-zero then condition becomes true. (a or b) is True.
not Logical NOT	Used to reverse the logical state of its operand. Not(a and b) is True.

Python Membership Operators

Python's membership operators test for membership in a sequence, such as strings, lists, or tuples.

Operator	Description
in	Evaluates to true, if it finds a variable in the specified sequence and false otherwise. x in y , here in results in a 1 if x is a member of sequence y .
not in	Evaluates to true, if it does not find a variable in the specified sequence and false otherwise. x not in y , here not in results in a 1 if x is not a member of sequence y .

Example

```
#!/usr/bin/python3
```

```
a = 10
```

```
b = 20
```

```
list = [1, 2, 3, 4, 5]
```

```
if (a in list):
```

```
    print ("Line 1 - a is available in the given list")
```

```
else:
```

```
    print ("Line 1 - a is not available in the given list")
```

```

if ( b not in list ):
    print ("Line 2 - b is not available in the given list")
else:
    print ("Line 2 - b is available in the given list")
c=b/a

if ( c in list ):
    print ("Line 3 - a is available in the given list")
else:
    print ("Line 3 - a is not available in the given list")

```

Python Identity Operators

Identity operators compare the memory locations of two objects. There are two Identity operators as explained below:

Operator	Description
is	Evaluates to true if the variables on either side of the operator point to the same object and false otherwise. x is y, here is results in 1 if id(x) equals id(y).
is not	Evaluates to false if the variables on either side of the operator point to the same object and true otherwise. x is not y, here is not results in 1 if id(x) is not equal to id(y).

Example

```

#!/usr/bin/python3
a = 20
b = 20
print ('Line 1','a=',a,':',id(a), 'b=',b,':',id(b))

if ( a is b ):
    print ("Line 2 - a and b have same identity")
else:
    print ("Line 2 - a and b do not have same identity")

if ( id(a) == id(b) ):
    print ("Line 3 - a and b have same identity")
else:
    print ("Line 3 - a and b do not have same identity")

b = 30
print ('Line 4','a=',a,':',id(a), 'b=',b,':',id(b))

if ( a is not b ):
    print ("Line 5 - a and b do not have same identity")
else:
    print ("Line 5 - a and b have same identity")

```

Decision Making

If Statement

The IF statement is similar to that of other languages. The if statement contains a logical expression using which the data is compared and a decision is made based on the result of the comparison.

```
#!/usr/bin/python3
var1 = 100
if var1:
    print ("1 - Got a true expression value")
    print (var1)

var2 = 0
if var2:
    print ("2 - Got a true expression value")
    print (var2)

print ("Good bye!")
```

If...else Statement

An else statement can be combined with an if statement. An else statement contains a block of code that executes if the conditional expression in the if statement resolves to 0 or a FALSE value.

The else statement is an optional statement and there could be at the most only one else statement following if.

```
#!/usr/bin/python3
amount=int(input("Enter amount: "))
if amount<1000:
    discount=amount*0.05
    print ("Discount",discount)
else:
    discount=amount*0.10
    print ("Discount",discount)

print ("Net payable:",amount-discount)
```

The elif Statement

The elif statement allows you to check multiple expressions for TRUE and execute a block of code as soon as one of the conditions evaluates to TRUE.

Similar to the else, the elif statement is optional. However, unlike else, for which there can be at the most one statement, there can be an arbitrary number of elif statements following an if.

```
#!/usr/bin/python3
amount=int(input("Enter amount: "))

if amount<1000:
```

```
discount=amount*0.05
print ("Discount",discount)

elif amount<5000:
    discount=amount*0.10
    print ("Discount",discount)

else:
    discount=amount*0.15
    print ("Discount",discount)

print ("Net payable:",amount-discount)
```

Nested IF Statements

There may be a situation when you want to check for another condition after a condition resolves to true. In such a situation, you can use the nested if construct.

In a nested if construct, you can have an if...elif...else construct inside another if...elif...else construct.

```
if expression1:
    statement(s)
    if expression2:
        statement(s)
    elif expression3:
        statement(s)
    else
        statement(s)
elif expression4:
    statement(s)
else:
    statement(s)
```

Single Statement Suites

If the suite of an if clause consists only of a single line, it may go on the same line as the header statement.

Example:

```
#!/usr/bin/python3
var = 100
if ( var == 100 ) : print ("Value of expression is 100")
print ("Good bye!")
```

Loops

While Loop

A while loop statement in Python programming language repeatedly executes a target statement as long as a given condition is true.

```
#!/usr/bin/python3
count = 0
```

```
while (count < 9):
    print ('The count is:', count)
    count = count + 1

print ("Good bye!")
```

Using else Statement with Loops

Python supports having an else statement associated with a loop statement. If the else statement is used with a for loop, the else statement is executed when the loop has exhausted iterating the list. If the else statement is used with a while loop, the else statement is executed when the condition becomes false.

```
#!/usr/bin/python3
count = 0

while count < 5:
    print (count, " is less than 5")
    count = count + 1

else:
    print (count, " is not less than 5")
```

For Loop

The for statement in Python has the ability to iterate over the items of any sequence, such as a list or a string.

The range() function

The built-in function range() is the right function to iterate over a sequence of numbers. It generates an iterator of arithmetic progressions.

```
>>> range(5)
range(0, 5)
```

```
>>> list(range(5))
[0, 1, 2, 3, 4]
```

range() generates an iterator to progress integers starting with 0 upto n-1. To obtain a list object of the sequence, it is typecasted to list(). Now this list can be iterated using the for statement.

```
>>> for var in list(range(5)):
    print (var)
```

Another example:

```
#!/usr/bin/python3

for letter in 'Python': # traversal of a string sequence
    print ('Current Letter :', letter)

print()
fruits = ['banana', 'apple', 'mango']
```

```
for fruit in fruits: # traversal of List sequence
    print ('Current fruit :', fruit)

print ("Good bye!")
```

Iterating by Sequence Index

An alternative way of iterating through each item is by index offset into the sequence itself.

```
#!/usr/bin/python3
fruits = ['banana', 'apple', 'mango']

for index in range(len(fruits)):
    print ('Current fruit :', fruits[index])

print ("Good bye!")
```

Using else Statement with Loops

Python supports having an else statement associated with a loop statement.

- If the else statement is used with a for loop, the else block is executed only if for loops terminates normally (and not by encountering break statement).
- If the else statement is used with a while loop, the else statement is executed when the condition becomes false.

Example:

```
#!/usr/bin/python3
numbers=[11,33,55,39,55,75,37,21,23,41,13]

for num in numbers:
    if num%2==0:
        print ('the list contains an even number')
        break

else:
    print ('the list doesnot contain even number')
```

Nested loops

Python programming language allows the use of one loop inside another loop. The following section shows a few examples to illustrate the concept.

Syntax

```
for iterating_var in sequence:
    for iterating_var in sequence:
        statements(s)
    statements(s)
```

The syntax for a nested while loop statement in Python programming language is as follows

```
while expression:  
    while expression:  
        statement(s)  
        statement(s)
```

A final note on loop nesting is that you can put any type of loop inside any other type of loop. For example a for loop can be inside a while loop or vice versa.

Example

The following program uses a nested-for loop to display multiplication tables from 1-10.

```
#!/usr/bin/python3  
import sys  
  
for i in range(1,11):  
    for j in range(1,11):  
        k=i*j  
        print (k, end=' ')  
    print()
```

The print() function inner loop has end=' ' which appends a space instead of default newline. Hence, the numbers will appear in one row. Last print() will be executed at the end of inner for loop.

Loop Control Statements

The Loop control statements change the execution from its normal sequence. When the execution leaves a scope, all automatic objects that were created in that scope are destroyed.

Python supports the following control statements:

Control Statement	Description
break statement	Terminates the loop statement and transfers execution to the statement immediately following the loop.
continue statement	Causes the loop to skip the remainder of its body and immediately retest its condition prior to reiterating.
pass statement	The pass statement in Python is used when a statement is required syntactically but you do not want any command or code to execute.

Break Statement

The break statement is used for premature termination of the current loop. After abandoning the loop, execution at the next statement is resumed, just like the traditional break statement in C.

The most common use of break is when some external condition is triggered requiring a hasty exit from a loop. The break statement can be used in both while and for loops. If you are using nested loops, the break statement stops the execution of the innermost loop and starts executing the next line of the code after the block.

Example

```
#!/usr/bin/python3
```

```
for letter in 'Python':    # First Example
    if letter == 'h':
        break
    print ('Current Letter :', letter)

var = 10 # Second Example
while var > 0:
    print ('Current variable value :', var)
    var = var -1
    if var == 5:
        break
print ("Good bye!")
```

The following program demonstrates the use of break in a for loop iterating over a list. User inputs a number, which is searched in the list. If it is found, then the loop terminates with the 'found' message.

```
#!/usr/bin/python3
no=int(input('any number: '))
numbers=[11,33,55,39,55,75,37,21,23,41,13]

for num in numbers:
    if num==no:
        print ('number found in list')
        break
else:
    print ('number not found in list')
```

Continue Statement

The continue statement in Python returns the control to the beginning of the current loop. When encountered, the loop starts next iteration without executing the remaining statements in the current iteration. The continue statement can be used in both while and for loops.

Example

```
#!/usr/bin/python3

for letter in 'Python':    # First Example
    if letter == 'h':
        continue
    print ('Current Letter :', letter)

var = 10 # Second Example
while var > 0:
    var = var -1
    if var == 5:
        continue
    print ('Current variable value :', var)
print ("Good bye!")
```

When the above code is executed, it produces the following result

```
Current Letter : P
Current Letter : y
Current Letter : t
Current Letter : o
Current Letter : n
Current variable value : 9
Current variable value : 8
Current variable value : 7
Current variable value : 6
Current variable value : 4
Current variable value : 3
Current variable value : 2
Current variable value : 1
Current variable value : 0
Good bye!
```

Pass Statement

It is used when a statement is required syntactically but you do not want any command or code to execute. The pass statement is a null operation; nothing happens when it executes. The pass statement is also useful in places where your code will eventually go, but has not been written yet i.e. in stubs).

Example

```
#!/usr/bin/python3

for letter in 'Python':
    if letter == 'h':
        pass
    print ('This is pass block')
    print ('Current Letter :', letter)
print ("Good bye!")
```

When the above code is executed, it produces the following result

```
Current Letter : P
Current Letter : y
Current Letter : t
This is pass block
Current Letter : h
Current Letter : o
Current Letter : n
Good bye!
```

Numbers and Number Functions

Number data types store numeric values. They are immutable data types. This means, changing the value of a number data type results in a newly allocated object.

Number objects are created when you assign a value to them.

```
var1 = 1
var2 = 10
```

You can also delete the reference to a number object by using the del statement.

```
del var1[,var2[,var3[....,varN]]]]
```

You can delete a single object or multiple objects by using the del statement.

```
del var
del var_a, var_b
```

Python supports different numerical types-

- **int (signed integers):** They are often called just integers or ints. They are positive or negative whole numbers with no decimal point. Integers in Python 3 are of unlimited size. Python 2 has two integer types - int and long. There is no 'long integer' in Python 3 anymore.
- **float (floating point real values):** Also called floats, they represent real numbers and are written with a decimal point dividing the integer and the fractional parts. Floats may also be in scientific notation, with E or e indicating the power of 10 ($2.5e2 = 2.5 \times 10^2 = 250$).
- **complex (complex numbers):** are of the form $a + bJ$, where a and b are floats and J (or j) represents the square root of -1 (which is an imaginary number). The real part of the number is a, and the imaginary part is b. Complex numbers are not used much in Python programming.

It is possible to represent an integer in hexa-decimal or octal form.

```
>>> number = 0xA0F #Hexa-decimal
>>> number
2575
>>> number=0o37 #Octal
>>> number
31
```

Number Type Conversion

Python converts numbers internally in an expression containing mixed types to a common type for evaluation.

- `int(x)` to convert x to a plain integer.
- `long(x)` to convert x to a long integer.
- `float(x)` to convert x to a floating-point number.
- `complex(x)` to convert x to a complex number with real part x and imaginary part zero.
- `complex(x, y)` to convert x and y to a complex number with real part x and imaginary part y. x and y are numeric expressions.

Mathematical Functions

Python includes the following functions that perform mathematical calculations.

Function	Returns (Description)
<code>abs(x)</code>	The absolute value of x: the (positive) distance between x and zero.
<code>ceil(x)</code>	The ceiling of x: the smallest integer not less than x.

<code>cmp(x, y)</code>	-1 if $x < y$, 0 if $x == y$, or 1 if $x > y$. Depreciated in Python 3; Instead use <code>return (x>y)-(x<y)</code> .
<code>exp(x)</code>	The exponential of x : e^x
<code>fabs(x)</code>	The absolute value of x .
<code>floor(x)</code>	The floor of x : the largest integer not greater than x .
<code>log(x)</code>	The natural logarithm of x , for $x > 0$.
<code>log10(x)</code>	The base-10 logarithm of x for $x > 0$.
<code>max(x1, x2,...)</code>	The largest of its arguments: the value closest to positive infinity.
<code>min(x1, x2,...)</code>	The smallest of its arguments: the value closest to negative infinity.
<code>modf(x)</code>	The fractional and integer parts of x in a two-item tuple. Both parts have the same sign as x . The integer part is returned as a float.
<code>pow(x, y)</code>	The value of $x^{**}y$.
<code>round(x [,n])</code>	x rounded to n digits from the decimal point. Python rounds away from zero as a tie-breaker: <code>round(0.5)</code> is 1.0 and <code>round(-0.5)</code> is -1.0.
<code>sqrt(x)</code>	The square root of x for $x > 0$.

Random Number Functions

Random numbers are used for games, simulations, testing, security, and privacy applications. Python includes the following functions that are commonly used.

Function	Description
<code>choice(seq)</code>	A random item from a list, tuple, or string.
<code>randrange ([start,] stop [,step])</code>	A randomly selected element from <code>range(start, stop, step)</code> .
<code>random()</code>	A random float r , such that 0 is less than or equal to r and r is less than 1.
<code>seed([x])</code>	Sets the integer starting value used in generating random numbers. Call this function before calling any other random module function. Returns None.
<code>shuffle(lst)</code>	Randomizes the items of a list in place. Returns None.
<code>uniform(x, y)</code>	A random float r , such that x is less than or equal to r and r is less than y .

Strings and String Functions

Accessing Values in Strings

Python does not support a character type; these are treated as strings of length one, thus also considered a substring. To access substrings, use the square brackets for slicing along with the index or indices to obtain your substring.

```
#!/usr/bin/python3
var1 = 'Hello World!'
var2 = "Python Programming"
print ("var1[0]: ", var1[0])
print ("var2[1:5]: ", var2[1:5])
```

When the above code is executed, it produces the following result

```
var1[0]: H
var2[1:5]: ytho
```

Updating Strings

You can "update" an existing string by (re)assigning a variable to another string. The new value can be related to its previous value or to a completely different string altogether.

```
#!/usr/bin/python3
var1 = 'Hello World!'
```

```
print ("Updated String :- ", var1[:6] + 'Python')
```

When the above code is executed, it produces the following result-

```
Updated String :- Hello Python
```

Escape Characters

Following is a list of escape or non-printable characters that can be represented with backslash notation.

An escape character gets interpreted; in a single quoted as well as double quoted strings.

Backslash notation	Hexadecimal character	Description
a	0x07	Bell or alert
b	0x08	Backspace
\cx		Control-x
\C-x		Control-x
\e	0x1b	Escape
\f	0x0c	Form feed
\M-\C-x		Meta-Control-x
\n	0x0a	Newline
\nnn		Octal notation, where n is in the range 0-7
\r	0x0d	Carriage return
\s	0x20	Space
\t	0x09	Tab
\v	0x0b	Vertical tab
\x		Character x
\xnn		Hexadecimal notation, where n is in the range 0-9, a-f, or A-F

String Special Operators

Assume string variable a holds 'Hello' and variable b holds 'Python':

Operator	Description
+	Concatenation - Adds values on either side of the operator a + b will give HelloPython
*	Repetition - Creates new strings, concatenating multiple copies of the same string a*2 will give - HelloHello
[]	Slice - Gives the character from the given index a[1] will give e
[:]	Range Slice - Gives the characters from the given range a[1:4] will give ell
in	Membership - Returns true if a character exists in the given string H in a will give 1
not in	Membership - Returns true if a character does not exist in the given string M not in a will give 1
r/R	Raw String - Suppresses actual meaning of Escape characters. The syntax for raw strings is exactly the same as for normal strings with the exception of the raw string operator, the letter "r," which precedes the quotation marks. The "r" can be lowercase (r) or uppercase (R) and must be placed immediately preceding the first quote mark. print r'\n' prints \n and print R'\n' prints \n

String Formatting Operator

One of Python's coolest features is the string format operator %. This operator is unique to strings and makes up for the lack of having functions from C's printf() family.

```
#!/usr/bin/python3
print ("My name is %s and weight is %d kg!" % ('Zara', 21))
```

When the above code is executed, it produces the following result –

```
My name is Zara and weight is 21 kg!
```

Format Symbol Conversion

%c	character
%s	string conversion via str() prior to formatting
%i	signed decimal integer
%d	signed decimal integer
%u	unsigned decimal integer
%o	octal integer
%x	hexadecimal integer (lowercase letters)
%X	hexadecimal integer (UPPERcase letters)
%e	exponential notation (with lowercase 'e')
%E	exponential notation (with UPPERcase 'E')
%f	floating point real number
%g	the shorter of %f and %e
%G	the shorter of %f and %E

Triple Quotes

Python's triple quotes comes to the rescue by allowing strings to span multiple lines, including verbatim NEWLINES, TABs, and any other special characters.

The syntax for triple quotes consists of three consecutive single or double quotes.

```
#!/usr/bin/python3
para_str = """this is a long string that is made up of
several lines and non-printable characters such as
TAB ( \t ) and they will show up that way when displayed.
NEWLINES within the string, whether explicitly given like
this within the brackets [ \n ], or just a NEWLINE within
the variable assignment will also show up.
"""
print (para_str)
```

When the above code is executed, it produces the following result. Note how every single special character has been converted to its printed form, right down to the last NEWLINE at the end of the string between the "up." and closing triple quotes. Also note that NEWLINES occur either with an explicit carriage return at the end of a line or its escape code (\n) –

```
this is a long string that is made up of
several lines and non-printable characters such as
TAB ( ) and they will show up that way when displayed.
NEWLINES within the string, whether explicitly given like
this within the brackets [
], or just a NEWLINE within
```

the variable assignment will also show up.

Raw strings do not treat the backslash as a special character at all. Every character you put into a raw string stays the way you wrote it-

```
#!/usr/bin/python3
print('C:\\nowhere')
```

When the above code is executed, it produces the following result-

```
C:\nowhere
```

Now let us make use of raw string. We would put expression in r'expression' as follows-

```
#!/usr/bin/python3
print(r'C:\\nowhere')
```

When the above code is executed, it produces the following result-

```
C:\\nowhere
```

Built-in String Methods

Methods

capitalize()
center(width, fillchar)

count(str, beg= 0,end=len(string))

decode(encoding='UTF-8',errors='strict')

encode(encoding='UTF-8',errors='strict')

endswith(suffix, beg=0, end=len(string))

expandtabs(tabsize=8)

find(str, beg=0 end=len(string))

index(str, beg=0, end=len(string))
isalnum()

isalpha()

isdigit()
islower()

isnumeric()

isspace()

Description

Capitalizes first letter of string
Returns a string padded with fillchar with the original string centered to a total of width columns.
Counts how many times str occurs in string or in a substring of string if starting index beg and ending index end are given.
Decodes the string using the codec registered for encoding. encoding defaults to the default string encoding.
Returns encoded string version of string; on error, default is to raise a ValueError unless errors is given with 'ignore' or 'replace'.
Determines if string or a substring of string (if starting index beg and ending index end are given) ends with suffix; returns true if so and false otherwise.
Expands tabs in string to multiple spaces; defaults to 8 spaces per tab if tabsize not provided.
Determine if str occurs in string or in a substring of string if starting index beg and ending index end are given returns index if found and -1 otherwise.
Same as find(), but raises an exception if str not found.
Returns true if string has at least 1 character and all characters are alphanumeric and false otherwise.
Returns true if string has at least 1 character and all characters are alphabetic and false otherwise.
Returns true if the string contains only digits and false otherwise.
Returns true if string has at least 1 cased character and all cased characters are in lowercase and false otherwise.
Returns true if a unicode string contains only numeric characters and false otherwise.
Returns true if string contains only whitespace characters and false otherwise.

<code>istitle()</code>	Returns true if string is properly "titlecased" and false otherwise.
<code>isupper()</code>	Returns true if string has at least one cased character and all cased characters are in uppercase and false otherwise.
<code>join(seq)</code>	Merges (concatenates) the string representations of elements in sequence <code>seq</code> into a string, with separator string.
<code>len(string)</code>	Returns the length of the string
<code>ljust(width[, fillchar])</code>	Returns a space-padded string with the original string left-justified to a total of width columns.
<code>lower()</code>	Converts all uppercase letters in string to lowercase.
<code>lstrip()</code>	Removes all leading whitespace in string.
<code>maketrans()</code>	Returns a translation table to be used in translate function.
<code>max(str)</code>	Returns the max alphabetical character from the string <code>str</code> .
<code>min(str)</code>	Returns the min alphabetical character from the string <code>str</code> .
<code>replace(old, new [, max])</code>	Replaces all occurrences of <code>old</code> in string with <code>new</code> or at most <code>max</code> occurrences if <code>max</code> given.
<code>rfind(str, beg=0, end=len(string))</code>	Same as <code>find()</code> , but search backwards in string.
<code>rindex(str, beg=0, end=len(string))</code>	Same as <code>index()</code> , but search backwards in string.
<code>rjust(width[, fillchar])</code>	Returns a space-padded string with the original string right-justified to a total of width columns.
<code>rstrip()</code>	Removes all trailing whitespace of string.
<code>split(str="", num=string.count(str))</code>	Splits string according to delimiter <code>str</code> (space if not provided) and returns list of substrings; split into at most <code>num</code> substrings if given.
<code>splitlines(num=string.count('\n'))</code>	Splits string at all (or <code>num</code>) NEWLINEs and returns a list of each line with NEWLINEs removed.
<code>startswith(str, beg=0, end=len(string))</code>	Determines if string or a substring of string (if starting index <code>beg</code> and ending index <code>end</code> are given) starts with substring <code>str</code> ; returns true if so and false otherwise.
<code>strip([chars])</code>	Performs both <code>lstrip()</code> and <code>rstrip()</code> on string
<code>swapcase()</code>	Inverts case for all letters in string.
<code>title()</code>	Returns "title cased" version of string, that is, all words begin with uppercase and the rest are lowercase.
<code>translate(table, deletechars="")</code>	Translates string according to translation table <code>str(256 chars)</code> , removing those in the <code>del</code> string.
<code>upper()</code>	Converts lowercase letters in string to uppercase.
<code>zfill (width)</code>	Returns original string left padded with zeros to a total of width characters; intended for numbers, <code>zfill()</code> retains any sign given (less one zero).
<code>isdecimal()</code>	Returns true if a unicode string contains only decimal characters and false otherwise.

Lists and List Functions

The list is the most versatile datatype available in Python, which can be written as a list of comma-separated values (items) between square brackets. Important thing about a list is that the items in a list need not be of the same type.

Creating a list is as simple as putting different comma-separated values between square brackets. For example

```
list1 = ['physics', 'chemistry', 1997, 2000];
```

```
list2 = [1, 2, 3, 4, 5 ];  
list3 = ["a", "b", "c", "d"];
```

Similar to string indices, list indices start at 0, and lists can be sliced, concatenated and so on.

Accessing Values in Lists

To access values in lists, use the square brackets for slicing along with the index or indices to obtain value available at that index. For example-

```
#!/usr/bin/python3  
list1 = ['physics', 'chemistry', 1997, 2000]  
list2 = [1, 2, 3, 4, 5, 6, 7 ]  
print ("list1[0]: ", list1[0])  
print ("list2[1:5]: ", list2[1:5])
```

When the above code is executed, it produces the following result –

```
list1[0]: physics  
list2[1:5]: [2, 3, 4, 5]
```

Updating Lists

You can update single or multiple elements of lists by giving the slice on the left-hand side of the assignment operator, and you can add to elements in a list with the `append()` method. For example-

```
#!/usr/bin/python3  
list = ['physics', 'chemistry', 1997, 2000]  
print ("Value available at index 2 : ", list[2])  
list[2] = 2001  
print ("New value available at index 2 : ", list[2])
```

Note: The `append()` method is discussed in the subsequent section.

When the above code is executed, it produces the following result –

```
Value available at index 2 :  
1997  
New value available at index 2 :  
2001
```

Delete List Elements

To remove a list element, you can use either the `del` statement if you know exactly which element(s) you are deleting. You can use the `remove()` method if you do not know exactly which items to delete. For example-

```
#!/usr/bin/python3  
list = ['physics', 'chemistry', 1997, 2000]  
print (list)  
del list[2]  
print ("After deleting value at index 2 : ", list)
```

When the above code is executed, it produces the following result-

```
['physics', 'chemistry', 1997, 2000]  
After deleting value at index 2 : ['physics', 'chemistry', 2000]
```

Note: `remove()` method is discussed in subsequent section.

Basic List Operations

Lists respond to the + and * operators much like strings; they mean concatenation and repetition here too, except that the result is a new list, not a string.

In fact, lists respond to all of the general sequence operations we used on strings in the prior chapter.

Python Expression	Results	Description
len([1, 2, 3])	3	Length
[1, 2, 3] + [4, 5, 6]	[1, 2, 3, 4, 5, 6]	Concatenation
['Hi!'] * 4	['Hi!', 'Hi!', 'Hi!', 'Hi!']	Repetition
3 in [1, 2, 3]	True	Membership
for x in [1,2,3] : print (x,end=' ')	1 2 3	Iteration

Indexing, Slicing and Matrixes

Since lists are sequences, indexing and slicing work the same way for lists as they do for strings.

Assuming the following input-

```
L=['C++', 'Java', 'Python']
```

Python Expression	Results	Description
L[2]	'Python'	Offsets start at zero
L[-2]	'Java'	Negative: count from the right
L[1:]	['Java', 'Python']	Slicing fetches sections

Built-in List Functions & Methods

Python includes the following list functions-

Function	Description
cmp(list1, list2)	No longer available in Python 3.
len(list)	Gives the total length of the list.
max(list)	Returns item from the list with max value.
min(list)	Returns item from the list with min value.
list(seq)	Converts a tuple into list.

Tuples and Tuple Functions

A tuple is a sequence of immutable Python objects. Tuples are sequences, just like lists. The main difference between the tuples and the lists is that the tuples cannot be changed unlike lists. Tuples use parentheses, whereas lists use square brackets.

Creating a tuple is as simple as putting different comma-separated values. Optionally, you can put these comma-separated values between parentheses also.

```
tup1 = ('physics', 'chemistry', 1997, 2000)
tup2 = (1, 2, 3, 4, 5 )
tup3 = "a", "b", "c", "d"
```

The empty tuple is written as two parentheses containing nothing.

```
tup1 = ();
```

To write a tuple containing a single value you have to include a comma, even though there is only one value.

```
tup1 = (50,)
```

Like string indices, tuple indices start at 0, and they can be sliced, concatenated, and so on.

Accessing Values in Tuples

To access values in tuple, use the square brackets for slicing along with the index or indices to obtain the value available at that index.

```
#!/usr/bin/python3
tup1 = ('physics', 'chemistry', 1997, 2000)
tup2 = (1, 2, 3, 4, 5, 6, 7)
print ("tup1[0]: ", tup1[0])
print ("tup2[1:5]: ", tup2[1:5])
```

When the above code is executed, it produces the following result

```
tup1[0]: physics
tup2[1:5]: [2, 3, 4, 5]
```

Updating Tuples

Tuples are immutable, which means you cannot update or change the values of tuple elements. You are able to take portions of the existing tuples to create new tuples as the following example demonstrates.

```
#!/usr/bin/python3
tup1 = (12, 34.56)
tup2 = ('abc', 'xyz')

# Following action is not valid for tuples
# tup1[0] = 100;

# So let's create a new tuple as follows
tup3 = tup1 + tup2
print (tup3)
```

When the above code is executed, it produces the following result-

```
(12, 34.56, 'abc', 'xyz')
```

Delete Tuple Elements

Removing individual tuple elements is not possible. There is, of course, nothing wrong with putting together another tuple with the undesired elements discarded.

To explicitly remove an entire tuple, just use the del statement.

```
#!/usr/bin/python3
tup = ('physics', 'chemistry', 1997, 2000);
print (tup)
del tup;
print "After deleting tup : "
print tup
```

This produces the following result.

Note: An exception is raised. This is because after `del tup`, tuple does not exist any more.

```
('physics', 'chemistry', 1997, 2000)
```

After deleting `tup` :

```
Traceback (most recent call last):
File "test.py", line 9, in <module>
print tup;

NameError: name 'tup' is not defined
```

Basic Tuples Operations

Tuples respond to the `+` and `*` operators much like strings; they mean concatenation and repetition here too, except that the result is a new tuple, not a string.

In fact, tuples respond to all of the general sequence operations we used on strings.

Python Expression	Results	Description
<code>len((1, 2, 3))</code>	3	Length
<code>(1, 2, 3) + (4, 5, 6)</code>	<code>(1, 2, 3, 4, 5, 6)</code>	Concatenation
<code>('Hi!') * 4</code>	<code>('Hi!', 'Hi!', 'Hi!', 'Hi!')</code>	Repetition
<code>3 in (1, 2, 3)</code>	True	Membership
<code>for x in (1,2,3) : print (x, end=' ')</code>	1 2 3	Iteration

Indexing, Slicing, and Matrixes

Since tuples are sequences, indexing and slicing work the same way for tuples as they do for strings, assuming the following input-

```
T=('C++', 'Java', 'Python')
```

Python Expression	Results	Description
<code>T[2]</code>	'Python'	Offsets start at zero
<code>T[-2]</code>	'Java'	Negative: count from the right
<code>T[1:]</code>	('Java', 'Python')	Slicing fetches sections

No Enclosing Delimiters

No enclosing Delimiters is any set of multiple objects, comma-separated, written without identifying symbols, i.e., brackets for lists, parentheses for tuples, etc., default to tuples, as indicated in these short examples.

Built-in Tuple Functions

Function	Description
<code>cmp(tuple1, tuple2)</code>	No longer available in Python 3.
<code>len(tuple)</code>	Gives the total length of the tuple.
<code>max(tuple)</code>	Returns item from the tuple with max value.
<code>min(tuple)</code>	Returns item from the tuple with min value.
<code>tuple(seq)</code>	Converts a list into tuple.

Dictionary and Dictionary Functions

Each key is separated from its value by a colon (:), the items are separated by commas, and the whole thing is enclosed in curly braces. An empty dictionary without any items is written with just two curly braces, like this: {}.

Keys are unique within a dictionary while values may not be. The values of a dictionary can be of any type, but the keys must be of an immutable data type such as strings, numbers, or tuples.

Accessing Values in Dictionary

To access dictionary elements, you can use the familiar square brackets along with the key to obtain its value. Following is a simple example.

```
#!/usr/bin/python3
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}
print ("dict['Name']: ", dict['Name'])
print ("dict['Age']: ", dict['Age'])
```

When the above code is executed, it produces the following result

```
dict['Name']: Zara
dict['Age']: 7
```

If we attempt to access a data item with a key, which is not a part of the dictionary, we get an error as follows-

```
#!/usr/bin/python3
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'};
print "dict['Alice']: ", dict['Alice']
```

When the above code is executed, it produces the following result

```
dict['Zara']:
Traceback (most recent call last):
File "test.py", line 4, in <module>
print "dict['Alice']: ", dict['Alice'];

KeyError: 'Alice'
```

Updating Dictionary

You can update a dictionary by adding a new entry or a key-value pair, modifying an existing entry, or deleting an existing entry as shown in a simple example given below.

```
#!/usr/bin/python3
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}
dict['Age'] = 8;           # update existing entry
dict['School'] = "DPS School" # Add new entry
print ("dict['Age']: ", dict['Age'])
print ("dict['School']: ", dict['School'])
```

When the above code is executed, it produces the following result

```
dict['Age']: 8
dict['School']: DPS School
```

Delete Dictionary Elements

You can either remove individual dictionary elements or clear the entire contents of a dictionary. You can also delete entire dictionary in a single operation.

To explicitly remove an entire dictionary, just use the del statement.

```
#!/usr/bin/python3
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}
del dict['Name']           # remove entry with key 'Name'
dict.clear()              # remove all entries in dict
del dict                  # delete entire dictionary
print ("dict['Age']: ", dict['Age'])
print ("dict['School']: ", dict['School'])
```

This produces the following result.

Note: An exception is raised because after del dict, the dictionary does not exist anymore.

```
dict['Age']:
Traceback (most recent call last):
File "test.py", line 8, in <module>
print "dict['Age']: ", dict['Age'];

TypeError: 'type' object is unsubscriptable
```

Properties of Dictionary Keys

Dictionary values have no restrictions. They can be any arbitrary Python object, either standard objects or user-defined objects. However, same is not true for the keys.

There are two important points to remember about dictionary keys-

1. More than one entry per key is not allowed. This means no duplicate key is allowed. When duplicate keys are encountered during assignment, the last assignment wins.

```
#!/usr/bin/python3
dict = {'Name': 'Zara', 'Age': 7, 'Name': 'Manni'}
print ("dict['Name']: ", dict['Name'])
```

When the above code is executed, it produces the following result

```
dict['Name']: Manni
```

2. Keys must be immutable. This means you can use strings, numbers or tuples as dictionary keys but something like ['key'] is not allowed.

```
#!/usr/bin/python3
dict = {'Name': 'Zara', 'Age': 7}
print ("dict['Name']: ", dict['Name'])
```

When the above code is executed:

```
Traceback (most recent call last):
File "test.py", line 3, in <module>
dict = {'Name': 'Zara', 'Age': 7}
```

TypeError: list objects are unhashable

Built-in Dictionary Functions & Methods

Functions	Description
<code>cmp(dict1, dict2)</code>	No longer available in Python 3.
<code>len(dict)</code>	Gives the total length of the dictionary. This would be equal to the number of items in the dictionary.
<code>str(dict)</code>	Produces a printable string representation of a dictionary.
<code>type(variable)</code>	Returns the type of the passed variable. If passed variable is dictionary, then it would return a dictionary type.

Python includes the following dictionary methods-

Methods	Description
<code>dict.clear()</code>	Removes all elements of dictionary dict.
<code>dict.copy()</code>	Returns a shallow copy of dictionary dict.
<code>dict.fromkeys()</code>	Create a new dictionary with keys from seq and values set to value.
<code>dict.get(key, default=None)</code>	For key key, returns value or default if key not in dictionary.
<code>dict.has_key(key)</code>	Removed, use the <code>in</code> operation instead.
<code>dict.items()</code>	Returns a list of dict's (key, value) tuple pairs.
<code>dict.keys()</code>	Returns list of dictionary dict's keys.
<code>dict.setdefault(key, default=None)</code>	Similar to <code>get()</code> , but will set <code>dict[key]=default</code> if key is not already in dict.
<code>dict.update(dict2)</code>	Adds dictionary dict2's key-values pairs to dict.
<code>dict.values()</code>	Returns list of dictionary dict's values.

Date and Time and Date-Time Functions

A Python program can handle date and time in several ways. Converting between date formats is a common chore for computers. Python's time and calendar modules help track dates and times.

Python Time Tick

Time intervals are floating-point numbers in units of seconds. Particular instants in time are expressed in seconds since 12:00am, January 1, 1970(epoch).

There is a popular time module available in Python, which provides functions for working with times, and for converting between representations. The function `time.time()` returns the current system time in ticks since 12:00am, January 1, 1970(epoch).

```
#!/usr/bin/python3
import time;           # This is required to include time module.
ticks = time.time()
print ("Number of ticks since 12:00am, January 1, 1970:", ticks)
```

This would produce a result something as follows-

```
Number of ticks since 12:00am, January 1, 1970: 1455508609.34375
```

Date arithmetic is easy to do with ticks. However, dates before the epoch cannot be represented in this form. Dates in the far future also cannot be represented this way – the cutoff point is sometime in 2038 for UNIX and Windows.

Using TimeTuple

Many of the Python's time functions handle time as a tuple of 9 numbers, as shown below-

Index	Field	Values
0	4-digit year	2016
1	Month	1 to 12
2	Day	1 to 31
3	Hour	0 to 23
4	Minute	0 to 59
5	Second	0 to 61 (60 or 61 are leap-seconds)
6	Day of Week	0 to 6 (0 is Monday)
7	Day of year	1 to 366 (Julian day)
8	Daylight savings	-1, 0, 1, -1 means library determines DST

For Example-

```
>>>import time
>>> print (time.localtime())
```

This would produce a result as followtime.

```
struct_time(tm_year=2016, tm_mon=2, tm_mday=15, tm_hour=9, tm_min=29,
tm_sec=2, tm_wday=0, tm_yday=46, tm_isdst=0)
```

The above tuple is equivalent to struct_time structure. This structure has the following attributes-

Index	Attributes	Values
0	tm_year	2016
1	tm_mon	1 to 12
2	tm_mday	1 to 31
3	tm_hour	0 to 23
4	tm_min	0 to 59
5	tm_sec	0 to 61 (60 or 61 are leap-seconds)
6	tm_wday	0 to 6 (0 is Monday)
7	tm_yday	1 to 366 (Julian day)
8	tm_isdst	-1, 0, 1, -1 means library determines DST

Getting current time

To translate a time instant from seconds since the epoch floating-point value into a timetuple, pass the floating-point value to a function (e.g., localtime) that returns a time-tuple with all valid nine items.

```
#!/usr/bin/python3
import time
localtime = time.localtime(time.time())
print ("Local current time :", localtime)
```

This would produce the following result, which could be formatted in any other presentable form-

```
Local current time : time.struct_time(tm_year=2016, tm_mon=2, tm_mday=15,
tm_hour=9, tm_min=29, tm_sec=2, tm_wday=0, tm_yday=46, tm_isdst=0)
```

Getting formatted time

You can format any time as per your requirement, but a simple method to get time in a readable format is asctime() –

```
#!/usr/bin/python3
import time
localtime = time.asctime( time.localtime(time.time()) )
print ("Local current time :", localtime)
```

This would produce the following result-

```
Local current time : Mon Feb 15 09:34:03 2016
```

Getting calendar for a month

The calendar module gives a wide range of methods to play with yearly and monthly calendars. Here, we print a calendar for a given month (Jan 2008).

```
#!/usr/bin/python3
import calendar
cal = calendar.month(2016, 2)
print ("Here is the calendar:")
print (cal)
```

This would produce the following result-

```
Here is the calendar:
February 2016
Mo Tu We Th Fr Sa Su
1 2 3 4 5 6 7
8 9 10 11 12 13 14
15 16 17 18 19 20 21
22 23 24 25 26 27 28
29
```

The time Module

There is a popular time module available in Python, which provides functions for working with times and for converting between representations. Here is the list of all available methods.

Function	Description
time.altzone	The offset of the local DST timezone, in seconds west of UTC, if one is defined. This is negative if the local DST timezone is east of UTC (as in Western Europe, including the UK). Use this if the daylight is nonzero.
time.asctime([tupletime])	Accepts a time-tuple and returns a readable 24-character string such as 'Tue Dec 11 18:07:14 2008'.
time.clock()	Returns the current CPU time as a floating-point number of seconds. To measure computational costs of different approaches, the value of time.clock is more useful than that of time.time().
time.ctime([secs]) time.gmtime([secs])	Like asctime(localtime(secs)) and without arguments is like asctime() Accepts an instant expressed in seconds since the epoch and returns a time-tuple t with the UTC time. Note : t.tm_isdst is always 0
time.localtime([secs])	Accepts an instant expressed in seconds since the epoch and returns a time-tuple t with the local time (t.tm_isdst is 0 or 1, depending on whether DST applies to instant secs by local rules).
time.mktime(tupletime)	Accepts an instant expressed as a time-tuple in local time and returns a floatingpoint value with the instant expressed in seconds since the epoch.
time.sleep(secs)	Suspends the calling thread for secs seconds.
time.time()	Returns the current time instant, a floating-point number of seconds since the

	epoch.
<code>time.tzset()</code>	Resets the time conversion rules used by the library routines. The environment variable TZ specifies how this is done.
<code>time.strptime(fmt[,tupletime])</code>	Accepts an instant expressed as a time-tuple in local time and returns a string representing the instant as specified by string fmt.
<code>time.strptime(str,fmt='%a %b %d %H:%M:%S %Y')</code>	Parses str according to format string fmt and returns the instant in time-tuple format.

The following directives can be embedded in the format string-

Directive

<code>%a</code>	- abbreviated weekday name
<code>%A</code>	- full weekday name
<code>%b</code>	- abbreviated month name
<code>%B</code>	- full month name
<code>%c</code>	- preferred date and time representation
<code>%C</code>	- century number (the year divided by 100, range 00 to 99)
<code>%d</code>	- day of the month (01 to 31)
<code>%D</code>	- same as <code>%m/%d/%y</code>
<code>%e</code>	- day of the month (1 to 31)
<code>%g</code>	- like <code>%G</code> , but without the century
<code>%G</code>	- 4-digit year corresponding to the ISO week number (see <code>%V</code>).
<code>%h</code>	- same as <code>%b</code>
<code>%H</code>	- hour, using a 24-hour clock (00 to 23)
<code>%I</code>	- hour, using a 12-hour clock (01 to 12)
<code>%j</code>	- day of the year (001 to 366)
<code>%m</code>	- month (01 to 12)
<code>%M</code>	- minute
<code>%n</code>	- newline character
<code>%p</code>	- either am or pm according to the given time value
<code>%r</code>	- time in a.m. and p.m. notation
<code>%R</code>	- time in 24 hour notation
<code>%S</code>	- second
<code>%t</code>	- tab character
<code>%T</code>	- current time, equal to <code>%H:%M:%S</code>
<code>%u</code>	- weekday as a number (1 to 7), Monday=1. Warning: In Sun Solaris Sunday=1
<code>%U</code>	- week number of the current year, starting with the first Sunday as the first day of the first week
<code>%V</code>	- The ISO 8601 week number of the current year (01 to 53), where week 1 is the first week that has at least 4 days in the current year, and with Monday as the first day of the week
<code>%W</code>	- week number of the current year, starting with the first Monday as the first day of the first week
<code>%w</code>	- day of the week as a decimal, Sunday=0
<code>%x</code>	- preferred date representation without the time
<code>%X</code>	- preferred time representation without the date
<code>%y</code>	- year without a century (range 00 to 99)
<code>%Y</code>	- year including the century
<code>%Z</code> or <code>%z</code>	- time zone or name or abbreviation
<code>%%</code>	- a literal <code>%</code> character

Example

The following example shows the usage of `strftime()` method.

```
#!/usr/bin/python3
import time
t = (2015, 12, 31, 10, 39, 45, 1, 48, 0)
t = time.mktime(t)
print (time.strftime("%b %d %Y %H:%M:%S", time.localtime(t)))
```

When we run the above program, it produces the following result-

```
Dec 31 2015 10:39:45
```

Example

The following example shows the usage of `strptime()` method.

```
#!/usr/bin/python3
#this activity shows how to get the current date and time elements
import datetime

now=datetime.datetime.now()
print('this is a demonstration of the datetime module in python')

#show date time format of 2010-12-25 2:00:00 am
print('\n'+ str(now.year) + '-' + str(now.month) + '-' + str(now.day) + \
' ' + str(now.hour) + ':' + str(now.minute) + ':' + str(now.second))

#you can also use strftime() function to format the datetime
format1 = '%Y/%m/%d %l:%M %p'
now2 = now.strftime(format1)
print('\n' + now2)

#####
#so, basically, the easiest method perhaps would be to:
#####

#1.create a reference for the current datetime
dateref=datetime.datetime.now()

#2.then use its strftime() function with the format you want to extract
print('year:' + dateref.strftime('%Y'))
print('month:' + dateref.strftime('%m'))
print('month name short:' + dateref.strftime('%b'))
print('month name long:' + dateref.strftime('%B'))
print('day:' + dateref.strftime('%Y'))
print('day of week:' + dateref.strftime('%A'))
print('hour (24 hour):' + dateref.strftime('%H'))
print('hour (12 hour):' + dateref.strftime('%I'))
print('minutes:' + dateref.strftime('%M'))
print('seconds:' + dateref.strftime('%S'))
print('time of day:' + dateref.strftime('%l:%M %p')) #%p must be used with 12-hour format %l
```

When we run the above program, it produces the following result

this is a demonstration of the datetime module in python

2018-3-9 22:51:17

2018/03/09 10:51 PM
year:2018
month:03
month name short:Mar
month name long:March
day:09
day of week:Saturday
hour (24 hour):22
hour (12 hour):10
minutes:51
seconds:17
time of day:10:51 PM

There are two important attributes available with time module. They are-

Attribute	Description
time.timezone	Attribute time.timezone is the offset in seconds of the local time zone (without DST) from UTC (>0 in the Americas; <=0 in most of Europe, Asia, Africa).
time.tzname	Attribute time.tzname is a pair of locale-dependent strings, which are the names of the local time zone without and with DST, respectively.

The calendar Module

The calendar module supplies calendar-related functions, including functions to print a text calendar for a given month or year. By default, calendar takes Monday as the first day of the week and Sunday as the last one. To change this, call the calendar.setfirstweekday() function.

Function	Description
calendar.calendar(year,w=2,l=1,c=6)	Returns a multiline string with a calendar for year year formatted into three columns separated by c spaces. w is the width in characters of each date; each line has length 21*w+18+2*c. l is the number of lines for each week.
calendar.firstweekday()	Returns the current setting for the weekday that starts each week. By default, when calendar is first imported, this is 0, meaning Monday.
calendar.isleap(year)	Returns True if year is a leap year; otherwise, False.
calendar.leapdays(y1,y2)	Returns the total number of leap days in the years within range(y1,y2).
calendar.month(year,month,w=2,l=1)	Returns a multiline string with a calendar for month month of year year, one line per week plus two header lines. w is the width in characters of each date; each line has length 7*w+6. l is the number of lines for each week.
calendar.monthcalendar(year,month)	Returns a list of lists of ints. Each sublist denotes a week. Days outside month month of year year are set to 0; days within the month are set to their day-of-month, 1 and up.
calendar.monthrange(year,month)	Returns two integers. The first one is the code of the weekday for the first day of the month <i>month</i> in year <i>year</i> ; the second one is the number of days in the

	month. Weekday codes are 0 (Monday) to 6 (Sunday); month numbers are 1 to 12.
<code>calendar.prcal(year,w=2,l=1,c=6)</code>	Like <code>print calendar.calendar(year,w,l,c)</code> .
<code>calendar.prmonth(year,month,w=2,l=1)</code>	Like <code>print calendar.month(year,month,w,l)</code> .
<code>calendar.setfirstweekday(weekday)</code>	Sets the first day of each week to weekday code <code>weekday</code> . Weekday codes are 0 (Monday) to 6 (Sunday).
<code>calendar.timegm(tupletime)</code>	The inverse of <code>time.gmtime</code> : accepts a time instant in time-tuple form and returns the same instant as a floating-point number of seconds since the epoch.
<code>calendar.weekday(year,month,day)</code>	Returns the weekday code for the given date. Weekday codes are 0 (Monday) to 6 (Sunday); month numbers are 1 (January) to 12 (December).

Date Differences

```
import datetime

date1=datetime.datetime.strptime('October 13, 1985 8:00AM','%B %d, %Y %I:%M%p')
date2=datetime.datetime.now()

diff=date2-date1
print("difference in years:" + str(int(diff.days/365)) + "\n")
print("difference in hours:" + str(int(diff.total_seconds()/3600)) + "\n")
```

User-Defined Functions

A function is a block of organized, reusable code that is used to perform a single, related action. Functions provide better modularity for your application and a high degree of code reusing.

Defining a Function

You can define functions to provide the required functionality. Here are simple rules to define a function in Python.

- ✓ Function blocks begin with the keyword `def` followed by the function name and parentheses `( )`.
- ✓ Any input parameters or arguments should be placed within these parentheses. You can also define parameters inside these parentheses.
- ✓ The first statement of a function can be an optional statement - the documentation string of the function or `docstring`.
- ✓ The code block within every function starts with a colon `(:)` and is indented.
- ✓ The statement `return [expression]` exits a function, optionally passing back an expression to the caller. A return statement with no arguments is the same as `return None`.

Syntax

```
def functionname( parameters ):
    "function_docstring"
    function_suite
    return [expression]
```

By default, parameters have a positional behavior and you need to inform them in the same order that they were defined.

Example

The following function takes a string as input parameter and prints it on the standard screen.

```
def printme( str ):
    "This prints a passed string into this function"
    print (str)
    return
```

Calling a Function

Defining a function gives it a name, specifies the parameters that are to be included in the function and structures the blocks of code. Once the basic structure of a function is finalized, you can execute it by calling it from another function or directly from the Python prompt. Following is an example to call the printme() function-

```
#!/usr/bin/python3
# Function definition is here

def printme( str ):
    "This prints a passed string into this function"
    print (str)
    return
# Now you can call printme function
printme("This is first call to the user defined function!")
printme("Again second call to the same function")
```

When the above code is executed, it produces the following result-

```
This is first call to the user defined function!
Again second call to the same function
```

Pass by Reference vs Value

All parameters (arguments) in the Python language are passed by reference. It means if you change what a parameter refers to within a function, the change also reflects back in the calling function. For example-

```
#!/usr/bin/python3
# Function definition is here

def changeme( mylist ):
    "This changes a passed list into this function"
    print ("Values inside the function before change: ", mylist)
    mylist[2]=50
    print ("Values inside the function after change: ", mylist)
    return

# Now you can call changeme function
mylist = [10,20,30]
changeme( mylist )
print ("Values outside the function: ", mylist)
```

Here, we are maintaining reference of the passed object and appending values in the same object. Therefore, this would produce the following result-

```
Values inside the function before change: [10, 20, 30]
```

Values inside the function after change: [10, 20, 50]

Values outside the function: [10, 20, 50]

There is one more example where argument is being passed by reference and the reference is being overwritten inside the called function.

```
#!/usr/bin/python3
# Function definition is here

def changeme( mylist ):
    "This changes a passed list into this function"
    mylist = [1,2,3,4] # This would assi new reference in mylist
    print ("Values inside the function: ", mylist)
    return

# Now you can call changeme function
mylist = [10,20,30]
changeme( mylist )
print ("Values outside the function: ", mylist)
```

The parameter mylist is local to the function changeme. Changing mylist within the function does not affect mylist. The function accomplishes nothing and finally this would produce the following result-

Values inside the function: [1, 2, 3, 4]

Values outside the function: [10, 20, 30]

Function Arguments

You can call a function by using the following types of formal arguments-

- Required arguments
- Keyword arguments
- Default arguments
- Variable-length arguments

Required Arguments

Required arguments are the arguments passed to a function in correct positional order. Here, the number of arguments in the function call should match exactly with the function definition.

To call the function printme(), you definitely need to pass one argument, otherwise it gives a syntax error as follows-

```
#!/usr/bin/python3
# Function definition is here
def printme( str ):
    "This prints a passed string into this function"
    print (str)
    return

# Now you can call printme function
printme()
```

When the above code is executed, it produces the following result-

Traceback (most recent call last):

File "test.py", line 11, in <module>

```
printme()
TypeError: printme() missing 1 required positional argument: 'str'
```

Keyword Arguments

Keyword arguments are related to the function calls. When you use keyword arguments in a function call, the caller identifies the arguments by the parameter name. This allows you to skip arguments or place them out of order because the Python interpreter is able to use the keywords provided to match the values with parameters. You can also make keyword calls to the printme() function in the following ways-

```
#!/usr/bin/python3
# Function definition is here

def printme( str ):
    "This prints a passed string into this function"
    print (str)
    return

# Now you can call printme function
printme( str = "My string")
```

When the above code is executed, it produces the following result-

```
My string
```

The following example gives a clearer picture. Note that the order of parameters does not matter.

```
#!/usr/bin/python3
# Function definition is here

def printinfo( name, age ):
    "This prints a passed info into this function"
    print ("Name: ", name)
    print ("Age ", age)
    return

# Now you can call printinfo function
printinfo( age=50, name="miki" )
```

When the above code is executed, it produces the following result-

```
Name: miki
Age 50
```

Default Arguments

A default argument is an argument that assumes a default value if a value is not provided in the function call for that argument. The following example gives an idea on default arguments, it prints default age if it is not passed.

```
#!/usr/bin/python3
# Function definition is here

def printinfo( name, age = 35 ):
    "This prints a passed info into this function"
    print ("Name: ", name)
```

```
print ("Age ", age)
return
```

```
# Now you can call printinfo function
printinfo( age=50, name="miki" )
printinfo( name="miki" )
```

When the above code is executed, it produces the following result-

```
Name: miki
Age 50
Name: miki
Age 35
```

Variable-length Arguments

You may need to process a function for more arguments than you specified while defining the function. These arguments are called variable-length arguments and are not named in the function definition, unlike required and default arguments.

Syntax for a function with non-keyword variable arguments is given below

```
def functionname([formal_args,] *var_args_tuple ):
    "function_docstring"
    function_suite
    return [expression]
```

An asterisk (*) is placed before the variable name that holds the values of all non-keyword variable arguments. This tuple remains empty if no additional arguments are specified during the function call.

```
#!/usr/bin/python3
# Function definition is here

def printinfo( arg1, *vartuple ):
    "This prints a variable passed arguments"
    print ("Output is: ")
    print (arg1)
    for var in vartuple:
        print (var)
    return

# Now you can call printinfo function
printinfo( 10 )
printinfo( 70, 60, 50 )
```

When the above code is executed, it produces the following result-

```
Output is:
10
Output is:
70
60
50
```

The return Statement

The statement `return [expression]` exits a function, optionally passing back an expression to the caller. A return statement with no arguments is the same as `return None`.

You can return a value from a function as follows-

```
#!/usr/bin/python3
# Function definition is here

def sum( arg1, arg2 ):
    # Add both the parameters and return them."
    total = arg1 + arg2
    print ("Inside the function : ", total)
    return total

# Now you can call sum function
total = sum( 10, 20 )
print ("Outside the function : ", total )
```

When the above code is executed, it produces the following result-

```
Inside the function : 30
Outside the function : 30
```

Scope of Variables

All variables in a program may not be accessible at all locations in that program. This depends on where you have declared a variable. The scope of a variable determines the portion of the program where you can access a particular identifier. There are two basic scopes of variables in Python-

1. Global variables
2. Local variables

Global vs. Local variables

Variables that are defined inside a function body have a local scope, and those defined outside have a global scope. This means that local variables can be accessed only inside the function in which they are declared, whereas global variables can be accessed throughout the program body by all functions. When you call a function, the variables declared inside it are brought into scope.

Following is a simple example-

```
#!/usr/bin/python3
total = 0      # This is global variable.

# Function definition is here
def sum( arg1, arg2 ):

    # Add both the parameters and return them."
    total = arg1 + arg2; # Here total is local variable.
    print ("Inside the function local total : ", total)
    return total

# Now you can call sum function
```

```
sum( 10, 20 )
print ("Outside the function global total : ", total )
```

When the above code is executed, it produces the following result-

```
Inside the function local total : 30
Outside the function global total : 0
```

Python Modules

A module allows you to logically organize your Python code. Grouping related code into a module makes the code easier to understand and use. A module is a Python object with arbitrarily named attributes that you can bind and reference. Simply, a module is a file consisting of Python code. A module can define functions, classes and variables. A module can also include runnable code.

Example

The Python code for a module named `aname` normally resides in a file named `aname.py`.

Here is an example of a simple module, `support.py`

```
print_func( par ):
print "Hello : ", par
return
```

The import Statement

You can use any Python source file as a module by executing an import statement in some other Python source file. The import has the following syntax

```
import module1[, module2[,... moduleN]
```

When the interpreter encounters an import statement, it imports the module if the module is present in the search path. A search path is a list of directories that the interpreter searches before importing a module. For example, to import the module `hello.py`, you need to put the following command at the top of the script-

```
#!/usr/bin/python3
# Import module support
import support

# Now you can call defined function that module as follows
support.print_func("Zara")
```

When the above code is executed, it produces the following result-

```
Hello : Zara
```

A module is loaded only once, regardless of the number of times it is imported. This prevents the module execution from happening repeatedly, if multiple imports occur.

The from...import Statement

Python's from statement lets you import specific attributes from a module into the current namespace.

```
from modname import name1[, name2[, ... nameN]]
```

For example, to import the function fibonacci from the module fib, use the following statement-

```
#!/usr/bin/python3
# Fibonacci numbers module

def fib(n): # return Fibonacci series up to n
    result = []
    a, b = 0, 1
    while b < n:
        result.append(b)
        a, b = b, a+b
    return result

>>> from fib import fib
>>> fib(100)
[1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
```

This statement does not import the entire module fib into the current namespace; it just introduces the item fibonacci from the module fib into the global symbol table of the importing module.

The from...import * Statement:

It is also possible to import all the names from a module into the current namespace by using the following import statement

```
from modname import *
```

This provides an easy way to import all the items from a module into the current namespace; however, this statement should be used sparingly.

Executing Modules as Scripts

Within a module, the module's name (as a string) is available as the value of the global variable `__name__`. The code in the module will be executed, just as if you imported it, but with the `__name__` set to `"__main__"`.

Add this code at the end of your module-

```
#!/usr/bin/python3
# Fibonacci numbers module

def fib(n): # return Fibonacci series up to n
    result = []
    a, b = 0, 1
    while b < n:
        result.append(b)
        a, b = b, a+b
    return result
if __name__ == "__main__":
    f=fib(100)
    print(f)
```

When you run the above code, the following output will be displayed.

```
[1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
```

Locating Modules

When you import a module, the Python interpreter searches for the module in the following sequences-

- ✓ The current directory.
- ✓ If the module is not found, Python then searches each directory in the shell variable PYTHONPATH.
- ✓ If all else fails, Python checks the default path. On UNIX, this default path is normally /usr/local/lib/python3/.

The module search path is stored in the system module sys as the sys.path variable. The sys.path variable contains the current directory, PYTHONPATH, and the installation dependent default.

The PYTHONPATH Variable

The PYTHONPATH is an environment variable, consisting of a list of directories. The syntax of PYTHONPATH is the same as that of the shell variable PATH.

Here is a typical PYTHONPATH from a Windows systemset

```
PYTHONPATH=c:\python34\lib;
```

And here is a typical PYTHONPATH from a UNIX systemset

```
PYTHONPATH=/usr/local/lib/python
```

Namespaces and Scoping

Variables are names (identifiers) that map to objects. A namespace is a dictionary of variable names (keys) and their corresponding objects (values).

- ✓ A Python statement can access variables in a local namespace and in the global namespace. If a local and a global variable have the same name, the local variable shadows the global variable.
- ✓ Each function has its own local namespace. Class methods follow the same scoping rule as ordinary functions.
- ✓ Python makes educated guesses on whether variables are local or global. It assumes that any variable assigned a value in a function is local.
- ✓ Therefore, in order to assign a value to a global variable within a function, you must first use the global statement.
- ✓ The statement global VarName tells Python that VarName is a global variable. Python stops searching the local namespace for the variable.

For example, we define a variable Money in the global namespace. Within the function Money, we assign Money a value, therefore Python assumes Money as a local variable.

However, we accessed the value of the local variable Money before setting it, so an UnboundLocalError is the result. Uncommenting the global statement fixes the problem.

```
#!/usr/bin/python3
Money = 2000

def AddMoney():
    # Uncomment the following line to fix the code:
    # global Money
    Money = Money + 1
```

```
print (Money)
```

```
AddMoney()  
print (Money)
```

The dir() Function

The dir() built-in function returns a sorted list of strings containing the names defined by a module. The list contains the names of all the modules, variables and functions that are defined in a module. Following is a simple example-

```
#!/usr/bin/python3  
# Import built-in module math  
import math  
content = dir(math)  
print (content)
```

When the above code is executed, it produces the following result-

```
['__doc__', '__file__', '__name__', 'acos', 'asin', 'atan', 'atan2', 'ceil', 'cos', 'cosh', 'degrees', 'e', 'exp', 'fabs', 'floor', 'fmod',  
'frexp', 'hypot', 'ldexp', 'log', 'log10', 'modf', 'pi', 'pow', 'radians', 'sin', 'sinh', 'sqrt', 'tan', 'tanh']
```

Here, the special string variable __name__ is the module's name, and __file__ is the filename from which the module was loaded.

The globals() and locals() Functions

The globals() and locals() functions can be used to return the names in the global and local namespaces depending on the location from where they are called.

- If locals() is called from within a function, it will return all the names that can be accessed locally from that function.
- If globals() is called from within a function, it will return all the names that can be accessed globally from that function.

The return type of both these functions is dictionary. Therefore, names can be extracted using the keys() function.

The reload() Function

When a module is imported into a script, the code in the top-level portion of a module is executed only once. Therefore, if you want to reexecute the top-level code in a module, you can use the reload() function. The reload() function imports a previously imported module again.

```
reload(module_name)
```

Here, module_name is the name of the module you want to reload and not the string containing the module name. For example, to reload hello module, do the following

```
reload(hello)
```

Packages in Python

A package is a hierarchical file directory structure that defines a single Python application environment that consists of modules and subpackages and sub-subpackages, and so on. Consider a file Pots.py available in Phone directory. This file has the following line of source code-

```
#!/usr/bin/python3
```

```
def Pots():  
    print ("I'm Pots Phone")
```

Similarly, we have other two files having different functions with the same name as above.

1. Phone/Isdn.py file having function Isdn()
2. Phone/G3.py file having function G3()

Now, create one more file `__init__.py` in the Phone directory-

- Phone/`__init__.py`

To make all of your functions available when you have imported Phone, you need to put explicit import statements in `__init__.py` as follows

```
from Pots import Pots  
from Isdn import Isdn  
from G3 import G3
```

After you add these lines to `__init__.py`, you have all of these classes available when you import the Phone package.

```
#!/usr/bin/python3  
# Now import your Phone Package.  
import Phone  
Phone.Pots()  
Phone.Isdn()  
Phone.G3()
```

When the above code is executed, it produces the following result

```
Python  
3
```

Python – Files I/O

The input Function

The `input([prompt])` function is equivalent to `raw_input`, except that it assumes that the input is a valid Python expression and returns the evaluated result to you.

```
#!/usr/bin/python3  
>>> x=input("something:")  
something:10  
>>> x  
'10'  
>>> x=input("something:")  
something:'10'          #entered data treated as string with or without "  
>>> x  
"10"
```

Opening and Closing Files

Python provides basic functions and methods necessary to manipulate files by default. You can do most of the file manipulation using a file object.

The open Function

Before you can read or write a file, you have to open it using Python's built-in `open()` function. This function creates a file object, which would be utilized to call other support methods associated with it.

Syntax

<code>file object = open(file_name [, access_mode][, buffering])</code>

Here are parameter details-

- **file_name:** The `file_name` argument is a string value that contains the name of the file that you want to access.
- **access_mode:** The `access_mode` determines the mode in which the file has to be opened, i.e., read, write, append, etc. A complete list of possible values is given below in the table. This is an optional parameter and the default file access mode is read (r).
- **buffering:** If the buffering value is set to 0, no buffering takes place. If the buffering value is 1, line buffering is performed while accessing a file. If you specify the buffering value as an integer greater than 1, then buffering action is performed with the indicated buffer size. If negative, the buffer size is the system default (default behavior).

Here is a list of the different modes of opening a file-

Modes	Description
r	Opens a file for reading only. The file pointer is placed at the beginning of the file. This is the default mode.
rb	Opens a file for reading only in binary format. The file pointer is placed at the beginning of the file. This is the default mode.
r+	Opens a file for both reading and writing. The file pointer placed at the beginning of the file.
rb+	Opens a file for both reading and writing in binary format. The file pointer placed at the beginning of the file.
w	Opens a file for writing only. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing.
wb	Opens a file for writing only in binary format. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing.
w+	Opens a file for both writing and reading. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing.
wb+	Opens a file for both writing and reading in binary format. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing.
a	Opens a file for appending. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing.
ab	Opens a file for appending in binary format. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing.
a+	Opens a file for both appending and reading. The file pointer is at the end of the file if the file exists. The file opens in the append mode. If the file does not exist, it creates a new file for reading and writing.

ab+ Opens a file for both appending and reading in binary format. The file pointer is at the end of the file if the file exists. The file opens in the append mode. If the file does not exist, it creates a new file for reading and writing.

The file Object Attributes

Once a file is opened and you have one file object, you can get various information related to that file.

Attribute	Description
file.closed	Returns true if file is closed, false otherwise.
file.mode	Returns access mode with which file was opened.
file.name	Returns name of the file.

Example

```
#!/usr/bin/python3
# Open a file

fo = open("foo.txt", "wb")
print ("Name of the file: ", fo.name)
print ("Closed or not : ", fo.closed)
print ("Opening mode : ", fo.mode)
fo.close()
```

This produces the following result-

```
Name of the file: foo.txt
Closed or not : False
Opening mode : wb
```

The close() Method

The close() method of a file object flushes any unwritten information and closes the file object, after which no more writing can be done.

Python automatically closes a file when the reference object of a file is reassigned to another file. It is a good practice to use the close() method to close a file.

```
fileObject.close()
```

Example

```
#!/usr/bin/python3
# Open a file

fo = open("foo.txt", "wb")
print ("Name of the file: ", fo.name)
# Close opened file
fo.close()
```

This produces the following result-

Name of the file: foo.txt

Reading and Writing Files

The file object provides a set of access methods to make our lives easier. We would see how to use `read()` and `write()` methods to read and write files.

The write() Method

The `write()` method writes any string to an open file. It is important to note that Python strings can have binary data and not just text. The `write()` method does not add a newline character (`'\n'`) to the end of the string-

```
fileObject.write(string)
```

Here, passed parameter is the content to be written into the opened file.

Example

```
#!/usr/bin/python3

# Open a file
fo = open("foo.txt", "w")
fo.write( "Python is a great language.\nYeah its great!!\n")

# Close opened file
fo.close()
```

The above method would create `foo.txt` file and would write given content in that file and finally it would close that file. If you would open this file, it would have the following content-

```
Python is a great language.
Yeah its great!!
```

The read() Method

The `read()` method reads a string from an open file. It is important to note that Python strings can have binary data apart from the text data.

```
fileObject.read([count])
```

Here, passed parameter is the number of bytes to be read from the opened file. This method starts reading from the beginning of the file and if count is missing, then it tries to read as much as possible, maybe until the end of file.

Let us take a file `foo.txt`, which we created above.

```
#!/usr/bin/python3

# Open a file
fo = open("foo.txt", "r+")
str = fo.read(10)
print ("Read String is : ", str)

# Close opened file
```

```
fo.close()
```

This produces the following result-

```
Read String is : Python is
```

File Positions

The `tell()` method tells you the current position within the file; in other words, the next read or write will occur at that many bytes from the beginning of the file.

The `seek(offset[, from])` method changes the current file position. The `offset` argument indicates the number of bytes to be moved. The `from` argument specifies the reference position from where the bytes are to be moved.

If `from` is set to 0, the beginning of the file is used as the reference position. If it is set to 1, the current position is used as the reference position. If it is set to 2 then the end of the file would be taken as the reference position.

Let us take a file `foo.txt`, which we created.

```
#!/usr/bin/python3

# Open a file
fo = open("foo.txt", "r+")
str = fo.read(10)
print ("Read String is : ", str)

# Check current position
position = fo.tell()
print ("Current file position : ", position)

# Reposition pointer at the beginning once again
position = fo.seek(0, 0)
str = fo.read(10)
print ("Again read String is : ", str)

# Close opened file
fo.close()
```

This produces the following result-

```
Read String is : Python is
Current file position : 10
Again read String is : Python is
```

Renaming and Deleting Files

Python `os` module provides methods that help you perform file-processing operations, such as renaming and deleting files. To use this module, you need to import it first and then you can call any related functions.

The rename() Method

The rename() method takes two arguments, the current filename and the new filename.

```
os.rename(current_file_name, new_file_name)
```

Example

Following is an example to rename an existing file test1.txt-

```
#!/usr/bin/python3
import os

# Rename a file from test1.txt to test2.txt
os.rename( "test1.txt", "test2.txt" )
```

The remove() Method

You can use the remove() method to delete files by supplying the name of the file to be deleted as the argument.

```
os.remove(file_name)
```

Example

Following is an example to delete an existing file test2.txt-

```
#!/usr/bin/python3
import os

# Delete file test2.txt
os.remove("text2.txt")
```

Directories in Python

All files are contained within various directories, and Python has no problem handling these too. The os module has several methods that help you create, remove, and change directories.

The mkdir() Method

You can use the mkdir() method of the os module to create directories in the current directory. You need to supply an argument to this method, which contains the name of the directory to be created.

```
os.mkdir("newdir")
```

Following is an example to create a directory test in the current directory-

```
#!/usr/bin/python3
import os

# Create a directory "test"
os.mkdir("test")
```

The chdir() Method

You can use the chdir() method to change the current directory. The chdir() method takes an argument, which is the name of the directory that you want to make the current directory.

```
os.chdir("newdir")
```

Following is an example to go into "/home/newdir" directory-

```
#!/usr/bin/python3
import os

# Changing a directory to "/home/newdir"
os.chdir("/home/newdir")
```

The getcwd() Method

The getcwd() method displays the current working directory.

```
os.getcwd()
```

Following is an example to give current directory-

```
#!/usr/bin/python3
import os
# This would give location of the current directory
os.getcwd()
```

The rmdir() Method

The rmdir() method deletes the directory, which is passed as an argument in the method. Before removing a directory, all the contents in it should be removed.

```
os.rmdir('dirname')
```

Following is an example to remove the "/tmp/test" directory. It is required to give fully qualified name of the directory, otherwise it would search for that directory in the current directory.

```
#!/usr/bin/python3
import os

# This would remove "/tmp/test" directory.
os.rmdir("/tmp/test")
```

File & Directory Related Methods

There are three important sources, which provide a wide range of utility methods to handle and manipulate files & directories on Windows and Unix operating systems. They are as follows-

- File Object Methods: The file object provides functions to manipulate files.
- OS Object Methods: This provides methods to process files as well as directories.

File Methods

A file object is created using open function and here is a list of functions which can be called on this object.

Methods	Description
<code>file.close()</code>	Close the file. A closed file cannot be read or written any more.
<code>file.flush()</code>	Flush the internal buffer, like <code>stdio's fflush</code> . This may be a no-op on some file-like objects.
<code>file.fileno()</code>	Returns the integer file descriptor that is used by the underlying implementation to request I/O operations from the operating system.
<code>file.isatty()</code>	Returns True if the file is connected to a tty(-like) device, else False.
<code>next(file)</code>	Returns the next line from the file each time it is being called.
<code>file.read([size])</code>	Reads at most <code>size</code> bytes from the file (less if the read hits EOF before obtaining <code>size</code> bytes).
<code>file.readline([size])</code>	Reads one entire line from the file. A trailing newline character is kept in the string.
<code>file.readlines([sizehint])</code>	Reads until EOF using <code>readline()</code> and return a list containing the lines. If the optional <code>sizehint</code> argument is present, instead of reading up to EOF, whole lines totaling approximately <code>sizehint</code> bytes (possibly after rounding up to an internal buffer size) are read.
<code>file.seek(offset[, whence])</code>	Sets the file's current position
<code>file.tell()</code>	Returns the file's current position
<code>file.truncate([size])</code>	Truncates the file's size. If the optional <code>size</code> argument is present, the file is truncated to (at most) that size.
<code>file.write(str)</code>	Writes a string to the file. There is no return value.
<code>file.writelines(sequence)</code>	Writes a sequence of strings to the file. The sequence can be any iterable object producing strings, typically a list of strings.

OS File/Directory Methods

The `os` module provides a big range of useful methods to manipulate files and directories.

Methods	Description
<code>os.access(path, mode)</code>	Use the real uid/gid to test for access to path.
<code>os.chdir(path)</code>	Change the current working directory to path
<code>os.chflags(path, flags)</code>	Set the flags of path to the numeric flags.
<code>os.chmod(path, mode)</code>	Change the mode of path to the numeric mode.
<code>os.chown(path, uid, gid)</code>	Change the owner and group id of path to the numeric uid and gid.
<code>os.chroot(path)</code>	Change the root directory of the current process to path.
<code>os.close(fd)</code>	Close file descriptor fd.
<code>os.closerange(fd_low, fd_high)</code>	Close all file descriptors from <code>fd_low</code> (inclusive) to <code>fd_high</code> (exclusive), ignoring errors.
<code>os.dup(fd)</code>	Return a duplicate of file descriptor fd.
<code>os.dup2(fd, fd2)</code>	Duplicate file descriptor fd to fd2, closing the latter first if necessary.
<code>os.fchdir(fd)</code>	Change the current working directory to the directory represented by the file descriptor fd.
<code>os.fchmod(fd, mode)</code>	Change the mode of the file given by fd to the numeric mode.
<code>os.fchown(fd, uid, gid)</code>	Change the owner and group id of the file given by fd to the numeric uid and gid.
<code>os.fdatasync(fd)</code>	Force write of file with filedescriptor fd to disk.
<code>os.fdopen(fd[, mode[, bufsize]])</code>	Return an open file object connected to the file descriptor fd.
<code>os.fpathconf(fd, name)</code>	Return system configuration information relevant to an open file. <code>name</code> specifies the configuration value to retrieve.

<code>os.fstat(fd)</code>	Return status for file descriptor <code>fd</code> , like <code>stat()</code> .
<code>os.fstatvfs(fd)</code>	Return information about the filesystem containing the file associated with file descriptor <code>fd</code> , like <code>statvfs()</code> .
<code>os.fsync(fd)</code>	Force write of file with file descriptor <code>fd</code> to disk.
<code>os.ftruncate(fd, length)</code>	Truncate the file corresponding to file descriptor <code>fd</code> , so that it is at most <code>length</code> bytes in size.
<code>os.getcwd()</code>	Return a string representing the current working directory.
<code>os.getcwdu()</code>	Return a Unicode object representing the current working directory.
<code>os.isatty(fd)</code>	Return True if the file descriptor <code>fd</code> is open and connected to a tty(-like) device, else False.
<code>os.lchflags(path, flags)</code>	Set the flags of <code>path</code> to the numeric flags, like <code>chflags()</code> , but do not follow symbolic links.
<code>os.lchmod(path, mode)</code>	Change the mode of <code>path</code> to the numeric mode.
<code>os.lchown(path, uid, gid)</code>	Change the owner and group id of <code>path</code> to the numeric <code>uid</code> and <code>gid</code> . This function will not follow symbolic links.
<code>os.link(src, dst)</code>	Create a hard link pointing to <code>src</code> named <code>dst</code> .
<code>os.listdir(path)</code>	Return a list containing the names of the entries in the directory given by <code>path</code> .
<code>os.lseek(fd, pos, how)</code>	Set the current position of file descriptor <code>fd</code> to position <code>pos</code> , modified by <code>how</code> .
<code>os.lstat(path)</code>	Like <code>stat()</code> , but do not follow symbolic links.
<code>os.major(device)</code>	Extract the device major number from a raw device number.
<code>os.makedev(major, minor)</code>	Compose a raw device number from the major and minor device numbers.
<code>os.makedirs(path[, mode])</code>	Recursive directory creation function.
<code>os.minor(device)</code>	Extract the device minor number from a raw device number .
<code>os.mkdir(path[, mode])</code>	Create a directory named <code>path</code> with numeric mode <code>mode</code> .
<code>os.mkfifo(path[, mode])</code>	Create a FIFO (a named pipe) named <code>path</code> with numeric mode <code>mode</code> . The default mode is 0666 (octal).
<code>os.mknod(filename[, mode=0600, device])</code>	Create a filesystem node (file, device special file or named pipe) named <code>filename</code> .
<code>os.open(file, flags[, mode])</code>	Open the file <code>file</code> and set various flags according to <code>flags</code> and possibly its mode according to <code>mode</code> .
<code>os.openpty()</code>	Open a new pseudo-terminal pair. Return a pair of file descriptors (master, slave) for the pty and the tty, respectively.
<code>os.pathconf(path, name)</code>	Return system configuration information relevant to a named file.
<code>os.pipe()</code>	Create a pipe. Return a pair of file descriptors (<code>r</code> , <code>w</code>) usable for reading and writing, respectively.
<code>os.popen(command[, mode[, bufsize]])</code>	Open a pipe to or from <code>command</code> .
<code>os.read(fd, n)</code>	Read at most <code>n</code> bytes from file descriptor <code>fd</code> . Return a string containing the bytes read. If the end of the file referred to by <code>fd</code> has been reached, an empty string is returned.
<code>os.readlink(path)</code>	Return a string representing the path to which the symbolic link points.
<code>os.remove(path)</code>	Remove the file <code>path</code> .
<code>os.removedirs(path)</code>	Remove directories recursively.
<code>os.rename(src, dst)</code>	Rename the file or directory <code>src</code> to <code>dst</code> .
<code>os.renames(old, new)</code>	Recursive directory or file renaming function.
<code>os.rmdir(path)</code>	Remove the directory <code>path</code>
<code>os.stat(path)</code>	Perform a <code>stat</code> system call on the given path.
<code>os.stat_float_times([newvalue])</code>	Determine whether <code>stat_result</code> represents time stamps as float objects.
<code>os.statvfs(path)</code>	Perform a <code>statvfs</code> system call on the given path.
<code>os.symlink(src, dst)</code>	Create a symbolic link pointing to <code>src</code> named <code>dst</code> .
<code>os.tcgetpgrp(fd)</code>	Return the process group associated with the terminal given by <code>fd</code> (an open file

<code>os.tcsetpgrp(fd, pg)</code>	descriptor as returned by <code>open()</code> . Set the process group associated with the terminal given by <code>fd</code> (an open file descriptor as returned by <code>open()</code>) to <code>pg</code> .
<code>os.tempnam([dir[, prefix]])</code>	Return a unique path name that is reasonable for creating a temporary file.
<code>os.tmpfile()</code>	Return a new file object opened in update mode (<code>w+b</code>).
<code>os.tmpnam()</code>	Return a unique path name that is reasonable for creating a temporary file.
<code>os.ttyname(fd)</code>	Return a string which specifies the terminal device associated with file descriptor <code>fd</code> . If <code>fd</code> is not associated with a terminal device, an exception is raised.
<code>os.unlink(path)</code>	Remove the file path.
<code>os.utime(path, times)</code>	Set the access and modified times of the file specified by path.
<code>os.walk(top[, topdown=True[, onerror=None[, followlinks=False]])]</code>	Generate the file names in a directory tree by walking the tree either top-down or bottom-up.
<code>os.write(fd, str)</code>	Write the string <code>str</code> to file descriptor <code>fd</code> . Return the number of bytes actually written.

Handling Exceptions

Python provides two very important features to handle any unexpected error in your Python programs and to add debugging capabilities in them-

- Exception Handling.
- Assertions

Standard Exceptions

Exception Name	Description
Exception	Base class for all exceptions
StopIteration	Raised when the <code>next()</code> method of an iterator does not point to any object.
SystemExit	Raised by the <code>sys.exit()</code> function.
StandardError	Base class for all built-in exceptions except <code>StopIteration</code> and <code>SystemExit</code> .
ArithmeticError	Base class for all errors that occur for numeric calculation.
OverflowError	Raised when a calculation exceeds maximum limit for a numeric type.
FloatingPointError	Raised when a floating point calculation fails.
ZeroDivisonError	Raised when division or modulo by zero takes place for all numeric types.
AssertionError	Raised in case of failure of the <code>Assert</code> statement.
AttributeError	Raised in case of failure of attribute reference or assignment.
EOFError	Raised when there is no input from either the <code>raw_input()</code> or <code>input()</code> function and the end of file is reached.
ImportError	Raised when an import statement fails.
KeyboardInterrupt	Raised when the user interrupts program execution, usually by pressing <code>Ctrl+c</code> .
LookupError	Base class for all lookup errors.
IndexError	Raised when an index is not found in a sequence.
KeyError	Raised when the specified key is not found in the dictionary.

NameError	Raised when an identifier is not found in the local or global namespace.
UnboundLocalError	Raised when trying to access a local variable in a function or method but no value has been assigned to it.
EnvironmentError	Base class for all exceptions that occur outside the Python environment.
IOError	Raised when an input/ output operation fails, such as the print statement or the open() function when trying to open a file that does not exist.
OSError	Raised for operating system-related errors.
SyntaxError	Raised when there is an error in Python syntax.
IndentationError	Raised when indentation is not specified properly.
SystemError	Raised when the interpreter finds an internal problem, but when this error is encountered the Python interpreter does not exit.
SystemExit	Raised when Python interpreter is quit by using the sys.exit() function. If not handled in the code, causes the interpreter to exit.
TypeError	Raised when an operation or function is attempted that is invalid for the specified data type.
ValueError	Raised when the built-in function for a data type has the valid type of arguments, but the arguments have invalid values specified.
RuntimeError	Raised when a generated error does not fall into any category.
NotImplementedError	Raised when an abstract method that needs to be implemented in an inherited class is not actually implemented.

Assertions in Python

An assertion is a sanity-check that you can turn on or turn off when you are done with your testing of the program. The easiest way to think of an assertion is to liken it to a raise-if statement (or to be more accurate, a raise-if-not statement). An expression is tested, and if the result comes up false, an exception is raised.

Assertions are carried out by the assert statement, the newest keyword to Python, introduced in version 1.5. Programmers often place assertions at the start of a function to check for valid input, and after a function call to check for valid output.

The assert Statement

When it encounters an assert statement, Python evaluates the accompanying expression, which is hopefully true. If the expression is false, Python raises an AssertionError exception.

```
assert Expression[, Arguments]
```

If the assertion fails, Python uses ArgumentExpression as the argument for the AssertionError. AssertionError exceptions can be caught and handled like any other exception, using the try-except statement. If they are not handled, they will terminate the program and produce a traceback.

Example

Here is a function that converts a given temperature from degrees Kelvin to degrees Fahrenheit. Since 0° K is as cold as it gets, the function bails out if it sees a negative temperature –

```
#!/usr/bin/python3

def KelvinToFahrenheit(Temperature):
    assert (Temperature >= 0), "Colder than absolute zero!"
    return ((Temperature-273)*1.8)+32
```

```
print (KelvinToFahrenheit(273))
print (int(KelvinToFahrenheit(505.78)))
print (KelvinToFahrenheit(-5))
```

When the above code is executed, it produces the following result-

```
32.0
451
```

```
Traceback (most recent call last):
File "test.py", line 9, in
print KelvinToFahrenheit(-5)
File "test.py", line 4, in KelvinToFahrenheit
assert (Temperature >= 0),"Colder than absolute zero!"
AssertionError: Colder than absolute zero!
```

Python Exceptions

An exception is an event, which occurs during the execution of a program that disrupts the normal flow of the program's instructions. In general, when a Python script encounters a situation that it cannot cope with, it raises an exception. An exception is a Python object that represents an error.

When a Python script raises an exception, it must either handle the exception immediately otherwise it terminates and quits.

Handling an Exception

If you have some suspicious code that may raise an exception, you can defend your program by placing the suspicious code in a try: block. After the try: block, include an except: statement, followed by a block of code which handles the problem as elegantly as possible.

Here is simple syntax of try....except...else blocks

```
try:
    You do your operations here
    .....

except ExceptionI:
    If there is ExceptionI, then execute this block.

except ExceptionII:
    If there is ExceptionII, then execute this block.
    .....
else:
    If there is no exception then execute this block.
```

Here are few important points about the above-mentioned syntax-

- ✓ A single try statement can have multiple except statements. This is useful when the try block contains statements that may throw different types of exceptions.
- ✓ You can also provide a generic except clause, which handles any exception.
- ✓ After the except clause(s), you can include an else-clause. The code in the else-block executes if the code in the try: block does not raise an exception.

- ✓ The else-block is a good place for code that does not need the try: block's protection.

Example

This example opens a file, writes content in the file and comes out gracefully because there is no problem at all.

```
#!/usr/bin/python3

try:
    fh = open("testfile", "w")
    fh.write("This is my test file for exception handling!!")
except IOError:
    print ("Error: can't find file or read data")
else:
    print ("Written content in the file successfully")
    fh.close()
```

This produces the following result-

```
Written content in the file successfully
```

Example

This example tries to open a file where you do not have the write permission, so it raises an exception

```
#!/usr/bin/python3

try:
    fh = open("testfile", "r")
    fh.write("This is my test file for exception handling!!")
except IOError:
    print ("Error: can't find file or read data")
else:
    print ("Written content in the file successfully")
```

This produces the following result-

```
Error: can't find file or read data
```

The except Clause with No Exceptions

You can also use the except statement with no exceptions defined as follows

```
try:
    You do your operations here
    .....
except:
    If there is any exception, then execute this block.
    .....
else:
    If there is no exception then execute this block.
```

This kind of a try-except statement catches all the exceptions that occur. Using this kind of try-except statement is not considered a good programming practice though, because it catches all exceptions but does not make the programmer identify the root cause of the problem that may occur.

The except Clause with Multiple Exceptions

You can also use the same except statement to handle multiple exceptions as follows

```
try:
    You do your operations here
    .....
except(Exception1[, Exception2[,...ExceptionN]]):
    If there is any exception from the given exception list,
    then execute this block.
    .....
else:
    If there is no exception then execute this block.
```

The try-finally Clause

You can use a finally: block along with a try: block. The finally: block is a place to put any code that must execute, whether the try-block raised an exception or not. The syntax of the try-finally statement is this

```
try:
    You do your operations here;
    .....
    Due to any exception, this may be skipped.
finally:
    This would always be executed.
    .....
```

Note: You can provide except clause(s), or a finally clause, but not both. You cannot use else clause as well along with a finally clause.

```
#!/usr/bin/python3

try:
    fh = open("testfile", "w")
    fh.write("This is my test file for exception handling!!")
finally:
    print ("Error: can't find file or read data")
    fh.close()
```

If you do not have permission to open the file in writing mode, then this will produce the following result-

```
Error: can't find file or read data
```

Same example can be written more cleanly as follows-

```
#!/usr/bin/python3
try:
    fh = open("testfile", "w")
    try:
        fh.write("This is my test file for exception handling!!")
    finally:
        print ("Going to close the file")
        fh.close()
except IOError:
    print ("Error: can't find file or read data")
```

When an exception is thrown in the try block, the execution immediately passes to the finally block. After all the statements in the finally block are executed, the exception is raised again and is handled in the except statements if present in the next higher layer of the try-except statement.

Argument of an Exception

An exception can have an argument, which is a value that gives additional information about the problem. The contents of the argument vary by exception. You capture an exception's argument by supplying a variable in the except clause as follows

```
try:
    You do your operations here
    .....
except ExceptionType as Argument:
    You can print value of Argument here...
```

If you write the code to handle a single exception, you can have a variable follow the name of the exception in the except statement. If you are trapping multiple exceptions, you can have a variable follow the tuple of the exception. This variable receives the value of the exception mostly containing the cause of the exception. The variable can receive a single value or multiple values in the form of a tuple. This tuple usually contains the error string, the error number, and an error location.

Following is an example for a single exception-

```
#!/usr/bin/python3
# Define a function here.

def temp_convert(var):
    try:
        returnint(var)
    except ValueError as Argument:
        print("The argument does not contain numbers\n",Argument)

# Call above function here.
temp_convert("xyz")
```

This produces the following result-

```
The argument does not contain numbers invalid literal for int() with base 10: 'xyz'
```

Raising an Exception

You can raise exceptions in several ways by using the raise statement. The general syntax for the raise statement is as follows-

```
raise [Exception [, args [, traceback]]]
```

Here, Exception is the type of exception (for example, NameError) and argument is a value for the exception argument. The argument is optional; if not supplied, the exception argument is None.

The final argument, traceback, is also optional (and rarely used in practice), and if present, is the traceback object used for the exception.

Example

An exception can be a string, a class or an object. Most of the exceptions that the Python core raises are classes, with an argument that is an instance of the class. Defining new exceptions is quite easy and can be done as follows

```
def functionName( level ):  
    if level <1:  
        raise Exception(level)  
        # The code below to this would not be executed  
        # if we raise the exception  
    return level
```

Note: In order to catch an exception, an "except" clause must refer to the same exception thrown either as a class object or a simple string. For example, to capture the above exception, we must write the except clause as follows

```
try:  
    Business Logic here...  
except Exception as e:  
    Exception handling here using e.args...  
else:  
    Rest of the code here...
```

User-Defined Exceptions

Python also allows you to create your own exceptions by deriving classes from the standard built-in exceptions. Here is an example related to `RuntimeError`. Here, a class is created that is sub-classed from `RuntimeError`. This is useful when you need to display more specific information when an exception is caught.

In the try block, the user-defined exception is raised and caught in the except block. The variable `e` is used to create an instance of the class `Networkerror`.

```
class Networkerror(RuntimeError):  
    def __init__(self, arg):  
        self.args = arg
```

So once you have defined the above class, you can raise the exception as follows

```
try:  
    raise Networkerror("Bad hostname")  
except Networkerror,e:  
    print e.args
```

Classes and Objects

Overview of OOP Terminology

- **Class:** A user-defined prototype for an object that defines a set of attributes that characterize any object of the class. The attributes are data members (class variables and instance variables) and methods, accessed via dot notation.
- **Class variable:** A variable that is shared by all instances of a class. Class variables are defined within a class but outside any of the class's methods. Class variables are not used as frequently as instance variables are.
- **Data member:** A class variable or instance variable that holds data associated with a class and its objects.

- Function overloading: The assignment of more than one behavior to a particular function. The operation performed varies by the types of objects or arguments involved.
- Instance variable: A variable that is defined inside a method and belongs only to the current instance of a class.
- Inheritance: The transfer of the characteristics of a class to other classes that are derived from it.
- Instance: An individual object of a certain class. An object obj that belongs to a class Circle, for example, is an instance of the class Circle.
- Instantiation: The creation of an instance of a class.
- Method : A special kind of function that is defined in a class definition.
- Object: A unique instance of a data structure that is defined by its class. An object comprises both data members (class variables and instance variables) and methods.
- Operator overloading: The assignment of more than one function to a particular operator.

Creating Classes

The class statement creates a new class definition. The name of the class immediately follows the keyword class followed by a colon as follows

```
class ClassName:
    'Optional class documentation string'
    class_suite
```

- The class has a documentation string, which can be accessed via **ClassName.__doc__**.
- The class_suite consists of all the component statements defining class members, data attributes and functions.

Following is an example of a simple Python class

```
class Employee:
    'Common base class for all employees'
    empCount = 0

    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

        Employee.empCount += 1

    def displayCount(self):
        print "Total Employee %d" % Employee.empCount

    def displayEmployee(self):
        print ("Name : ", self.name, " , Salary: ", self.salary)
```

The variable empCount is a class variable whose value is shared among all the instances of a in this class. This can be accessed as Employee.empCount from inside the class or outside the class.

- The first method __init__() is a special method, which is called class constructor or initialization method that Python calls when you create a new instance of this class.
- You declare other class methods like normal functions with the exception that the first argument to each method is self. Python adds the self argument to the list for you; you do not need to include it when you call the methods.

Creating Instance Objects

To create instances of a class, you call the class using class name and pass in whatever arguments its `__init__` method accepts.

This would create first object of Employee class

```
emp1 = Employee("Zara", 2000)
```

This would create second object of Employee class

```
emp2 = Employee("Manni", 5000)
```

Accessing Attributes

You access the object's attributes using the dot operator with object. Class variable would be accessed using class name as follows

```
emp1.displayEmployee()  
emp2.displayEmployee()  
print ("Total Employee %d" % Employee.empCount)
```

Now, putting all the concepts together-

```
#!/usr/bin/python3  
  
class Employee:  
    'Common base class for all employees'  
    empCount = 0  
  
    def __init__(self, name, salary):  
        self.name = name  
        self.salary = salary  
        Employee.empCount += 1  
  
    def displayCount(self):  
        print ("Total Employee %d" % Employee.empCount)  
  
    def displayEmployee(self):  
        print ("Name : ", self.name, ", Salary: ", self.salary)  
  
#This would create first object of Employee class"  
emp1 = Employee("Zara", 2000)  
  
#This would create second object of Employee class"  
emp2 = Employee("Manni", 5000)  
emp1.displayEmployee()  
emp2.displayEmployee()  
print ("Total Employee %d" % Employee.empCount)
```

When the above code is executed, it produces the following result-

```
Name : Zara ,Salary: 2000
Name : Manni ,Salary: 5000
Total Employee 2
```

You can add, remove, or modify attributes of classes and objects at any time

```
emp1.salary = 7000    # Add an 'salary' attribute.
emp1.name = 'xyz'      # Modify 'age' attribute.
del emp1.salary        # Delete 'age' attribute.
```

Instead of using the normal statements to access attributes, you can use the following functions-

- The `getattr(obj, name[, default])`: to access the attribute of object.
- The `hasattr(obj,name)`: to check if an attribute exists or not.
- The `setattr(obj,name,value)`: to set an attribute. If attribute does not exist, then it would be created.
- The `delattr(obj, name)`: to delete an attribute.

```
hasattr(emp1, 'salary')      # Returns true if 'salary' attribute exists
getattr(emp1, 'salary')      # Returns value of 'salary' attribute
setattr(emp1, 'salary', 7000) # Set attribute 'age' at 8
delattr(emp1, 'salary')      # Delete attribute 'age'
```

Built-In Class Attributes

Every Python class keeps the following built-in attributes and they can be accessed using dot operator like any other attribute –

- `__dict__`: Dictionary containing the class's namespace.
- `__doc__`: Class documentation string or none, if undefined.
- `__name__`: Class name.
- `__module__`: Module name in which the class is defined. This attribute is "`__main__`" in interactive mode.
- `__bases__`: A possibly empty tuple containing the base classes, in the order of their occurrence in the base class list.

For the above class let us try to access all these attributes-

```
#!/usr/bin/python3
class Employee:
    'Common base class for all employees'
    empCount = 0

    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
        Employee.empCount += 1

    def displayCount(self):
        print ("Total Employee %d" % Employee.empCount)

    def displayEmployee(self):
        print ("Name : ", self.name, ", Salary: ", self.salary)
emp1 = Employee("Zara", 2000)
```

```
emp2 = Employee("Manni", 5000)

print ("Employee.__doc__:", Employee.__doc__)
print ("Employee.__name__:", Employee.__name__)
print ("Employee.__module__:", Employee.__module__)
print ("Employee.__bases__:", Employee.__bases__)
print ("Employee.__dict__:", Employee.__dict__)
```

When the above code is executed, it produces the following result-

```
Employee.__doc__: Common base class for all employees
Employee.__name__: Employee
Employee.__module__: __main__
Employee.__bases__: (,)
Employee.__dict__: {'displayCount': , '__module__': '__main__', '__doc__': 'Common base class for all employees',
'empCount': 2, '__init__': , 'displayEmployee': , '__weakref__': , '__dict__': }
```

Destroying Objects (Garbage Collection)

Python deletes unneeded objects (built-in types or class instances) automatically to free the memory space. The process by which Python periodically reclaims blocks of memory that no longer are in use is termed as Garbage Collection.

Python's garbage collector runs during program execution and is triggered when an object's reference count reaches zero. An object's reference count changes as the number of aliases that point to it changes.

An object's reference count increases when it is assigned a new name or placed in a container (list, tuple, or dictionary). The object's reference count decreases when it is deleted with `del`, its reference is reassigned, or its reference goes out of scope. When an object's reference count reaches zero, Python collects it automatically.

```
a = 40          # Create object <40>
b = a          # Increase ref. count of <40>
c = [b]        # Increase ref. count of <40>
del a          # Decrease ref. count of <40>
b = 100        # Decrease ref. count of <40>
c[0] = -1      # Decrease ref. count of <40>
```

You normally will not notice when the garbage collector destroys an orphaned instance and reclaims its space. However, a class can implement the special method `__del__()`, called a destructor, that is invoked when the instance is about to be destroyed. This method might be used to clean up any non-memory resources used by an instance.

Example

This `__del__()` destructor prints the class name of an instance that is about to be destroyed.

```
#!/usr/bin/python3
class Point:
    def __init__( self, x=0, y=0):
        self.x = x
        self.y = y

    def __del__(self):
        class_name = self.__class__.__name__
        print (class_name, "destroyed")
```

```
pt1 = Point()
pt2 = pt1
pt3 = pt1
print (id(pt1), id(pt2), id(pt3) # prints the ids of the objects)
del pt1
del pt2
del pt3
```

When the above code is executed, it produces the following result-

```
3083401324 3083401324 3083401324
Point destroyed
```

Note: Ideally, you should define your classes in a separate file, then you should import them in your main program file using import statement.

In the above example, assuming definition of a Point class is contained in point.py and there is no other executable code in it.

```
#!/usr/bin/python3
import point
p1=point.Point()
```

Class Inheritance

Instead of starting from a scratch, you can create a class by deriving it from a pre-existing class by listing the parent class in parentheses after the new class name.

The child class inherits the attributes of its parent class, and you can use those attributes as if they were defined in the child class. A child class can also override data members and methods from the parent.

Syntax

Derived classes are declared much like their parent class; however, a list of base classes to inherit from is given after the class name –

```
class SubClassName (ParentClass1[, ParentClass2, ...]):
    'Optional class documentation string'
    class_suite
```

Example

```
#!/usr/bin/python3

class Parent: # define parent class
    parentAttr = 100

    def __init__(self):
        print ("Calling parent constructor")

    def parentMethod(self):
        print ('Calling parent method')

    def setAttr(self, attr):
```

```

    Parent.parentAttr = attr

def getAttr(self):
    print ("Parent attribute :", Parent.parentAttr)
class Child(Parent): # define child class

def __init__(self):
    print ("Calling child constructor")

def childMethod(self):
    print ('Calling child method')

c = Child() # instance of child
c.childMethod() # child calls its method
c.parentMethod() # calls parent's method
c.setAttr(200) # again call parent's method
c.getAttr() # again call parent's method

```

When the above code is executed, it produces the following result-

```

Calling child constructor
Calling child method
Calling parent method
Parent attribute : 200

```

In a similar way, you can drive a class from multiple parent classes as follows

```

class A: # define your class A
    ....
class B: # define your calss B
    ....
class C(A, B): # subclass of A and B
    ....

```

You can use `issubclass()` or `isinstance()` functions to check a relationship of two classes and instances.

- The `issubclass(sub, sup)` boolean function returns True, if the given subclass `sub` is indeed a subclass of the superclass `sup`.
- The `isinstance(obj, Class)` boolean function returns True, if `obj` is an instance of class `Class` or is an instance of a subclass of `Class`.

Overriding Methods

You can always override your parent class methods. One reason for overriding parent's methods is that you may want special or different functionality in your subclass.

Example

```

#!/usr/bin/python3
class Parent:          # define parent class

    def myMethod(self):
        print ('Calling parent method')
class Child(Parent): # define child class

```

```
def myMethod(self):
    print ('Calling child method')
```

```
c = Child()           # instance of child
c.myMethod()          # child calls overridden method
```

When the above code is executed, it produces the following result-

```
Calling child method
```

Base Overloading Methods

The following table lists some generic functionality that you can override in your own classes-

Method	Description	Sample Call
<code>__init__ (self [,args...])</code>	Constructor (with any optional arguments)	<code>obj = className(args)</code>
<code>__del__(self)</code>	Destructor, deletes an object	<code>del obj</code>
<code>__repr__(self)</code>	Evaluatable string representation	<code>repr(obj)</code>
<code>__str__(self)</code>	Printable string representation	<code>str(obj)</code>
<code>__cmp__(self, x)</code>	Object comparison	<code>cmp(obj, x)</code>

Data Hiding

An object's attributes may or may not be visible outside the class definition. You need to name attributes with a double underscore prefix, and those attributes then will not be directly visible to outsiders.

Example

```
#!/usr/bin/python3
class JustCounter:
    __secretCount = 0

    def count(self):
        self.__secretCount += 1
        print (self.__secretCount)

counter = JustCounter()
counter.count()
counter.count()
print (counter.__secretCount)
```

When the above code is executed, it produces the following result-

```
1
2
Traceback (most recent call last):
File "test.py", line 12, in <module>
print counter.__secretCount
AttributeError: JustCounter instance has no attribute '__secretCount'
```

Python protects those members by internally changing the name to include the class name. You can access such attributes as `object._className__attrName`. If you would replace your last line as following, then it works for you-

```
.....  
print (counter._JustCounter__secretCount)
```

When the above code is executed, it produces the following result

```
1  
2  
2
```

Regular Expressions

Basic patterns that match single chars

- a, X, 9, < -- ordinary characters just match themselves exactly.
- . (a period) -- matches any single character except newline '\n'
- \w -- matches a "word" character: a letter or digit or underbar [a-zA-Z0-9_].
- \W -- matches any non-word character.
- \b -- boundary between word and non-word
- \s -- matches a single whitespace character -- space, newline, return, tab
- \S -- matches any non-whitespace character.
- \t, \n, \r -- tab, newline, return
- \d -- decimal digit [0-9]
- ^ -- matches start of the string
- \$ -- match the end of the string
- \ -- inhibit the "specialness" of a character.

The match Function

This function attempts to match RE pattern to string with optional flags.

```
re.match(pattern, string, flags=0)
```

Parameter	Description
pattern	This is the regular expression to be matched.
string	This is the string, which would be searched to match the pattern at the beginning of string.
flags	You can specify different flags using bitwise OR (). These are modifiers, which are listed in the table below.

The re.match function returns a match object on success, None on failure. We use group(num) or groups() function of match object to get matched expression.

- group(num=0) This method returns entire match (or specific subgroup num)
- groups() This method returns all matching subgroups in a tuple (empty if there weren't any)

Example

```
#!/usr/bin/python3  
import re  
line = "Cats are smarter than dogs"  
matchObj = re.match( r'(.*) are (.*?) .*', line, re.M|re.I)  
  
if matchObj:  
    print ("matchObj.group() : ", matchObj.group())
```

```
print ("matchObj.group(1) : ", matchObj.group(1))
print ("matchObj.group(2) : ", matchObj.group(2))
else:
    print ("No match!!")
```

When the above code is executed, it produces the following result

```
matchObj.group() : Cats are smarter than dogs
matchObj.group(1) : Cats
matchObj.group(2) : smarter
```

The search Function

This function searches for first occurrence of RE pattern within the string, with optional flags.

Here is the syntax for this function

```
re.search(pattern, string, flags=0)
```

Parameter	Description
pattern	This is the regular expression to be matched.
string	This is the string, which would be searched to match the pattern anywhere in the string.
flags	You can specify different flags using bitwise OR (). These are modifiers, which are listed in the table below.

The re.search function returns a match object on success, none on failure. We use group(num) or groups() function of match object to get the matched expression.

- group(num=0) This method returns entire match (or specific subgroup num)
- groups() This method returns all matching subgroups in a tuple (empty if there weren't any)

Example

```
#!/usr/bin/python3
import re
line = "Cats are smarter than dogs";
searchObj = re.search( r'(.*) are (.*?) .*', line, re.M|re.I)
if searchObj:
    print ("searchObj.group() : ", searchObj.group())
    print ("searchObj.group(1) : ", searchObj.group(1))
    print ("searchObj.group(2) : ", searchObj.group(2))
else:
    print ("Nothing found!!")
```

When the above code is executed, it produces following result

```
matchObj.group() : Cats are smarter than dogs
matchObj.group(1) : Cats
matchObj.group(2) : smarter
```

Matching Versus Searching

Python offers two different primitive operations based on regular expressions :match checks for a match only at the beginning of the string, while search checks for a match anywhere in the string (this is what Perl does by default).

```
#!/usr/bin/python3
import re
```

```

line = "Cats are smarter than dogs";
matchObj = re.match( r'dogs', line, re.M|re.I)

if matchObj:
    print ("match --> matchObj.group() : ", matchObj.group())
else:
    print ("No match!!")
    searchObj = re.search( r'dogs', line, re.M|re.I)

if searchObj:
    print ("search --> searchObj.group() : ", searchObj.group())
else:
    print ("Nothing found!!")

```

When the above code is executed, it produces the following result-

```

No match!!
search --> matchObj.group() : dogs

```

Search and Replace

One of the most important re methods that use regular expressions is sub.

```
re.sub(pattern, repl, string, max=0)
```

This method replaces all occurrences of the RE pattern in string with repl, substituting all occurrences unless max is provided. This method returns modified string.

```

#!/usr/bin/python3
import re
phone = "2004-959-559 # This is Phone Number"

# Delete Python-style comments
num = re.sub(r'#. *$', "", phone)
print ("Phone Num : ", num)

# Remove anything other than digits
num = re.sub(r'\D', "", phone)
print ("Phone Num : ", num)

```

When the above code is executed, it produces the following result-

```

Phone Num : 2004-959-559
Phone Num : 2004959559

```

Regular Expression Modifiers: Option Flags

Regular expression literals may include an optional modifier to control various aspects of matching. The modifiers are specified as an optional flag. You can provide multiple modifiers using exclusive OR (|), as shown previously and may be represented by one of these-

Modifier	Description
re.I	Performs case-insensitive matching.
re.L	Interprets words according to the current locale. This interpretation affects the alphabetic group (\w and \W), as well as word boundary behavior (\b and \B).

re.M	Makes \$ match the end of a line (not just the end of the string) and makes ^ match the start of any line (not just the start of the string).
re.S	Makes a period (dot) match any character, including a newline.
re.U	Interprets letters according to the Unicode character set. This flag affects the behavior of \w, \W, \b, \B.
re.X	Permits "cater" regular expression syntax. It ignores whitespace (except inside a set [] or when escaped by a backslash) and treats unescaped # as a comment marker.

Regular Expression Patterns

Except for the control characters, (+ ? . * ^ \$ () [] { } | \), all characters match themselves. You can escape a control character by preceding it with a backslash.

The following table lists the regular expression syntax that is available in Python-

Pattern	Description
^	Matches beginning of line.
\$	Matches end of line.
.	Matches any single character except newline. Using m option allows it to match newline as well.
[...]	Matches any single character in brackets.
[^...]	Matches any single character not in brackets
re*	Matches 0 or more occurrences of preceding expression.
re+	Matches 1 or more occurrence of preceding expression.
re?	Matches 0 or 1 occurrence of preceding expression.
re{ n}	Matches exactly n number of occurrences of preceding expression.
re{ n,}	Matches n or more occurrences of preceding expression.
re{ n, m}	Matches at least n and at most m occurrences of preceding expression.
a b	Matches either a or b.
(re)	Groups regular expressions and remembers matched text.
(?imx)	Temporarily toggles on i, m, or x options within a regular expression. If in parentheses, only that area is affected.
(?-imx)	Temporarily toggles off i, m, or x options within a regular expression. If in parentheses, only that area is affected.
(?: re)	Groups regular expressions without remembering matched text.
(?imx: re)	Temporarily toggles on i, m, or x options within parentheses.
(?-imx: re)	Temporarily toggles off i, m, or x options within parentheses.
(?#...)	Comment.
(?= re)	Specifies position using a pattern. Does not have a range.
(?! re)	Specifies position using pattern negation. Does not have a range.
(?> re)	Matches independent pattern without backtracking.
\w	Matches word characters.
\W	Matches nonword characters.
\s	Matches whitespace. Equivalent to [\t\n\r\f].
\S	Matches nonwhitespace.
\d	Matches digits. Equivalent to [0-9].
\D	Matches nondigits.
\A	Matches beginning of string.
\Z	Matches end of string. If a newline exists, it matches just before newline.
\z	Matches end of string.
\G	Matches point where last match finished.

<code>\b</code>	Matches word boundaries when outside brackets. Matches backspace (0x08) when inside brackets.
<code>\B</code>	Matches nonword boundaries.
<code>\n, \t, etc.</code>	Matches newlines, carriage returns, tabs, etc.
<code>\1...\9</code>	Matches nth grouped subexpression.
<code>\10</code>	Matches nth grouped subexpression if it matched already.

Character classes

Example	Description
<code>[Pp]ython</code>	Match "Python" or "python"
<code>rub[ye]</code>	Match "ruby" or "rube"
<code>[aeiou]</code>	Match any one lowercase vowel
<code>[0-9]</code>	Match any digit; same as <code>[0123456789]</code>
<code>[a-z]</code>	Match any lowercase ASCII letter
<code>[A-Z]</code>	Match any uppercase ASCII letter
<code>[a-zA-Z0-9]</code>	Match any of the above
<code>[^aeiou]</code>	Match anything other than a lowercase vowel
<code>[^0-9]</code>	Match anything other than a digit

Special Character Classes

Example	Description
<code>.</code>	Match any character except newline
<code>\d</code>	Match a digit: <code>[0-9]</code>
<code>\D</code>	Match a nondigit: <code>[^0-9]</code>
<code>\s</code>	Match a whitespace character: <code>[\t\r\n\f]</code>
<code>\S</code>	Match nonwhitespace: <code>[^\t\r\n\f]</code>
<code>\w</code>	Match a single word character: <code>[A-Za-z0-9_]</code>
<code>\W</code>	Match a nonword character: <code>[^A-Za-z0-9_]</code>

Repetition Cases

Example	Description
<code>ruby?</code>	Match "rub" or "ruby": the y is optional
<code>ruby*</code>	Match "rub" plus 0 or more ys
<code>ruby+</code>	Match "rub" plus 1 or more ys
<code>\d{3}</code>	Match exactly 3 digits
<code>\d{3,}</code>	Match 3 or more digits
<code>\d{3,5}</code>	Match 3, 4, or 5 digits

CGI Programming

CGI

The Common Gateway Interface, or CGI, is a standard for external gateway programs to interface with information servers such as HTTP servers.

Web Browsing

To understand the concept of CGI, let us see what happens when we click a hyperlink to browse a particular web page or URL.

- Your browser contacts the HTTP web server and demands for the URL, i.e., filename.
- The web server parses the URL and looks for the filename. If it finds the particular file, then it sends it back to the browser, otherwise sends an error message indicating that you requested a wrong file.
- The web browser takes response from the web server and displays either, the received file or error message.

However, it is possible to set up the HTTP server so that whenever a file in a certain directory is requested that file is not sent back. Instead, it is executed as a program, and whatever that output of the program, is sent back for your browser to display. This function is called the Common Gateway Interface or CGI and the programs are called CGI scripts. These CGI programs can be Python Script, PERL Script, Shell Script, C or C++ program, etc.

Web Server Support and Configuration

Before you proceed with CGI Programming, make sure that your Web Server supports CGI and it is configured to handle CGI Programs. All the CGI Programs, which are to be executed by the HTTP server, are kept in a pre-configured directory. This directory is called CGI Directory and by convention it is named as /var/www/cgi-bin. By convention, CGI files have extension as .cgi, but you can keep your files with python extension .py as well.

By default, the Linux server is configured to run only the scripts in the cgi-bin directory in /var/www. If you want to specify any other directory to run your CGI scripts, comment the following lines in the httpd.conf file –

```
<Directory "/var/www/cgi-bin">
    AllowOverride None
    Options ExecCGI
    Order allow,deny
    Allow from all
</Directory>

<Directory "/var/www/cgi-bin">
    Options All
</Directory>
```

The following line should also be added for apache server to treat .py file as cgi script.

```
AddHandler cgi-script .py
```

Here, we assume that you have Web Server up and running successfully and you are able to run any other CGI program like Perl or Shell, etc.

First CGI Program

Here is a simple link, which is linked to a CGI script called hello.py. This file is kept in /var/www/cgi-bin directory and it has the following content. Before running your CGI program, make sure you have changed the mode of file using chmod 755 hello.py, the UNIX command to make file executable.

```
#!/usr/bin/python3
print ("Content-type:text/html")
print()
print("<html>")
print('<head>')
print('<title>Hello Word - First CGI Program</title>')
print('</head>')
```

```
print('<body>')
print('<h2>Hello Word! This is my first CGI program</h2>')
print('</body>')
print('</html>')
```

Note: First line in the script must be the path to Python executable. In Linux, it should be

```
#!/usr/bin/python3
```

Enter the following URL in your browser -

```
http://localhost/cgi-bin/hello.py
```

Output:

```
Hello Word! This is my first CGI program
```

HTTP Header

The line Content-type:text/html\r\n\r\n is part of HTTP header which is sent to the browser to understand the content. All the HTTP header will be in the following form-

```
HTTP Field Name: Field Content
```

For Example

```
Content-type: text/html\r\n\r\n
```

There are few other important HTTP headers, which you will use frequently in your CGI Programming.

Header	Description
Content-type:	A MIME string defining the format of the file being returned. Example is Content-type:text/html
Expires:	Date The date the information becomes invalid. It is used by the browser to decide when a page needs to be refreshed. A valid date string is in the format 01 Jan 1998 12:00:00 GMT.
Location:	URL The URL that is returned instead of the URL requested. You can use this field to redirect a request to any file.
Last-modified:	Date The date of last modification of the resource.
Content-length:	N The length, in bytes, of the data being returned. The browser uses this value to report the estimated download time for a file.
Set-Cookie:	String Set the cookie passed through the string

CGI Environment Variables

All the CGI programs have access to the following environment variables. These variables play an important role while writing any CGI program.

Variable Name	Description
CONTENT_TYPE	The data type of the content. Used when the client is sending attached content to the server. For example, file upload.
CONTENT_LENGTH	The length of the query information. It is available only for POST requests.
HTTP_COOKIE	Returns the set cookies in the form of key & value pair.

HTTP_USER_AGENT	The User-Agent request-header field contains information about the user agent originating the request. It is name of the web browser.
PATH_INFO	The path for the CGI script.
QUERY_STRING	The URL-encoded information that is sent with GET method request.
REMOTE_ADDR	The IP address of the remote host making the request. This is useful logging or for authentication.
REMOTE_HOST	The fully qualified name of the host making the request. If this information is not available, then REMOTE_ADDR can be used to get IR address.
REQUEST_METHOD	The method used to make the request. The most common methods are GET and POST.
SCRIPT_FILENAME	The full path to the CGI script.
SCRIPT_NAME	The name of the CGI script.
SERVER_NAME	The server's hostname or IP Address
SERVER_SOFTWARE	The name and version of the software the server is running.

Here is a small CGI program to list out all the CGI variables. Click this link to see the result [Get Environment](#).

```
#!/usr/bin/python3
import os
print ("Content-type: text/html")
print ()
print ("<font size=+1>Environment</font><\br>");
for param in os.environ.keys():
    print ("<b>%20s</b>: %s<\br>" % (param, os.environ[param]))
```

Passing Information using GET method

The GET method sends the encoded user information appended to the page request. The page and the encoded information are separated by the ? character as follows

```
http://www.test.com/cgi-in/hello.py?key1=value1&key2=value2
```

- The GET method is the default method to pass information from the browser to the web server and it produces a long string that appears in your browser's Location:box.
- Never use GET method if you have password or other sensitive information to pass to the server.
- The GET method has size limitation: only 1024 characters can be sent in a request string.
- The GET method sends information using QUERY_STRING header and will be accessible in your CGI Program through QUERY_STRING environment variable.

You can pass information by simply concatenating key and value pairs along with any URL or you can use HTML <FORM> tags to pass information using GET method.

Simple URL Example – Get Method

Here is a simple URL, which passes two values to hello_get.py program using GET method.

```
/cgi-bin/hello_get.py?first_name=Malhar&last_name=Lathkar
```

Given below is the hello_get.py script to handle the input given by web browser. We are going to use the cgi module, which makes it very easy to access the passed information-

```
#!/usr/bin/python3
# Import modules for CGI handling
import cgi, cgitb

# Create instance of FieldStorage
form = cgi.FieldStorage()

# Get data from fields
first_name = form.getvalue('first_name')
last_name = form.getvalue('last_name')
print ("Content-type:text/html")
print()
print("<html>")
print("<head>")
print("<title>Hello - Second CGI Program</title>")
print("</head>")
print("<body>")
print("<h2>Hello %s %s</h2>" % (first_name, last_name))
print("</body>")
print("</html>")
```

This would generate the following result-

```
Hello ZARA ALI
```

Simple FORM Example – GET Method

This example passes two values using HTML FORM and submit button. We use the same CGI script hello_get.py to handle this input.

```
<form action="/cgi-bin/hello_get.py" method="get">
First Name: <input type="text" name="first_name"> <br />
Last Name: <input type="text" name="last_name" />
<input type="submit" value="Submit" />
</form>
```

Passing Information Using POST Method

A generally more reliable method of passing information to a CGI program is the POST method. This packages the information in exactly the same way as the GET methods, but instead of sending it as a text string after a ? in the URL, it sends it as a separate message. This message comes into the CGI script in the form of the standard input.

Given below is same hello_get.py script, which handles GET as well as the POST method.

```
#!/usr/bin/python3
# Import modules for CGI handling
import cgi, cgitb

# Create instance of FieldStorage
form = cgi.FieldStorage()

# Get data from fields
```

```

first_name = form.getvalue('first_name')
last_name = form.getvalue('last_name')
print ("Content-type:text/html")
print()
print("<html>")
print("<head>")
print("<title>Hello - Second CGI Program</title>")
print("</head>")
print("<body>")
print("<h2>Hello %s %s</h2>" % (first_name, last_name))
print("</body>")
print("</html>")

```

Let us again take the same example as above, which passes two values using the HTML FORM and the submit button. We use the same CGI script `hello_get.py` to handle this input.

```

<form action="/cgi-bin/hello_get.py" method="post">
First Name: <input type="text" name="first_name"><br />
Last Name: <input type="text" name="last_name" />
<input type="submit" value="Submit" />
</form>

```

Passing Checkbox Data to CGI Program

Checkboxes are used when more than one option is required to be selected.

```

<form action="/cgi-bin/checkbox.py" method="POST" target="_blank">
<input type="checkbox" name="maths" value="on" /> Maths
<input type="checkbox" name="physics" value="on" /> Physics
<input type="submit" value="Select Subject" />
</form>

```

Given below is the `checkbox.cgi` script to handle the input given by web browser for checkbox button.

```

#!/usr/bin/python3
# Import modules for CGI handling
import cgi, cgitb

# Create instance of FieldStorage
form = cgi.FieldStorage()

# Get data from fields
if form.getvalue('maths'):
    math_flag = "ON"
else:
    math_flag = "OFF"

if form.getvalue('physics'):
    physics_flag = "ON"
else:
    physics_flag = "OFF"

```

```

print ("Content-type:text/html")
print()
print("<html>")
print("<head>")
print("<title>Checkbox - Third CGI Program</title>")
print("</head>")
print("<body>")
print("<h2> CheckBox Maths is : %s</h2>" % math_flag)
print("<h2> CheckBox Physics is : %s</h2>" % physics_flag)
print("</body>")
print("</html>")

```

Passing Radio Button Data to CGI Program

Radio Buttons are used when only one option is required to be selected.

```

<form action="/cgi-bin/radiobutton.py" method="post" target="_blank">
<input type="radio" name="subject" value="maths" /> Maths
<input type="radio" name="subject" value="physics" /> Physics
<input type="submit" value="Select Subject" />
</form>

```

Below is radiobutton.py script to handle input given by web browser for radio button-

```

#!/usr/bin/python3
# Import modules for CGI handling
import cgi, cgitb

# Create instance of FieldStorage
form = cgi.FieldStorage()

# Get data from fields
if form.getvalue('subject'):
    subject = form.getvalue('subject')
else:
    subject = "Not set"

print "Content-type:text/html"
print()
print("<html>")
print("<head>")
print("<title>Radio - Fourth CGI Program</title>")
print("</head>")
print("<body>")
print("<h2> Selected Subject is %s</h2>" % subject)
print("</body>")
print("</html>")

```

Passing Text Area Data to CGI Program

TEXTAREA element is used when multiline text has to be passed to the CGI Program.

```
<form action="/cgi-bin/textarea.py" method="post" target="_blank">
<textarea name="textcontent" cols="40" rows="4">
Type your text here...
</textarea>
<input type="submit" value="Submit" />
</form>
```

Given below is the textarea.cgi script to handle input given by web browser-

```
#!/usr/bin/python3
# Import modules for CGI handling
import cgi, cgitb

# Create instance of FieldStorage
form = cgi.FieldStorage()

# Get data from fields
if form.getvalue('textcontent'):
    text_content = form.getvalue('textcontent')
else:
    text_content = "Not entered"

print "Content-type:text/html"
print()
print("<html>")
print("<head>;")
print("<title>Text Area - Fifth CGI Program</title>")
print("</head>")
print("<body>")
print("<h2> Entered Text Content is %s</h2>" % text_content)
print("</body>")
```

Passing Drop Down Box Data to CGI Program

The Drop-Down Box is used when we have many options available.

```
<form action="/cgi-bin/dropdown.py" method="post" target="_blank">
<select name="dropdown">
<option value="Maths" selected>Maths</option>
<option value="Physics">Physics</option>
</select>
<input type="submit" value="Submit"/>
</form>
```

Following is the dropdown.py script to handle the input given by web browser.

```
#!/usr/bin/python3
# Import modules for CGI handling
import cgi, cgitb
```

```

# Create instance of FieldStorage
form = cgi.FieldStorage()

# Get data from fields
if form.getvalue('dropdown'):
    subject = form.getvalue('dropdown')
else:
    subject = "Not entered"

print "Content-type:text/html"
print()
print("<html>")
print("<head>")
print("<title>Dropdown Box - Sixth CGI Program</title>")
print("</head>")
print("<body>")
print("<h2> Selected Subject is %s</h2>" % subject)
print("</body>")
print("</html>")

```

Using Cookies in CGI

HTTP protocol is a stateless protocol. For a commercial website, it is required to maintain session information among different pages. For example, one user registration ends after completing many pages.

In many situations, using cookies is the most efficient method of remembering and tracking preferences, purchases, commissions, and other information required for better visitor experience or site statistics.

Cookies are a plain text data record of five variable-length fields-

- Expires: The date the cookie will expire. If this is blank, the cookie will expire when the visitor quits the browser.
- Domain: The domain name of your site.
- Path: The path to the directory or web page that sets the cookie. This may be blank if you want to retrieve the cookie from any directory or page.
- Secure: If this field contains the word "secure", then the cookie may only be retrieved with a secure server. If this field is blank, no such restriction exists.
- Name=Value: Cookies are set and retrieved in the form of key and value pairs.

Setting up Cookies

It is very easy to send cookies to the browser. These cookies are sent along with the HTTP Header before the Content-type field is sent. Assuming you want to set the User ID and Password as cookies, Cookies are set as follows-

```

#!/usr/bin/python3
print("Set-Cookie:UserID=XYZ;\r\n")
print("Set-Cookie:Password=XYZ123;\r\n")
print("Set-Cookie:Expires=Tuesday, 31-Dec-2007 23:12:40 GMT";\r\n")
print("Set-Cookie:Domain=www.tutorialspoint.com;\r\n")
print("Set-Cookie:Path=/perl;\n")
print("Content-type:text/html\r\n\r\n")
.....Rest of the HTML Content....

```

From this example, you must have understood how to set cookies. We use Set-Cookie HTTP header to set the cookies.

It is optional to set cookies attributes like Expires, Domain, and Path. It is notable that the cookies are set before sending the magic line "Content-type:text/html\r\n\r\n".

Retrieving Cookies

It is very easy to retrieve all the set cookies. Cookies are stored in CGI environment variable HTTP_COOKIE and they will have the following form

```
key1=value1;key2=value2;key3=value3....
```

Here is an example of how to retrieve cookies-

```
#!/usr/bin/python3
# Import modules for CGI handling
from os import environ
import cgi, cgitb

if environ.has_key('HTTP_COOKIE'):
    for cookie in map(strip, split(environ['HTTP_COOKIE'], ';')):
        (key, value) = split(cookie, '=');
        if key == "UserID":
            user_id = value

        if key == "Password":
            password = value

print ("User ID = %s" % user_id)
print ("Password = %s" % password)
```

This produces the following result for the cookies set by the above script-

```
User ID = XYZ
Password = XYZ123
```

File Upload Example

To upload a file, the HTML form must have the enctype attribute set to multipart/formdata. The input tag with the file type creates a "Browse" button.

```
<html>
<body>
<form enctype="multipart/form-data"
action="save_file.py" method="post">
<p>File: <input type="file" name="filename" /></p>
<p><input type="submit" value="Upload" /></p>
</form>
</body>
</html>
```

Here is the script save_file.py to handle file upload-

```
#!/usr/bin/python3
import cgi, os
import cgi; cgi.enable()

form = cgi.FieldStorage()

# Get filename here.
fileitem = form['filename']

# Test if the file was uploaded
if fileitem.filename:
    # strip leading path from file name to avoid
    # directory traversal attacks
    fn = os.path.basename(fileitem.filename)
    open('/tmp/' + fn, 'wb').write(fileitem.file.read())
    message = 'The file "' + fn + '" was uploaded successfully'
else:
    message = 'No file was uploaded'

print ("""
Content-Type: text/html
<html>
<body>
<p>%s</p>
</body>
</html>
""" % (message,))
```

If you run the above script on Unix/Linux, then you need to take care of replacing file separator as follows, otherwise on your windows machine above open() statement should work fine.

Database Access (MySQL)

PyMySQL is an interface for connecting to a MySQL database server from Python. It implements the Python Database API v2.0 and contains a pure-Python MySQL client library. The goal of PyMySQL is to be a drop-in replacement for MySQLdb .

Before proceeding further, you make sure you have PyMySQL installed on your machine. Just type the following in your Python script and execute it-

```
#!/usr/bin/python3
import PyMySQL
```

If it produces the following result, then it means MySQLdb module is not installed-

```
Traceback (most recent call last):
File "test.py", line 3, in <module>
Import PyMySQL
ImportError: No module named PyMySQL
```

The last stable release is available on PyPI and can be installed with pip:

```
pip install PyMySQL
```

Alternatively (e.g. if pip is not available), a tarball can be downloaded from GitHub and installed with Setuptools as follows-

```
$ # X.X is the desired PyMySQL version (e.g. 0.5 or 0.6).  
$ curl -L https://github.com/PyMySQL/PyMySQL/tarball/pymysql-X.X | tar xz  
$ cd PyMySQL*  
$ python setup.py install  
$ # The folder PyMySQL* can be safely removed now.
```

Note: Make sure you have root privilege to install the above module.

Database Connection

Before connecting to a MySQL database, make sure of the following points-

- ✓ You have created a database TESTDB.
- ✓ You have created a table EMPLOYEE in TESTDB.
- ✓ This table has fields FIRST_NAME, LAST_NAME, AGE, SEX and INCOME.
- ✓ User ID "testuser" and password "test123" are set to access TESTDB.
- ✓ Python module PyMySQL is installed properly on your machine.
- ✓ You have gone through MySQL tutorial to understand MySQL Basics.

Following is an example of connecting with MySQL database "TESTDB"-

```
#!/usr/bin/python3  
import PyMySQL  
  
# Open database connection  
db = PyMySQL.connect("localhost","testuser","test123","TESTDB" )  
  
# prepare a cursor object using cursor() method  
cursor = db.cursor()  
  
# execute SQL query using execute() method.  
cursor.execute("SELECT VERSION()")  
  
# Fetch a single row using fetchone() method.  
data = cursor.fetchone()  
print ("Database version : %s " % data)  
  
# disconnect from server  
db.close()
```

While running this script, it produces the following result-

```
Database version : 5.5.20-log
```

If a connection is established with the datasource, then a Connection Object is returned and saved into db for further use, otherwise db is set to None. Next, db object is used to create a cursor object, which in turn is used to execute SQL queries. Finally, before coming out, it ensures that the database connection is closed and resources are released.

Creating Database Table

Once a database connection is established, we are ready to create tables or records into the database tables using execute method of the created cursor.

Let us create a Database table EMPLOYEE-

```
#!/usr/bin/python3
import PyMySQL

# Open database connection
db = PyMySQL.connect("localhost","testuser","test123","TESTDB" )

# prepare a cursor object using cursor() method
cursor = db.cursor()

# Drop table if it already exist using execute() method.
cursor.execute("DROP TABLE IF EXISTS EMPLOYEE")

# Create table as per requirement
sql = """CREATE TABLE EMPLOYEE (
  FIRST_NAME CHAR(20) NOT NULL,
  LAST_NAME CHAR(20),
  AGE INT,
  SEX CHAR(1),
  INCOME FLOAT )"""

cursor.execute(sql)

# disconnect from server
db.close()
```

INSERT Operation

The INSERT Operation is required when you want to create your records into a database table.

```
#!/usr/bin/python3
import PyMySQL

# Open database connection
db = PyMySQL.connect("localhost","testuser","test123","TESTDB" )

# prepare a cursor object using cursor() method
cursor = db.cursor()
```

```
# Prepare SQL query to INSERT a record into the database.
```

```
sql = """INSERT INTO EMPLOYEE(FIRST_NAME,  
    LAST_NAME, AGE, SEX, INCOME)  
    VALUES ('Mac', 'Mohan', 20, 'M', 2000)"""
```

```
try:
```

```
    # Execute the SQL command  
    cursor.execute(sql)
```

```
    # Commit your changes in the database  
    db.commit()
```

```
except:
```

```
    # Rollback in case there is any error  
    db.rollback()
```

```
# disconnect from server  
db.close()
```

The above example can be written as follows to create SQL queries dynamically-

```
#!/usr/bin/python3  
import PyMySQL
```

```
# Open database connection
```

```
db = PyMySQL.connect("localhost","testuser","test123","TESTDB" )
```

```
# prepare a cursor object using cursor() method  
cursor = db.cursor()
```

```
# Prepare SQL query to INSERT a record into the database.
```

```
sql = "INSERT INTO EMPLOYEE(FIRST_NAME, \  
    LAST_NAME, AGE, SEX, INCOME) \  
    VALUES ('%s', '%s', '%d', '%c', '%d' )" % \  
    ('Mac', 'Mohan', 20, 'M', 2000)
```

```
try:
```

```
    # Execute the SQL command  
    cursor.execute(sql)
```

```
    # Commit your changes in the database  
    db.commit()
```

```
except:
```

```
    # Rollback in case there is any error  
    db.rollback()
```

```
# disconnect from server  
db.close()
```

Example

The following code segment is another form of execution where you can pass parameters directly-

```
.....
user_id = "test123"
password = "password"
con.execute('insert into Login values("%s", "%s")' % \
(user_id, password))
.....
```

READ Operation

READ Operation on any database means to fetch some useful information from the database. Once the database connection is established, you are ready to make a query into this database. You can use either fetchone() method to fetch a single record or fetchall() method to fetch multiple values from a database table.

- fetchone(): It fetches the next row of a query result set. A result set is an object that is returned when a cursor object is used to query a table.
- fetchall(): It fetches all the rows in a result set. If some rows have already been extracted from the result set, then it retrieves the remaining rows from the result set.
- rowcount: This is a read-only attribute and returns the number of rows that were affected by an execute() method.

The following procedure queries all the records from EMPLOYEE table having salary more than 1000-

```
#!/usr/bin/python3
import PyMySQL

# Open database connection
db = PyMySQL.connect("localhost","testuser","test123","TESTDB" )

# prepare a cursor object using cursor() method
cursor = db.cursor()

# Prepare SQL query to INSERT a record into the database.
sql = "SELECT * FROM EMPLOYEE \
WHERE INCOME > '%d'" % (1000)

try:
    # Execute the SQL command
    cursor.execute(sql)

    # Fetch all the rows in a list of lists.
    results = cursor.fetchall()
    for row in results:
        fname = row[0]
        lname = row[1]
        age = row[2]
        sex = row[3]
        income = row[4]

        # Now print fetched result
```

```
print ("fname=%s,lname=%s,age=%d,sex=%s,income=%d" % \
      (fname, lname, age, sex, income ))
except:
    print ("Error: unable to fetch data")

# disconnect from server
db.close()
```

This will produce the following result

```
fname=Mac, lname=Mohan, age=20, sex=M, income=2000
```

UPDATE Operation

UPDATE Operation on any database means to update one or more records, which are already available in the database. The following procedure updates all the records having SEX as 'M'. Here, we increase the AGE of all the males by one year.

```
#!/usr/bin/python3
import PyMySQL

# Open database connection
db = PyMySQL.connect("localhost","testuser","test123","TESTDB" )

# prepare a cursor object using cursor() method
cursor = db.cursor()

# Prepare SQL query to UPDATE required records
sql = "UPDATE EMPLOYEE SET AGE = AGE + 1
      WHERE SEX = '%c'" % ('M')

try:
    # Execute the SQL command
    cursor.execute(sql)

    # Commit your changes in the database
    db.commit()

except:
    # Rollback in case there is any error
    db.rollback()

# disconnect from server
db.close()
```

DELETE Operation

DELETE operation is required when you want to delete some records from your database. Following is the procedure to delete all the records from EMPLOYEE where AGE is more than 20-

```
#!/usr/bin/python3
import PyMySQL
```

```
# Open database connection
db = PyMySQL.connect("localhost","testuser","test123","TESTDB" )

# prepare a cursor object using cursor() method
cursor = db.cursor()

# Prepare SQL query to DELETE required records
sql = "DELETE FROM EMPLOYEE WHERE AGE > '%d'" % (20)

try:
    # Execute the SQL command
    cursor.execute(sql)

    # Commit your changes in the database
    db.commit()

except:
    # Rollback in case there is any error
    db.rollback()

# disconnect from server
db.close()
```

Disconnecting Database

To disconnect the Database connection, use the close() method.

```
db.close()
```

If the connection to a database is closed by the user with the close() method, any outstanding transactions are rolled back by the DB. However, instead of depending on any of the DB lower level implementation details, your application would be better off calling commit or rollback explicitly.

GUI Programming

Tkinter Programming

Tkinter is the standard GUI library for Python. Python when combined with Tkinter provides a fast and easy way to create GUI applications. Tkinter provides a powerful object-oriented interface to the Tk GUI toolkit.

Creating a GUI application using Tkinter is an easy task. All you need to do is perform the following steps –

- a. Import the Tkinter module.
- b. Create the GUI application main window.
- c. Add one or more of the above-mentioned widgets to the GUI application.
- d. Enter the main event loop to take action against each event triggered by the user.

Example

```
#!/usr/bin/python3
import tkinter # note that module name has changed from Tkinter in Python 2 to tkinter in Python 3
```

```
top = tkinter.Tk()
```

```
# Code to add widgets will go here...  
top.mainloop()
```

Tkinter Button

The Button widget is used to add buttons in a Python application. These buttons can display text or images that convey the purpose of the buttons. You can attach a function or a method to a button which is called automatically when you click the button.

```
w= Button ( master, option=value, ... )
```

Parameters

- master: This represents the parent window.
- options: Here is the list of most commonly used options for this widget. These options can be used as key-value pairs separated by commas.

Option	Description
activebackground	Background color when the button is under the cursor.
activeforeground	Foreground color when the button is under the cursor.
bd	Border width in pixels. Default is 2.
bg	Normal background color.
command	Function or method to be called when the button is clicked.
fg	Normal foreground (text) color.
font	Text font to be used for the button's label.
height	Height of the button in text lines (for textual buttons) or pixels(for images).
highlightcolor	The color of the focus highlight when the widget has focus.
image	Image to be displayed on the button (instead of text).
justify	How to show multiple text lines: LEFT to left-justify each line; CENTER to center them; or RIGHT to right-justify.
padx	Additional padding left and right of the text.
pady	Additional padding above and below the text.
relief	Relief specifies the type of the border. Some of the values are SUNKEN, RAISED, GROOVE, and RIDGE.
state	Set this option to DISABLED to gray out the button and make it unresponsive. Has the value ACTIVE when the mouse is over it. Default is NORMAL.
underline	Default is -1, meaning that no character of the text on the button will be underlined. If nonnegative, the corresponding text character will be underlined.
width	Width of the button in letters (if displaying text) or pixels (if displaying an image).
wrlength	If this value is set to a positive number, the text lines will be wrapped to fit within this length.

Methods

Method	Description
flash()	Causes the button to flash several times between active and normal colors. Leaves the button in the state it was in originally. Ignored if the button is disabled.

`invoke()` Calls the button's callback, and returns what that function returns. Has no effect if the button is disabled or there is no callback.

Example

```
#!/usr/bin/python3
from tkinter import *
from tkinter import messagebox

top = Tk()
top.geometry("100x100")

def helloCallBack():
    msg=messagebox.showinfo( "Hello Python", "Hello World")

B = Button(top, text ="Hello", command = helloCallBack)
B.place(x=50,y=50)
top.mainloop()
```

Tkinter Checkbutton

The Checkbutton widget is used to display a number of options to a user as toggle buttons. The user can then select one or more options by clicking the button corresponding to each option.

```
w = Checkbutton ( master, option, ... )
```

Parameters

- **master:** This represents the parent window.
- **options:** Here is the list of most commonly used options for this widget. These options can be used as key-value pairs separated by commas.

Option	Description
<code>activebackground</code>	Background color when the checkbutton is under the cursor.
<code>activeforeground</code>	Foreground color when the checkbutton is under the cursor.
<code>bg</code>	The normal background color displayed behind the label and indicator.
<code>bitmap</code>	To display a monochrome image on a button.
<code>bd</code>	The size of the border around the indicator. Default is 2 pixels.
<code>command</code>	A procedure to be called every time the user changes the state of this checkbutton.
<code>cursor</code>	If you set this option to a cursor name (arrow, dot etc.), the mouse cursor will change to that pattern when it is over the checkbutton.
<code>disabledforeground</code>	The foreground color used to render the text of a disabled checkbutton. The default is a stippled version of the default foreground color.
<code>font</code>	The font used for the text.
<code>fg</code>	The color used to render the text.
<code>height</code>	The number of lines of text on the checkbutton. Default is 1.
<code>highlightcolor</code>	The color of the focus highlight when the checkbutton has the focus.
<code>image</code>	To display a graphic image on the button.
<code>justify</code>	If the text contains multiple lines, this option controls how the text is justified: CENTER, LEFT, or RIGHT.

offvalue	Normally, a checkbox's associated control variable will be set to 0 when it is cleared (off). You can supply an alternate value for the off state by setting offvalue to that value.
onvalue	Normally, a checkbox's associated control variable will be set to 1 when it is set (on). You can supply an alternate value for the on state by setting onvalue to that value.
padx	How much space to leave to the left and right of the checkbox and text. Default is 1 pixel.
pady	How much space to leave above and below the checkbox and text. Default is 1 pixel.
relief	With the default value, relief=FLAT, the checkbox does not stand out from its background. You may set this option to any of the other styles
selectcolor	The color of the checkbox when it is set. Default is selectcolor="red".
selectimage	If you set this option to an image, that image will appear in the checkbox when it is set.
state	The default is state=NORMAL, but you can use state=DISABLED to gray out the control and make it unresponsive. If the cursor is currently over the checkbox, the state is ACTIVE.
text	The label displayed next to the checkbox. Use newlines ("\n") to display multiple lines of text.
underline	With the default value of -1, none of the characters of the text label are underlined. Set this option to the index of a character in the text (counting from zero) to underline that character.
variable	The control variable that tracks the current state of the checkbox. Normally this variable is an IntVar, and 0 means cleared and 1 means set, but see the offvalue and onvalue options above.
width	The default width of a checkbox is determined by the size of the displayed image or text. You can set this option to a number of characters and the checkbox will always have room for that many characters.
wraplength	Normally, lines are not wrapped. You can set this option to a number of characters and all lines will be broken into pieces no longer than that number.

Methods

Method	Description
deselect()	Clears (turns off) the checkbox.
flash()	Flashes the checkbox a few times between its active and normal colors, but leaves it the way it started.
invoke()	You can call this method to get the same actions that would occur if the user clicked on the checkbox to change its state.
select()	Sets (turns on) the checkbox.
toggle()	Clears the checkbox if set, sets it if cleared.

Example

```
#!/usr/bin/python3
from tkinter import *
import tkinter
top = Tk()
CheckVar1 = IntVar()
CheckVar2 = IntVar()

C1 = Checkbutton(top, text = "Music", variable = CheckVar1, \
onvalue = 1, offvalue = 0, height=5, \
width = 20, )

C2 = Checkbutton(top, text = "Video", variable = CheckVar2, \
```

```
onvalue = 1, offvalue = 0, height=5, \
width = 20)
```

```
C1.pack()
C2.pack()
top.mainloop()
```

Tkinter Entry

The Entry widget is used to accept single-line text strings from a user.

```
w = Entry( master, option, ... )
```

Parameters

- master: This represents the parent window.
- options: Here is the list of most commonly used options for this widget. These options can be used as key-value pairs separated by commas.

Option	Description
bg	The normal background color displayed behind the label and indicator.
bd	The size of the border around the indicator. Default is 2 pixels.
command	A procedure to be called every time the user changes the state of this checkbox.
cursor	If you set this option to a cursor name (arrow, dot etc.), the mouse cursor will change to that pattern when it is over the checkbox.
font	The font used for the text.
exportselection	By default, if you select text within an Entry widget, it is automatically exported to the clipboard. To avoid this exportation, use exportselection=0.
fg	The color used to render the text.
highlightcolor	The color of the focus highlight when the checkbox has the focus.
justify	If the text contains multiple lines, this option controls how the text is justified: CENTER, LEFT, or RIGHT.
relief	With the default value, relief=FLAT, the checkbox does not stand out from its background. You may set this option to any of the other styles
selectbackground	The background color to use displaying selected text.
selectborderwidth	The width of the border to use around selected text. The default is one pixel.
selectforeground	The foreground (text) color of selected text.
show	Normally, the characters that the user types appear in the entry. To make a .password. entry that echoes each character as an asterisk, set show="*".
state	The default is state=NORMAL, but you can use state=DISABLED to gray out the control and make it unresponsive. If the cursor is currently over the checkbox, the state is ACTIVE.
textvariable	In order to be able to retrieve the current text from your entry widget, you must set this option to an instance of the StringVar class.
width	The default width of a checkbox is determined by the size of the displayed image or text. You can set this option to a number of characters and the checkbox will always have room for that many characters.
xscrollcommand	If you expect that users will often enter more text than the onscreen size of the widget, you can link your entry widget to a scrollbar.

Methods

Method

Description

<code>delete (first, last=None)</code>	Deletes characters from the widget, starting with the one at index first, up to but not including the character at position last. If the second argument is omitted, only the single character at position first is deleted.
<code>get()</code>	Returns the entry's current text as a string.
<code>icursor (index)</code>	Set the insertion cursor just before the character at the given index.
<code>index (index)</code>	Shift the contents of the entry so that the character at the given index is the leftmost visible character. Has no effect if the text fits entirely within the entry.
<code>insert (index, s)</code>	Inserts string s before the character at the given index.
<code>select_adjust (index)</code>	This method is used to make sure that the selection includes the character at the specified index.
<code>select_clear()</code>	Clears the selection. If there isn't currently a selection, has no effect.
<code>select_from (index)</code>	Sets the ANCHOR index position to the character selected by index, and selects that character.
<code>select_present()</code>	If there is a selection, returns true, else returns false.
<code>select_range (start, end)</code>	Sets the selection under program control. Selects the text starting at the start index, up to but not including the character at the end index. The start position must be before the end position.
<code>select_to (index)</code>	Selects all the text from the ANCHOR position up to but not including the character at the given index.
<code>xview (index)</code>	This method is useful in linking the Entry widget to a horizontal scrollbar.
<code>xview_scroll (number, what)</code>	Used to scroll the entry horizontally. The what argument must be either UNITS, to scroll by character widths, or PAGES, to scroll by chunks the size of the entry widget. The number is positive to scroll left to right, negative to scroll right to left.

```
# !/usr/bin/python3
from tkinter import *
top = Tk()
L1 = Label(top, text="User Name")
L1.pack( side = LEFT)
E1 = Entry(top, bd =5)
E1.pack(side = RIGHT)
top.mainloop()
```

Tkinter Frame

The Frame widget is very important for the process of grouping and organizing other widgets in a somehow friendly way. It works like a container, which is responsible for arranging the position of other widgets. It uses rectangular areas in the screen to organize the layout and to provide padding of these widgets. A frame can also be used as a foundation class to implement complex widgets.

```
w = Frame ( master, option, ... )
```

Parameters

- master: This represents the parent window.

- options: Here is the list of most commonly used options for this widget. These options can be used as key-value pairs separated by commas.

Options	Description
bg	The normal background color displayed behind the label and indicator.
bd	The size of the border around the indicator. Default is 2 pixels.
cursor	If you set this option to a cursor name (arrow, dot etc.), the mouse cursor will change to that pattern when it is over the checkbutton.
height	The vertical dimension of the new frame.
highlightbackground	Color of the focus highlight when the frame does not have focus.
highlightcolor	Color shown in the focus highlight when the frame has the focus.
highlightthickness	Thickness of the focus highlight.
relief	With the default value, relief=FLAT, the checkbutton does not stand out from its background. You may set this option to any of the other styles
width	The default width of a checkbutton is determined by the size of the displayed image or text. You can set this option to a number of characters and the checkbutton will always have room for that many characters.

Example

```
# !/usr/bin/python3
from tkinter import *
root = Tk()
frame = Frame(root)
frame.pack()
bottomframe = Frame(root)
bottomframe.pack( side = BOTTOM )

redbutton = Button(frame, text="Red", fg="red")
redbutton.pack( side = LEFT)

greenbutton = Button(frame, text="Brown", fg="brown")
greenbutton.pack( side = LEFT )

bluebutton = Button(frame, text="Blue", fg="blue")
bluebutton.pack( side = LEFT )

blackbutton = Button(bottomframe, text="Black", fg="black")
blackbutton.pack( side = BOTTOM)

root.mainloop()
```

Tkinter Label

This widget implements a display box where you can place text or images. The text displayed by this widget can be updated at any time you want. It is also possible to underline part of the text (like to identify a keyboard shortcut) and span the text across multiple lines.

```
w = Label ( master, option, ... )
```

Parameters

master: This represents the parent window.

options: Here is the list of most commonly used options for this widget. These options can be used as key-value pairs separated by commas.

Options	Description
anchor	This options controls where the text is positioned if the widget has more space than the text needs. The default is anchor=CENTER, which centers the text in the available space.
bg	The normal background color displayed behind the label and indicator.
bitmap	Set this option equal to a bitmap or image object and the label will display that graphic.
bd	The size of the border around the indicator. Default is 2 pixels.
cursor	If you set this option to a cursor name (arrow, dot etc.), the mouse cursor will change to that pattern when it is over the checkbutton. font If you are displaying text in this label (with the text or textvariable option, the font option specifies in what font that text will be displayed.
fg	If you are displaying text or a bitmap in this label, this option specifies the color of the text. If you are displaying a bitmap, this is the color that will appear at the position of the 1-bits in the bitmap.
height	The vertical dimension of the new frame.
image	To display a static image in the label widget, set this option to an image object.
justify	Specifies how multiple lines of text will be aligned with respect to each other: LEFT for flush left, CENTER for centered (the default), or RIGHT for right-justified.
padx	Extra space added to the left and right of the text within the widget. Default is 1.
pady	Extra space added above and below the text within the widget. Default is 1.
relief	Specifies the appearance of a decorative border around the label. The default is FLAT; for other values.
text	To display one or more lines of text in a label widget, set this option to a string containing the text. Internal newlines ("\n") will force a line break.
textvariable	To slave the text displayed in a label widget to a control variable of class StringVar, set this option to that variable.
underline	You can display an underline (_) below the nth letter of the text, counting from 0, by setting this option to n. The default is underline=-1, which means no underlining.
width	Width of the label in characters (not pixels!). If this option is not set, the label will be sized to fit its contents.
wrlength	You can limit the number of characters in each line by setting this option to the desired number. The default value, 0, means that lines will be broken only at newlines.

Example

```
# !/usr/bin/python3
from tkinter import *
root = Tk()
var = StringVar()
label = Label( root, textvariable=var, relief=RAISED )
var.set("Hey!? How are you doing?")
label.pack()
```

Tkinter Listbox

The Listbox widget is used to display a list of items from which a user can select a number of items

```
w= Listbox ( master, option, ... )
```

Parameters

- master: This represents the parent window.
- options: Here is the list of most commonly used options for this widget. These options can be used as key-value pairs separated by commas.

Options	Description
bg	The normal background color displayed behind the label and indicator.
bd	The size of the border around the indicator. Default is 2 pixels.
cursor	The cursor that appears when the mouse is over the listbox.
font	The font used for the text in the listbox.
fg	The color used for the text in the listbox.
height	Number of lines (not pixels!) shown in the listbox. Default is 10.
highlightcolor	Color shown in the focus highlight when the widget has the focus.
highlightthickness	Thickness of the focus highlight.
relief	Selects three-dimensional border shading effects. The default is <code>SUNKEN</code> .
selectbackground	The background color to use displaying selected text.
selectmode	Determines how many items can be selected, and how mouse drags affect the selection: • <code>BROWSE</code>: Normally, you can only select one line out of a listbox. If you click on an item and then drag to a different line, the selection will follow the mouse. This is the default. • <code>SINGLE</code>: You can only select one line, and you can't drag the mouse. wherever you click button 1, that line is selected. • <code>MULTIPLE</code>: You can select any number of lines at once. Clicking on any line toggles whether or not it is selected. • <code>EXTENDED</code>: You can select any adjacent group of lines at once by clicking on the first line and dragging to the last line.
width	The width of the widget in characters. The default is 20.
xscrollcommand	If you want to allow the user to scroll the listbox horizontally, you can link your listbox widget to a horizontal scrollbar.
yscrollcommand	If you want to allow the user to scroll the listbox vertically, you can link your listbox widget to a vertical scrollbar.

Methods

Options	Description
activate (index)	Selects the line specifies by the given index.
curselection()	Returns a tuple containing the line numbers of the selected element or elements, counting from 0. If nothing is selected, returns an empty tuple.
delete (first, last=None)	Deletes the lines whose indices are in the range [first, last]. If the second argument is omitted, the single line with index first is deleted.

<code>get (first, last=None)</code>	Returns a tuple containing the text of the lines with indices from first to last, inclusive. If the second argument is omitted, returns the text of the line closest to first.
<code>index (i)</code>	If possible, positions the visible part of the listbox so that the line containing index i is at the top of the widget.
<code>insert (index, *elements)</code>	Insert one or more new lines into the listbox before the line specified by index. Use END as the first argument if you want to add new lines to the end of the listbox.
<code>nearest (y)</code>	Return the index of the visible line closest to the y coordinate y relative to the listbox widget.
<code>see (index)</code>	Adjust the position of the listbox so that the line referred to by index is visible.
<code>size()</code>	Returns the number of lines in the listbox.
<code>xview()</code>	To make the listbox horizontally scrollable, set the command option of the associated horizontal scrollbar to this method.
<code>xview_moveto (fraction)</code>	Scroll the listbox so that the leftmost fraction of the width of its longest line is outside the left side of the listbox. Fraction is in the range [0,1].
<code>xview_scroll (number, what)</code>	Scrolls the listbox horizontally. For the what argument, use either UNITS to scroll by characters, or PAGES to scroll by pages, that is, by the width of the listbox. The number argument tells how many to scroll.
<code>yview()</code>	To make the listbox vertically scrollable, set the command option of the associated vertical scrollbar to this method.
<code>yview_moveto (fraction)</code>	Scroll the listbox so that the top fraction of the width of its longest line is outside the left side of the listbox. Fraction is in the range [0,1].
<code>yview_scroll (number, what)</code>	Scrolls the listbox vertically. For the what argument, use either UNITS to scroll by lines, or PAGES to scroll by pages, that is, by the height of the listbox. The number argument tells how many to scroll.

Example

```
# !/usr/bin/python3
from tkinter import *
import tkinter
top = Tk()
Lb1 = Listbox(top)
Lb1.insert(1, "Python")
Lb1.insert(2, "Perl")
Lb1.insert(3, "C")
Lb1.insert(4, "PHP")
Lb1.insert(5, "JSP")
Lb1.insert(6, "Ruby")
Lb1.pack()
top.mainloop()
```

Tkinter Menubutton

A menubutton is the part of a drop-down menu that stays on the screen all the time. Every menubutton is associated with a Menu widget that can display the choices for that menubutton when the user clicks on it.

```
w = Menubutton ( master, option, ... )
```

Parameters

- master: This represents the parent window.
- options: Here is the list of most commonly used options for this widget. These options can be used as key-value pairs separated by commas.

Options	Description
activebackground	The background color when the mouse is over the menubutton.
activeforeground	The foreground color when the mouse is over the menubutton.
anchor	This options controls where the text is positioned if the widget has more space than the text needs. The default is anchor=CENTER, which centers the text.
bg	The normal background color displayed behind the label and indicator.
bitmap	To display a bitmap on the menubutton, set this option to a bitmap name.
bd	The size of the border around the indicator. Default is 2 pixels.
cursor	The cursor that appears when the mouse is over this menubutton.
direction	Set direction=LEFT to display the menu to the left of the button; use direction=RIGHT to display the menu to the right of the button; or use direction='above' to place the menu above the button.
disabledforeground	The foreground color shown on this menubutton when it is disabled.
fg	The foreground color when the mouse is not over the menubutton.
height	The height of the menubutton in lines of text (not pixels!). The default is to fit the menubutton's size to its contents.
highlightcolor	Color shown in the focus highlight when the widget has the focus.
image	To display an image on this menubutton,
justify	This option controls where the text is located when the text doesn't fill the menubutton: use justify=LEFT to left-justify the text (this is the default); use justify=RIGHT to right-justify.
menu	To associate the menubutton with a set of choices, set this option to the Menu object containing those choices. That menu object must have been created by passing the associated menubutton to the constructor as its first argument.
padx	How much space to leave to the left and right of the text of the menubutton. Default is 1.
pady	How much space to leave above and below the text of the menubutton. Default is 1.
relief	Selects three-dimensional border shading effects. The default is RAISED.
state	Normally, menubuttons respond to the mouse. Set state=DISABLED to gray out the menubutton and make it unresponsive.
text	To display text on the menubutton, set this option to the string containing the desired text. Newlines ("\n") within the string will cause line breaks.
textvariable	You can associate a control variable of class StringVar with this menubutton. Setting that control variable will change the displayed text.
underline	Normally, no underline appears under the text on the menubutton. To underline one of the characters, set this option to the index of that character.
width	The width of the widget in characters. The default is 20.
wrlength	Normally, lines are not wrapped. You can set this option to a number of characters and all lines will be broken into pieces no longer than that number.

Example

```
# !/usr/bin/python3
```

```

from tkinter import *
import tkinter
top = Tk()
mb= Menubutton ( top, text="condiments", relief=RAISED )
mb.grid()
mb.menu = Menu ( mb, tearoff = 0 )
mb["menu"] = mb.menu
mayoVar = IntVar()
ketchVar = IntVar()
mb.menu.add_checkbutton ( label="mayo", variable=mayoVar )
mb.menu.add_checkbutton ( label="ketchup", variable=ketchVar )
mb.pack()
top.mainloop()

```

Tkinter Menu

The goal of this widget is to allow us to create all kinds of menus that can be used by our applications. The core functionality provides ways to create three menu types: pop-up, toplevel and pull-down. It is also possible to use other extended widgets to implement new types of menus, such as the OptionMenu widget, which implements a special type that generates a pop-up list of items within a selection.

```
w = Menu ( master, option, ... )
```

Parameters

- master: This represents the parent window.
- options: Here is the list of most commonly used options for this widget. These options can be used as key-value pairs separated by commas.

Options

Description

activebackground	The background color that will appear on a choice when it is under the mouse.
activeborderwidth	Specifies the width of a border drawn around a choice when it is under the mouse. Default is 1 pixel.
activeforeground	The foreground color that will appear on a choice when it is under the mouse.
bg	The background color for choices not under the mouse.
bd	The width of the border around all the choices. Default is 1.
cursor	The cursor that appears when the mouse is over the choices, but only when the menu has been torn off.
disabledforeground	The color of the text for items whose state is DISABLED.
font	The default font for textual choices.
fg	The foreground color used for choices not under the mouse.
postcommand	You can set this option to a procedure, and that procedure will be called every time someone brings up this menu.
relief	The default 3-D effect for menus is relief=RAISED.
image	To display an image on this menubutton.
selectcolor	Specifies the color displayed in checkbuttons and radiobuttons when they are selected.
tearoff	Normally, a menu can be torn off, the first position (position 0) in the list of choices is occupied by the tear-off element, and the additional choices are added starting at position 1. If you set

Methods

Option

add_command (options)
add_radiobutton(options)
add_checkbutton(options)
add_cascade(options)
add_separator()
add(type, options)
delete(startindex [, endindex])
entryconfig(index, options)

index(item)
insert_separator (index)

title
Normally, the title of a tear-off menu window will be the same as the text of the menubutton or cascade that lead to this menu. If you want to change the title of that window, set the title option to that string.

invoke (index)
Calls the command callback associated with the choice at position index. If a checkbutton, its state is toggled between set and cleared; if a radiobutton, that choice is set.

type (index)
Returns the type of the choice specified by index: either "cascade", "checkbutton", "command", "radiobutton", "separator", or "tearoff".

Description

Adds a menu item to the menu.

Creates a radio button menu item.

Creates a check button menu item.

Creates a new hierarchical menu by associating a given menu to a parent menu

Adds a separator line to the menu.

Adds a specific type of menu item to the menu.

Deletes the menu items ranging from startindex to endindex.

Allows you to modify a menu item, which is identified by the index, and change its options.

Returns the index number of the given menu item label.

Insert a new separator at the position specified by index. tearoff=0, the menu will not have a tear-off feature, and choices will be added starting at position 0.

Normally, the title of a tear-off menu window will be the same as the text of the menubutton or cascade that lead to this menu. If you want to change the title of that window, set the title option to that string.

Calls the command callback associated with the choice at position index. If a checkbutton, its state is toggled between set and cleared; if a radiobutton, that choice is set.

Returns the type of the choice specified by index: either "cascade", "checkbutton", "command", "radiobutton", "separator", or "tearoff".

Example

```
# !/usr/bin/python3
from tkinter import *
def donothing():
    filewin = Toplevel(root)
    button = Button(filewin, text="Do nothing button")
    button.pack()
    root = Tk()
    menubar = Menu(root)
    filemenu = Menu(menubar, tearoff=0)
    filemenu.add_command(label="New", command=donothing)
    filemenu.add_command(label="Open", command=donothing)
    filemenu.add_command(label="Save", command=donothing)
    filemenu.add_command(label="Save as...", command=donothing)
    filemenu.add_command(label="Close", command=donothing)
    filemenu.add_separator()
    filemenu.add_command(label="Exit", command=root.quit)
    menubar.add_cascade(label="File", menu=filemenu)
    editmenu = Menu(menubar, tearoff=0)
    editmenu.add_command(label="Undo", command=donothing)
    editmenu.add_separator()
    editmenu.add_command(label="Cut", command=donothing)
```

```

editmenu.add_command(label="Copy", command=donothing)
editmenu.add_command(label="Paste", command=donothing)
editmenu.add_command(label="Delete", command=donothing)
editmenu.add_command(label="Select All", command=donothing)
menubar.add_cascade(label="Edit", menu=editmenu)
helpmenu = Menu(menubar, tearoff=0)
helpmenu.add_command(label="Help Index", command=donothing)
helpmenu.add_command(label="About...", command=donothing)
menubar.add_cascade(label="Help", menu=helpmenu)
root.config(menu=menubar)
root.mainloop()

```

Tkinter Message

This widget provides a multiline and noneditable object that displays texts, automatically breaking lines and justifying their contents. Its functionality is very similar to the one provided by the Label widget, except that it can also automatically wrap the text, maintaining a given width or aspect ratio.

```
w = Message ( master, option, ... )
```

Parameters

- master: This represents the parent window.
- options: Here is the list of most commonly used options for this widget. These options can be used as key-value pairs separated by commas.

Options	Description
anchor	This options controls where the text is positioned if the widget has more space than the text needs. The default is anchor=CENTER, which centers the text in the available space.
bg	The normal background color displayed behind the label and indicator.
bitmap	Set this option equal to a bitmap or image object and the label will display that graphic.
bd	The size of the border around the indicator. Default is 2 pixels.
cursor	If you set this option to a cursor name (arrow, dot etc.), the mouse cursor will change to that pattern when it is over the checkbox.
font	If you are displaying text in this label (with the text or textvariable option, the font option specifies in what font that text will be displayed.
fg	If you are displaying text or a bitmap in this label, this option specifies the color of the text. If you are displaying a bitmap, this is the color that will appear at the position of the 1-bits in the bitmap.
height	The vertical dimension of the new frame.
image	To display a static image in the label widget, set this option to an image object.
justify	Specifies how multiple lines of text will be aligned with respect to each other: LEFT for flush left, CENTER for centered (the default), or RIGHT for right-justified.
padx	Extra space added to the left and right of the text within the widget. Default is 1.
pady	Extra space added above and below the text within the widget. Default is 1.
relief	Specifies the appearance of a decorative border around the label. The default is FLAT; for other values.

text	To display one or more lines of text in a label widget, set this option to a string containing the text. Internal newlines (" <code>\n</code> ") will force a line break.
textvariable	To slave the text displayed in a label widget to a control variable of class <code>StringVar</code> , set this option to that variable.
underline	You can display an underline (<code>_</code>) below the <i>n</i> th letter of the text, counting from 0, by setting this option to <i>n</i> . The default is <code>underline=-1</code> , which means no underlining.
width	Width of the label in characters (not pixels!). If this option is not set, the label will be sized to fit its contents.
wraplength	You can limit the number of characters in each line by setting this option to the desired number. The default value, 0, means that lines will be broken only at newlines.

Example

```
# !/usr/bin/python3
from tkinter import *
root = Tk()
var = StringVar()
label = Message( root, textvariable=var, relief=RAISED )
var.set("Hey!? How are you doing?")
label.pack()
root.mainloop()
```

Tkinter Radiobutton

This widget implements a multiple-choice button, which is a way to offer many possible selections to the user and lets user choose only one of them. In order to implement this functionality, each group of radiobuttons must be associated to the same variable and each one of the buttons must symbolize a single value. You can use the Tab key to switch from one radiobutton to another.

```
w = Radiobutton ( master, option, ... )
```

Parameters

- **master:** This represents the parent window.
- **options:** Here is the list of most commonly used options for this widget. These options can be used as key-value pairs separated by commas.

Options	Description
activebackground	The background color when the mouse is over the radiobutton.
activeforeground	The foreground color when the mouse is over the radiobutton.
anchor	If the widget inhabits a space larger than it needs, this option specifies where the radiobutton will sit in that space. The default is <code>anchor=CENTER</code> .
bg	The normal background color behind the indicator and label.
bitmap	To display a monochrome image on a radiobutton, set this option to a bitmap.
borderwidth	The size of the border around the indicator part itself. Default is 2 pixels.
command	A procedure to be called every time the user changes the state of this radiobutton.
cursor	If you set this option to a cursor name (arrow, dot etc.), the mouse cursor will change to that pattern when it is over the radiobutton.
font	The font used for the text.
fg	The color used to render the text.

height	The number of lines (not pixels) of text on the radiobutton. Default is 1.
highlightbackground	The color of the focus highlight when the radiobutton does not have focus.
highlightcolor	The color of the focus highlight when the radiobutton has the focus.
image	To display a graphic image instead of text for this radiobutton, set this option to an image object.
justify	If the text contains multiple lines, this option controls how the text is justified: CENTER (the default), LEFT, or RIGHT.
padx	How much space to leave to the left and right of the radiobutton and text. Default is 1.
pady	How much space to leave above and below the radiobutton and text. Default is 1.
relief	Specifies the appearance of a decorative border around the label. The default is FLAT; for other values.
selectcolor	The color of the radiobutton when it is set. Default is red.
selectimage	If you are using the image option to display a graphic instead of text when the radiobutton is cleared, you can set the selectimage option to a different image that will be displayed when the radiobutton is set.
state	The default is state=NORMAL, but you can set state=DISABLED to gray out the control and make it unresponsive. If the cursor is currently over the radiobutton, the state is ACTIVE.
text	The label displayed next to the radiobutton. Use newlines ("\n") to display multiple lines of text.
textvariable	To slave the text displayed in a label widget to a control variable of class StringVar, set this option to that variable.
underline	You can display an underline (_) below the nth letter of the text, counting from 0, by setting this option to n. The default is underline=-1, which means no underlining.
value	When a radiobutton is turned on by the user, its control variable is set to its current value option. If the control variable is an IntVar, give each radiobutton in the group a different integer value option. If the control variable is aStringVar, give each radiobutton a different string value option.
variable	The control variable that this radiobutton shares with the other radiobuttons in the group. This can be either an IntVar or a StringVar.
width	Width of the label in characters (not pixels!). If this option is not set, the label will be sized to fit its contents.
wraplength	You can limit the number of characters in each line by setting this option to the desired number. The default value, 0, means that lines will be broken only at newlines.

Methods

Methods	Description
deselect()	Clears (turns off) the radiobutton.
flash()	Flashes the radiobutton a few times between its active and normal colors, but leaves it the way it started.
invoke()	You can call this method to get the same actions that would occur if the user clicked on the radiobutton to change its state.
select()	Sets (turns on) the radiobutton.

Example

```
# !/usr/bin/python3
```

```

from tkinter import *
def sel():
    selection = "You selected the option " + str(var.get())
    label.config(text = selection)
root = Tk()
var = IntVar()

R1 = Radiobutton(root, text="Option 1", variable=var, value=1,command=sel)
R1.pack( anchor = W )

R2 = Radiobutton(root, text="Option 2", variable=var, value=2,command=sel)
R2.pack( anchor = W )

R3 = Radiobutton(root, text="Option 3", variable=var, value=3,command=sel)
R3.pack( anchor = W )

label = Label(root)
label.pack()
root.mainloop()

```

Tkinter Scale

The Scale widget provides a graphical slider object that allows you to select values from a specific scale.

w = Scale (master, option, ...)

Parameters

- master: This represents the parent window.
- options: Here is the list of most commonly used options for this widget. These options can be used as key-value pairs separated by commas.

Options	Description
activebackground	The background color when the mouse is over the scale.
bg	The background color of the parts of the widget that are outside the trough.
bd	Width of the 3-d border around the trough and slider. Default is 2 pixels.
command	A procedure to be called every time the slider is moved. This procedure will be passed one argument, the new scale value. If the slider is moved rapidly, you may not get a callback for every possible position, but you'll certainly get a callback when it settles.
cursor	If you set this option to a cursor name (arrow, dot etc.), the mouse cursor will change to that pattern when it is over the scale.
digits	The way your program reads the current value shown in a scale widget is through a control variable. The control variable for a scale can be an IntVar, a DoubleVar (float), or a StringVar. If it is a string variable, the digits option controls how many digits to use when the numeric scale value is converted to a string.
font	The font used for the label and annotations.
fg	The color of the text used for the label and annotations.
from_	A float or integer value that defines one end of the scale's range.
highlightbackground	The color of the focus highlight when the scale does not have focus.

highlightcolor	The color of the focus highlight when the scale has the focus.
label	You can display a label within the scale widget by setting this option to the label's text. The label appears in the top left corner if the scale is horizontal, or the top right corner if vertical. The default is no label.
length	The length of the scale widget. This is the x dimension if the scale is horizontal, or the y dimension if vertical. The default is 100 pixels.
orient	Set orient=HORIZONTAL if you want the scale to run along the x dimension, or orient=VERTICAL to run parallel to the y-axis. Default is horizontal.
relief	Specifies the appearance of a decorative border around the label. The default is FLAT; for other values.
repeatdelay	This option controls how long button 1 has to be held down in the trough before the slider starts moving in that direction repeatedly. Default is repeatdelay=300, and the units are milliseconds.
resolution	Normally, the user will only be able to change the scale in whole units. Set this option to some other value to change the smallest increment of the scale's value. For example, if from_=-1.0 and to=1.0, and you set resolution=0.5, the scale will have 5 possible values: -1.0, -0.5, 0.0, +0.5, and +1.0.
showvalue	Normally, the current value of the scale is displayed in text form by the slider (above it for horizontal scales, to the left for vertical scales). Set this option to 0 to suppress that label.
sliderlength	Normally the slider is 30 pixels along the length of the scale. You can change that length by setting the sliderlength option to your desired length.
state	Normally, scale widgets respond to mouse events, and when they have the focus, also keyboard events. Set state=DISABLED to make the widget unresponsive.
takefocus	Normally, the focus will cycle through scale widgets. Set this option to 0 if you don't want this behavior.
tickinterval	To display periodic scale values, set this option to a number, and ticks will be displayed on multiples of that value. For example, if from_=0.0, to=1.0, and tickinterval=0.25, labels will be displayed along the scale at values 0.0, 0.25, 0.50, 0.75, and 1.00. These labels appear below the scale if horizontal, to its left if vertical. Default is 0, which suppresses display of ticks.
to	A float or integer value that defines one end of the scale's range; the other end is defined by the from_ option, discussed above. The to value can be either greater than or less than the from_ value. For vertical scales, the to value defines the bottom of the scale; for horizontal scales, the right end.
troughcolor	The color of the trough.
variable	The control variable for this scale, if any. Control variables may be from class IntVar, DoubleVar (float), or StringVar. In the latter case, the numerical value will be converted to a string.
width	The width of the trough part of the widget. This is the x dimension for vertical scales and the y dimension if the scale has orient=HORIZONTAL. Default is 15 pixels.

Methods

Methods

	Description
get()	This method returns the current value of the scale.
set (value)	Sets the scale's value.

Example

```
# !/usr/bin/python3
```

```

from tkinter import *
def sel():
    selection = "Value = " + str(var.get())
    label.config(text = selection)
root = Tk()
var = DoubleVar()
scale = Scale( root, variable = var )
scale.pack(anchor=CENTER)
button = Button(root, text="Get Scale Value", command=sel)
button.pack(anchor=CENTER)
label = Label(root)
label.pack()
root.mainloop()

```

Tkinter Scrollbar

This widget provides a slide controller that is used to implement vertical scrolled widgets, such as Listbox, Text and Canvas. Note that you can also create horizontal scrollbars on Entry widgets.

```
w = Scrollbar ( master, option, ... )
```

Parameters

- master: This represents the parent window.
- options: Here is the list of most commonly used options for this widget. These options can be used as key-value pairs separated by commas.

Options	Description
activebackground	The color of the slider and arrowheads when the mouse is over them.
bg	The color of the slider and arrowheads when the mouse is not over them.
bd	The width of the 3-d borders around the entire perimeter of the trough, and also the width of the 3-d effects on the arrowheads and slider. Default is no border around the trough, and a 2-pixel border around the arrowheads and slider.
command	A procedure to be called whenever the scrollbar is moved.
cursor	The cursor that appears when the mouse is over the scrollbar.
elementborderwidth	The width of the borders around the arrowheads and slider. The default is elementborderwidth=-1, which means to use the value of the borderwidth option.
highlightbackground	The color of the focus highlight when the scrollbar does not have focus.
highlightcolor	The color of the focus highlight when the scrollbar has the focus.
highlightthickness	The thickness of the focus highlight. Default is 1. Set to 0 to suppress display of the focus highlight.
jump	This option controls what happens when a user drags the slider. Normally (jump=0), every small drag of the slider causes the command callback to be called. If you set this option to 1, the callback isn't called until the user releases the mouse button.
orient	Set orient=HORIZONTAL for a horizontal scrollbar, orient=VERTICAL for a vertical one.
repeatdelay	This option controls how long button 1 has to be held down in the trough before the slider starts moving in that direction repeatedly. Default is repeatdelay=300, and the units are milliseconds.
repeatinterval	repeatinterval

takefocus	Normally, you can tab the focus through a scrollbar widget. Set takefocus=0 if you don't want this behavior.
troughcolor	The color of the trough.
width	Width of the scrollbar (its y dimension if horizontal, and its x dimension if vertical). Default is 16.

Methods

Methods	Description
get()	Returns two numbers (a, b) describing the current position of the slider. The a value gives the position of the left or top edge of the slider, for horizontal and vertical scrollbars respectively; the b value gives the position of the right or bottom edge.
set (first, last)	To connect a scrollbar to another widget w, set w's xscrollcommand or yscrollcommand to the scrollbar's set() method. The arguments have the same meaning as the values returned by the get() method.

Example

```
# !/usr/bin/python3
from tkinter import *
root = Tk()
scrollbar = Scrollbar(root)
scrollbar.pack( side = RIGHT, fill=Y )
mylist = Listbox(root, yscrollcommand = scrollbar.set )
for line in range(100):
    mylist.insert(END, "This is line number " + str(line))
mylist.pack( side = LEFT, fill = BOTH )
scrollbar.config( command = mylist.yview )
mainloop()
```

Tkinter Text

Text widgets provide advanced capabilities that allow you to edit a multiline text and format the way it has to be displayed, such as changing its color and font. You can also use elegant structures like tabs and marks to locate specific sections of the text, and apply changes to those areas. Moreover, you can embed windows and images in the text because this widget was designed to handle both plain and formatted text.

```
w = Text ( master, option, ... )
```

Parameters

- master: This represents the parent window.
- options: Here is the list of most commonly used options for this widget. These options can be used as key-value pairs separated by commas.

Options	Description
bg	The default background color of the text widget.
bd	The width of the border around the text widget. Default is 2 pixels.
cursor	The cursor that will appear when the mouse is over the text widget.
exportselection	Normally, text selected within a text widget is exported to be the selection in the window manager. Set exportselection=0 if you don't want that behavior.
font	The default font for text inserted into the widget.

fg	The color used for text (and bitmaps) within the widget. You can change the color for tagged regions; this option is just the default.
height	The height of the widget in lines (not pixels!), measured according to the current font size.
highlightbackground	The color of the focus highlight when the text widget does not have focus.
highlightcolor	The color of the focus highlight when the text widget has the focus.
highlightthickness	The thickness of the focus highlight. Default is 1. Set highlightthickness=0 to suppress display of the focus highlight.
insertbackground	The color of the insertion cursor. Default is black.
insertborderwidth	Size of the 3-D border around the insertion cursor. Default is 0.
insertofftime	The number of milliseconds the insertion cursor is off during its blink cycle. Set this option to zero to suppress blinking. Default is 300.
insertontime	The number of milliseconds the insertion cursor is on during its blink cycle. Default is 600.
insertwidth	Width of the insertion cursor (its height is determined by the tallest item in its line). Default is 2 pixels.
padx	The size of the internal padding added to the left and right of the text area. Default is one pixel.
pady	The size of the internal padding added above and below the text area. Default is one pixel.
relief	The 3-D appearance of the text widget. Default is relief=SUNKEN.
selectbackground	The background color to use displaying selected text.
selectborderwidth	The width of the border to use around selected text.
spacing1	This option specifies how much extra vertical space is put above each line of text. If a line wraps, this space is added only before the first line it occupies on the display. Default is 0.
spacing2	This option specifies how much extra vertical space to add between displayed lines of text when a logical line wraps. Default is 0.
spacing3	This option specifies how much extra vertical space is added below each line of text. If a line wraps, this space is added only after the last line it occupies on the display. Default is 0.
state	Normally, text widgets respond to keyboard and mouse events; set state=NORMAL to get this behavior. If you set state=DISABLED, the text widget will not respond, and you won't be able to modify its contents programmatically either.
tabs	This option controls how tab characters position text.
width	The width of the widget in characters (not pixels!), measured according to the current font size.
wrap	This option controls the display of lines that are too wide. Set wrap=WORD and it will break the line after the last word that will fit. With the default behavior, wrap=CHAR, any line that gets too long will be broken at any character.
xscrollcommand	To make the text widget horizontally scrollable, set this option to the set() method of the horizontal scrollbar.
yscrollcommand	To make the text widget vertically scrollable, set this option to the set() method of the vertical scrollbar.

Methods

Methods	Description
delete(startindex [,endindex])	This method deletes a specific character or a range of text.
get(startindex [,endindex])	This method returns a specific character or a range of text.
index(index)	Returns the absolute value of an index based on the given index.
insert(index [,string]...)	This method inserts strings at the specified index location.
see(index)	This method returns true if the text located at the index position is visible.

Text widgets support three distinct helper structures: **Marks**, **Tabs**, and **Indexes**-

Marks are used to bookmark positions between two characters within a given text. We have the following methods available when handling marks:

Methods	Description
index(mark)	Returns the line and column location of a specific mark.
mark_gravity(mark [,gravity])	Returns the gravity of the given mark. If the second argument is provided, the gravity is set for the given mark.
mark_names()	Returns all marks from the Text widget.
mark_set(mark, index)	Informs a new position to the given mark.
mark_unset(mark)	Removes the given mark from the Text widget.

Tags are used to associate names to regions of text which makes easy the task of modifying the display settings of specific text areas. Tags are also used to bind event callbacks to specific ranges of text. Following are the available methods for handling tabs-

Methods	Description
tag_add(tagname, startindex[,endindex] ...)	This method tags either the position defined by startindex, or a range delimited by the positions startindex and endindex.
tag_config	You can use this method to configure the tag properties, which include, justify(center, left, or right), tabs(this property has the same functionality of the Text widget tabs's property), and underline(used to underline the tagged text).
tag_delete(tagname)	This method is used to delete and remove a given tag.
tag_remove(tagname [,startindex[,endindex]] ...)	After applying this method, the given tag is removed from the provided area without deleting the actual tag definition.

Example

```
# !/usr/bin/python3
from tkinter import *
root = Tk()
text = Text(root)
text.insert(INSERT, "Hello.....")
text.insert(END, "Bye Bye.....")
text.pack()
text.tag_add("here", "1.0", "1.4")
text.tag_add("start", "1.8", "1.13")
text.tag_config("here", background="yellow", foreground="blue")
text.tag_config("start", background="black", foreground="green")
root.mainloop()
```

Tkinter Toplevel

Toplevel widgets work as windows that are directly managed by the window manager. They do not necessarily have a parent widget on top of them. Your application can use any number of top-level windows.

```
w = Toplevel ( option, ... )
```

Parameters

- options: Here is the list of most commonly used options for this widget. These options can be used as key-value pairs separated by commas.

Options	Description
bg	The background color of the window.
bd	Border width in pixels; default is 0.
cursor	The cursor that appears when the mouse is in this window.
class_	Normally, text selected within a text widget is exported to be the selection in the window manager. Set exportselection=0 if you don't want that behavior.
font	The default font for text inserted into the widget.
fg	The color used for text (and bitmaps) within the widget. You can change the color for tagged regions; this option is just the default.
height	Window height.
relief	Normally, a top-level window will have no 3-d borders around it. To get a shaded border, set the bd option larger than its default value of zero, and set the relief option to one of the constants.
width	The desired width of the window.

Methods

Methods	Description
deiconify()	Displays the window, after using either the iconify or the withdraw methods.
frame()	Returns a system-specific window identifier.
group(window)	Adds the window to the window group administered by the given window.
iconify()	Turns the window into an icon, without destroying it.
protocol(name, function)	Registers a function as a callback which will be called for the given protocol.
state()	Returns the current state of the window. Possible values are normal, iconic, withdrawn and icon.
transient([master])	Turns the window into a temporary(transient) window for the given master or to the window's parent, when no argument is given.
withdraw()	Removes the window from the screen, without destroying it.
maxsize(width, height)	Defines the maximum size for this window.
minsize(width, height)	Defines the minimum size for this window.
positionfrom(who)	Defines the position controller.
resizable(width, height)	Defines the resize flags, which control whether the window can be resized.
sizefrom(who)	Defines the size controller.
title(string)	Defines the window title.

Example

```
# !/usr/bin/python3
from tkinter import *
root = Tk()
root.title("hello")
top = Toplevel()
top.title("Python")
top.mainloop()
```

Tkinter Spinbox

The Spinbox widget is a variant of the standard Tkinter Entry widget, which can be used to select from a fixed number of values.

w = Spinbox(master, option, ...)

Parameters

- master: This represents the parent window.
- options: Here is the list of most commonly used options for this widget. These options can be used as key-value pairs separated by commas.

Options

activebackground

bg

bd

command

cursor

disabledbackground

disabledforeground

fg

font

format

from_

justify

relief

repeatdelay

repeatinterval

state

textvariable

to

validate

validatecommand

values

vcmd

width

wrap

xscrollcommand

Description

The color of the slider and arrowheads when the mouse is over them.

The color of the slider and arrowheads when the mouse is not over them.

The width of the 3-d borders around the entire perimeter of the trough, and also the width of the 3-d effects on the arrowheads and slider. Default is no border around the trough, and a 2-pixel border around the arrowheads and slider.

A procedure to be called whenever the scrollbar is moved.

The cursor that appears when the mouse is over the scrollbar.

The background color to use when the widget is disabled.

The text color to use when the widget is disabled.

Text color.

The font to use in this widget.

Format string. No default value.

The minimum value. Used together with to to limit the spinbox range.

Default is LEFT

Default is SUNKEN.

Together with repeatinterval, this option controls button autorepeat. Both values are given in milliseconds.

See repeatdelay.

One of NORMAL, DISABLED, or "readonly". Default is NORMAL.

No default value.

See from.

Validation mode. Default is NONE.

Validation callback. No default value.

A tuple containing valid values for this widget. Overrides from/to/increment.

Same as validatecommand.

Widget width, in character units. Default is 20.

If true, the up and down buttons will wrap around.

Used to connect a spinbox field to a horizontal scrollbar. This option should be set to the set method of the corresponding scrollbar.

Methods

Methods

delete(startindex [,endindex])

get(startindex [,endindex])

Description

This method deletes a specific character or a range of text.

This method returns a specific character or a range of text.

identify(x, y)	Identifies the widget element at the given location.
index(index)	Returns the absolute value of an index based on the given index.
insert(index [,string]...)	This method inserts strings at the specified index location.
invoke(element)	Invokes a spinbox button.

Example

```
from tkinter import *
master = Tk()
w = Spinbox(master, from_=0, to=10)
w.pack()
mainloop()
```

Tkinter PanedWindow

A PanedWindow is a container widget that may contain any number of panes, arranged horizontally or vertically. Each pane contains one widget and each pair of panes is separated by a moveable (via mouse movements) sash. Moving a sash causes the widgets on either side of the sash to be resized.

```
w = PanedWindow( master, option, ... )
```

Parameters

- master: This represents the parent window.
- options: Here is the list of most commonly used options for this widget. These options can be used as key-value pairs separated by commas.

Option	Description
bg	The color of the slider and arrowheads when the mouse is not over them.
bd	The width of the 3-d borders around the entire perimeter of the trough, and also the width of the 3-d effects on the arrowheads and slider. Default is no border around the trough, and a 2-pixel border around the arrowheads and slider.
borderwidth	Default is 2.
cursor	The cursor that appears when the mouse is over the window.
handlepad	Default is 8.
handlesize	Default is 8.
height	No default value.
orient	Default is HORIZONTAL.
relief	Default is FLAT.
sashcursor	No default value.
sashrelief	Default is RAISED.
sashwidth	Default is 2.
showhandle	No default value
width	No default value.

Methods

Methods	Description
add(child, options)	Adds a child window to the paned window.
get(startindex [,endindex])	This method returns a specific character or a range of text.
config(options)	Modifies one or more widget options. If no options are given, the method returns a

dictionary containing all current option values.

Example

```
#create a 3-pane widget-
# !/usr/bin/python3
from tkinter import *
m1 = PanedWindow()
m1.pack(fill=BOTH, expand=1)
left = Entry(m1, bd=5)
m1.add(left)
m2 = PanedWindow(m1, orient=VERTICAL)
m1.add(m2)
top = Scale( m2, orient=HORIZONTAL)
m2.add(top)
bottom = Button(m2, text="OK")
m2.add(bottom)
mainloop()
```

Tkinter LabelFrame

A labelframe is a simple container widget. Its primary purpose is to act as a spacer or container for complex window layouts. This widget has the features of a frame plus the ability to display a label.

```
w = LabelFrame( master, option, ... )
```

Parameters

- master: This represents the parent window.
- options: Here is the list of most commonly used options for this widget. These options can be used as key-value pairs separated by commas.

Option	Description
bg	The normal background color displayed behind the label and indicator.
bd	The size of the border around the indicator. Default is 2 pixels.
cursor	If you set this option to a cursor name (arrow, dot etc.), the mouse cursor will change to that pattern when it is over the checkbutton.
font	The vertical dimension of the new frame.
height	The vertical dimension of the new frame.
labelAnchor	Specifies where to place the label.
highlightbackground	Color of the focus highlight when the frame does not have focus.
highlightcolor	Color shown in the focus highlight when the frame has the focus.
highlightthickness	Thickness of the focus highlight.
relief	With the default value, relief=FLAT, the checkbutton does not stand out from its background. You may set this option to any of the other styles
text	Specifies a string to be displayed inside the widget.
width	Specifies the desired width for the window.

Example

```
# !/usr/bin/python3
from tkinter import *
root = Tk()
labelframe = LabelFrame(root, text="This is a LabelFrame")
labelframe.pack(fill="both", expand="yes")
left = Label(labelframe, text="Inside the LabelFrame")
left.pack()
root.mainloop()
```

Tkinter tkMessageBox

The tkMessageBox module is used to display message boxes in your applications. This module provides a number of functions that you can use to display an appropriate message. Some of these functions are showinfo, showwarning, showerror, askquestion, askokcancel, askyesno, and askretryignore.

```
tkMessageBox.FunctionName(title, message [, options])
```

Parameters

- **FunctionName:** This is the name of the appropriate message box function.
- **title:** This is the text to be displayed in the title bar of a message box.
- **message:** This is the text to be displayed as a message.
- **options:** options are alternative choices that you may use to tailor a standard message box. Some of the options that you can use are default and parent. The default option is used to specify the default button, such as ABORT, RETRY, or IGNORE in the message box. The parent option is used to specify the window on top of which the message box is to be displayed.

You could use one of the following functions with dialogue box-

- showinfo()
- showwarning()
- showerror()
- askquestion()
- askokcancel()
- askyesno()
- askretrycancel()

Example

```
# !/usr/bin/python3
from tkinter import *
from tkinter import messagebox
top = Tk()
top.geometry("100x100")
def hello():
    messagebox.showinfo("Say Hello", "Hello World")
B1 = Button(top, text = "Say Hello", command = hello)
B1.place(x=35,y=50)
```

Standard Attributes

Let us look at how some of the common attributes, such as sizes, colors and fonts are specified.

Tkinter Dimensions

Various lengths, widths, and other dimensions of widgets can be described in many different units.

- ✓ If you set a dimension to an integer, it is assumed to be in pixels.
- ✓ You can specify units by setting a dimension to a string containing a number followed by.

Character	Description
c	Centimeters
i	Inches
m	Millimeters
p	Printer's points (about 1/72")

Length options

Tkinter expresses a length as an integer number of pixels. Here is the list of common length options-

- `borderwidth`: Width of the border which gives a three-dimensional look to the widget.
- `highlightthickness`: Width of the highlight rectangle when the widget has focus .
- `padX padY`: Extra space the widget requests from its layout manager beyond the minimum the widget needs to display its contents in the x and y directions.
- `selectborderwidth`: Width of the three-dimensional border around selected items of the widget.
- `wrlength`: Maximum line length for widgets that perform word wrapping.
- `height`: Desired height of the widget; must be greater than or equal to 1.
- `underline`: Index of the character to underline in the widget's text (0 is the first character, 1 the second one, and so on).
- `width`: Desired width of the widget.

Tkinter Colors

Tkinter represents colors with strings. There are two general ways to specify colors in Tkinter-

- ✓ You can use a string specifying the proportion of red, green and blue in hexadecimal digits. For example, "#fff" is white, "#000000" is black, "#000fff00" is pure green, and "#00ffff" is pure cyan (green plus blue).
- ✓ You can also use any locally defined standard color name. The colors "white", "black", "red", "green", "blue", "cyan", "yellow", and "magenta" will always be available.

Color options

The common color options are-

- `activebackground`: Background color for the widget when the widget is active.
- `activeforeground`: Foreground color for the widget when the widget is active.
- `background`: Background color for the widget. This can also be represented as `bg`.
- `disabledforeground`: Foreground color for the widget when the widget is disabled.
- `foreground`: Foreground color for the widget. This can also be represented as `fg`.
- `highlightbackground`: Background color of the highlight region when the widget has focus.
- `highlightcolor`: Foreground color of the highlight region when the widget has focus.

- selectbackground: Background color for the selected items of the widget.
- selectforeground: Foreground color for the selected items of the widget.

Tkinter Fonts

Simple Tuple Fonts

As a tuple whose first element is the font family, followed by a size in points, optionally followed by a string containing one or more of the style modifiers bold, italic, underline and overstrike.

Example

- ("Helvetica", "16") for a 16-point Helvetica regular.
- ("Times", "24", "bold italic") for a 24-point Times bold italic.

Font object Fonts

You can create a "font object" by importing the tkFont module and using its Font class constructor –

```
import tkFont
font = tkFont.Font ( option, ... )
```

Here is the list of options

- family: The font family name as a string.
- size: The font height as an integer in points. To get a font n pixels high, use -n.
- weight: "bold" for boldface, "normal" for regular weight.
- slant: "italic" for italic, "roman" for unslanted.
- underline: 1 for underlined text, 0 for normal.
- overstrike: 1 for overstruck text, 0 for normal.

Example

```
helv36 = tkFont.Font(family="Helvetica",size=36,weight="bold")
```

Tkinter Anchors

Anchors are used to define where text is positioned relative to a reference point. Here is list of possible constants, which can be used for Anchor attribute.

- | | | |
|------|----------|------|
| ▪ NW | ▪ W | ▪ SW |
| ▪ N | ▪ CENTER | ▪ S |
| ▪ NE | ▪ E | ▪ SE |

For example, if you use CENTER as a text anchor, the text will be centered horizontally and vertically around the reference point.

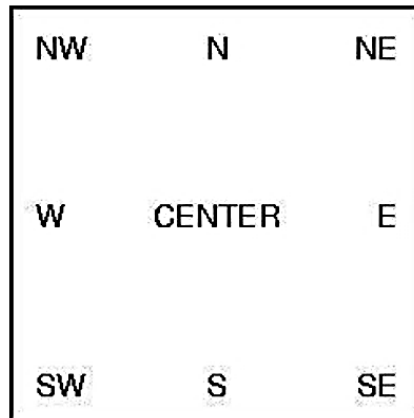
Anchor NW will position the text so that the reference point coincides with the northwest (top left) corner of the box containing the text.

Anchor W will center the text vertically around the reference point, with the left edge of the text box passing through that point, and so on.

If you create a small widget inside a large frame and use the anchor=SE option, the widget will be placed in the bottom right corner of the frame. If you used anchor=N instead, the widget would be centered along the top edge.

Example

The anchor constants are shown in this diagram-



Tkinter Relief styles

The relief style of a widget refers to certain simulated 3-D effects around the outside of the widget. Here is a screenshot of a row of buttons exhibiting all the possible relief styles.

Here is list of possible constants which can be used for relief attribute-

- FLAT
- RAISED
- SUNKEN
- GROOVE
- RIDGE

Example

```
#!/usr/bin/python3
from tkinter import *
import tkinter
top = Tk()
B1 = Button(top, text="FLAT", relief=FLAT )
B2 = Button(top, text="RAISED", relief=RAISED )
B3 = Button(top, text="SUNKEN", relief=SUNKEN )
B4 = Button(top, text="GROOVE", relief=GROOVE )
B5 = Button(top, text="RIDGE", relief=RIDGE )
B1.pack()
B2.pack()
B3.pack()
B4.pack()
B5.pack()
top.mainloop()
```

Geometry Management

All Tkinter widgets have access to the specific geometry management methods, which have the purpose of organizing widgets throughout the parent widget area. Tkinter exposes the following geometry manager classes: pack, grid, and place.

Tkinter pack() Method

This geometry manager organizes widgets in blocks before placing them in the parent widget.

```
widget.pack( pack_options )
```

Here is the list of possible options-

- **expand**: When set to true, widget expands to fill any space not otherwise used in widget's parent.
- **fill**: Determines whether widget fills any extra space allocated to it by the packer, or keeps its own minimal dimensions: NONE (default), X (fill only horizontally), Y (fill only vertically), or BOTH (fill both horizontally and vertically).
- **side**: Determines which side of the parent widget packs against: TOP (default), BOTTOM, LEFT, or RIGHT.

Example

Try the following example by moving cursor on different buttons-

```
# !/usr/bin/python3
from tkinter import *
root = Tk()
frame = Frame(root)
frame.pack()
bottomframe = Frame(root)
bottomframe.pack( side = BOTTOM )
redbutton = Button(frame, text="Red", fg="red")
redbutton.pack( side = LEFT)
greenbutton = Button(frame, text="Brown", fg="brown")
greenbutton.pack( side = LEFT )
bluebutton = Button(frame, text="Blue", fg="blue")
bluebutton.pack( side = LEFT )
blackbutton = Button(bottomframe, text="Black", fg="black")
blackbutton.pack( side = BOTTOM)
root.mainloop()
```

Tkinter grid() Method

This geometry manager organizes widgets in a table-like structure in the parent widget.

```
widget.grid( grid_options )
```

Here is the list of possible options-

- **column** : The column to put widget in; default 0 (leftmost column).
- **columnspan**: How many columns widget occupies; default 1.
- **ipadx, ipady** :How many pixels to pad widget, horizontally and vertically, inside widget's borders.
- **padx, pady** : How many pixels to pad widget, horizontally and vertically, outside v's borders.

- row: The row to put widget in; default the first row that is still empty.
- rowspan : How many rows widget occupies; default 1.
- sticky : What to do if the cell is larger than widget. By default, with sticky="", widget is centered in its cell. sticky may be the string concatenation of zero or more of N, E, S, W, NE, NW, SE, and SW, compass directions indicating the sides and corners of the cell to which widget sticks.

Example

```
# !/usr/bin/python3
from tkinter import *
root = Tk()
b=0
for r in range(6):
    for c in range(6):
        b=b+1
        Button(root, text=str(b),
        borderwidth=1).grid(row=r,column=c)
root.mainloop()
```

Tkinter place() Method

This geometry manager organizes widgets by placing them in a specific position in the parent widget.

```
widget.place( place_options )
```

Here is the list of possible options-

- anchor : The exact spot of widget other options refer to: may be N, E, S, W, NE, NW, SE, or SW, compass directions indicating the corners and sides of widget; default is NW (the upper left corner of widget)
- bordermode : INSIDE (the default) to indicate that other options refer to the parent's inside (ignoring the parent's border); OUTSIDE otherwise.
- height, width : Height and width in pixels.
- relheight, relwidth : Height and width as a float between 0.0 and 1.0, as a fraction of the height and width of the parent widget.
- relx, rely : Horizontal and vertical offset as a float between 0.0 and 1.0, as a fraction of the height and width of the parent widget.
- x, y : Horizontal and vertical offset in pixels.

Example

Try the following example by moving cursor on different buttons-

```
# !/usr/bin/python3
from tkinter import *
top = Tk()
L1 = Label(top, text="Physics")
L1.place(x=10,y=10)
E1 = Entry(top, bd =5)
E1.place(x=60,y=10)
L2=Label(top,text="Maths")
L2.place(x=10,y=50)
```

```
E2=Entry(top,bd=5)
E2.place(x=60,y=50)
L3=Label(top,text="Total")
L3.place(x=10,y=150)
E3=Entry(top,bd=5)
E3.place(x=60,y=150)
B = Button(top, text ="Add")
B.place(x=100, y=100)
top.geometry("250x250+10+10")
top.mainloop()
```

end of guide